

Spring + Hibernate + REST

Java Interview Prep — 2026 Edition · Learn & Excel

If your role is backend Java — and most are — this file matters more than any other in the pack. 160 questions across seven sections: Spring Core, Spring Boot, Spring MVC + REST, Spring Security, Spring Data + JPA, Hibernate, REST Design. Targeted at Spring Boot 3.x / Spring 6 / Jakarta EE 9+, which is what almost every team uses in 2026.

Same per-Q format as **01 Top 50** : question · asked-at · 30-second answer · step-by-step thought process · edge cases · what NOT to say · common follow-up.

Section 1 — Spring Core (30 Qs)

The foundations. If you don't have these solid, none of the layers above make sense.

Q1 · What is dependency injection, and why does Spring care?

Asked at: Most Spring interviews · **Difficulty:** Easy · **Topic:** Spring Core

The question: "Explain dependency injection in your own words. What problem does it solve?"

The 30-second answer: DI is a design pattern where objects receive their dependencies from outside instead of constructing them. Spring's IoC container does this for you — you declare what a bean needs, Spring wires it. Benefits: testability (swap real deps for mocks), loose coupling, single responsibility.

Step-by-step thought process: 1. "Without DI, a class creates its own dependencies — `UserService` does `new UserRepository()` . That hardcodes the implementation; hard to test, hard to swap." 2. "With DI, the caller (or framework) hands the dependency in: `UserService(UserRepository repo)` . The class doesn't know or care how it's constructed." 3. "Spring's IoC container reads bean definitions, instantiates them in dependency order, and wires them together." 4. "In practice: annotate with `@Service` / `@Component` , declare deps via constructor, Spring does the rest."

Edge cases to mention: - Constructor injection is the recommended style (vs field injection) — makes deps explicit, supports immutability, easier to test - Setter injection is for genuinely optional dependencies - Circular dependencies via constructor injection FAIL at startup — that's a feature, it forces you to fix the design

What NOT to say: - "DI is just passing parameters" — misses the framework's wiring role - "Use @Autowired on fields, it's cleaner" — actively bad advice in modern Spring

Common follow-up: *"Why is constructor injection preferred over field injection?"*

Q2 · @Component vs @Service vs @Repository vs @Controller

Asked at: Common at all levels · **Difficulty:** Easy · **Topic:** Spring stereotypes

The question: *"What's the difference between @Component, @Service, @Repository, and @Controller?"*

The 30-second answer: All are stereotype annotations — they mark a class as a Spring-managed bean and let component scanning find it. Behaviorally, `@Service` and `@Component` are equivalent.

`@Repository` adds automatic exception translation (JDBC/JPA exceptions → `DataAccessException`).

`@Controller` marks an MVC controller; Spring MVC scans for handler methods.

Step-by-step thought process: 1. "`@Component` is the base — anything annotated is a candidate bean. `@Service`, `@Repository`, `@Controller`, `@RestController` are all meta-annotated with `@Component`." 2. "Semantically they communicate intent — readers see `@Service` and know it's business logic, `@Repository` and know it's data access." 3. "`@Repository` is the only one with EXTRA behavior: Spring registers `PersistenceExceptionTranslationPostProcessor`, which wraps thrown JDBC/JPA exceptions in Spring's `DataAccessException` hierarchy. Cleaner exception handling at service layer." 4. "`@Controller` is for Spring MVC; `@RestController` is `@Controller` + `@ResponseBody` on every method (returns response body, not view name)."

Edge cases to mention: - You can define your own stereotypes (meta-annotate with `@Component`) for domain-specific tagging - Component scanning is configured via `@ComponentScan`; default scans the package of the `@SpringBootApplication` class and below - Stereotypes work with `@Configuration` classes too — `@Configuration` is also a `@Component`

What NOT to say: - "All four are exactly the same" — wrong, `@Repository` has exception translation - "Use `@Service` for any bean" — works but loses the intent signal

Common follow-up: *"What does PersistenceExceptionTranslationPostProcessor actually do?"*

Q3 · Bean lifecycle — what happens between definition and disposal?

Asked at: Mid-level + senior Spring rounds · **Difficulty:** Medium · **Topic:** Lifecycle

The question: *"Walk me through the Spring bean lifecycle — from container startup to shutdown."*

The 30-second answer: Container reads bean definitions → instantiates → injects dependencies → calls `BeanNameAware` / `BeanFactoryAware` / `ApplicationContextAware` if implemented → `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct` /

`InitializingBean.afterPropertiesSet()` / custom init-method →
`BeanPostProcessor.postProcessAfterInitialization` → bean is live. On shutdown:
`@PreDestroy` / `DisposableBean.destroy()` / custom destroy-method.

Step-by-step thought process: 1. "Container instantiation: Spring calls the constructor with resolved dependencies." 2. "Aware callbacks: if your bean implements `BeanNameAware`, Spring tells it its name; same for `BeanFactoryAware`, `ApplicationContextAware`. Lets the bean grab container context." 3. "BeanPostProcessor `postProcessBeforeInitialization`: runs across every bean. AOP proxies are often installed here." 4. "Initialization: `@PostConstruct` method, then `afterPropertiesSet()` if `InitializingBean`, then custom init-method. Multiple init hooks but they're called in this order." 5. "BeanPostProcessor `postProcessAfterInitialization`: more decoration possible. Returns the final bean reference (could even be a proxy)." 6. "On shutdown: `@PreDestroy`, then `destroy()` if `DisposableBean`, then custom destroy-method. Reverse of init order."

Edge cases to mention: - Prototype-scoped beans don't get destroy callbacks — Spring stops tracking them after creation - `@PostConstruct` is JSR-250, not Spring-specific — works in plain Jakarta EE too - `SmartLifecycle` is for beans that need start/stop semantics tied to `ApplicationContext` lifecycle, useful for things like message consumers

What NOT to say: - "Spring just calls the constructor and you're done" — misses init/destroy and post-processors - "Use `@PostConstruct` AND `InitializingBean` for safety" — both fire; you only need one

Common follow-up: "Where would you write code that needs to run after all beans are wired but before the app is ready?"

Q4 · Bean scopes — singleton vs prototype vs request vs session

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Scopes

The question: "What scopes does Spring support, and when would you use each?"

The 30-second answer: `singleton` (default — one instance per container), `prototype` (new instance per injection / `getBean()` call), `request` / `session` / `application` (web-aware scopes — one per HTTP request / HTTP session / `ServletContext`). 95% of beans should stay singleton; reach for prototype only when you really need fresh state per use.

Step-by-step thought process: 1. "Default scope is `singleton` — Spring creates one instance, every injection point gets the same reference. State is shared across all callers, so beans should be stateless." 2. " `prototype` — Spring creates a new instance every time the bean is requested (via `getBean()` or injected as a dependency, with caveats — see below)." 3. "Web scopes (require web context): `request` (per HTTP request), `session` (per HTTP session), `application` (per `ServletContext`)." 4. "Custom scopes possible — register a `Scope` implementation, useful for things like 'per-tenant' beans in multi-tenant apps."

Edge cases to mention: - Singleton injecting prototype is a trap — the singleton holds ONE prototype reference forever. Use `@Lookup` method or `ObjectFactory<T>` / `Provider<T>` to get a fresh prototype on each call - `@RequestScope` and `@SessionScope` are convenience composed annotations (Spring 4.3+) - Prototype beans don't get destroy callbacks — manage cleanup yourself

What NOT to say: - "Use prototype for thread safety" — doesn't help; instead make singletons thread-safe - "Singleton means one per JVM" — wrong, it's one per container

Common follow-up: *"If a singleton needs a prototype dependency, how do you wire that correctly?"*

Q5 · Constructor vs setter vs field injection

Asked at: Common code-review style question · **Difficulty:** Easy-Medium · **Topic:** Injection styles

The question: *"What's the difference between constructor, setter, and field injection? Which should you prefer in 2026?"*

The 30-second answer: Constructor = inject via constructor parameters. Setter = inject via setter methods. Field = `@Autowired` on a field, no setter or constructor. **Constructor injection is preferred** — makes dependencies explicit, enforces immutability (final fields), works without Spring (testable), fails fast on missing deps.

Step-by-step thought process: 1. *"Constructor injection: dependencies declared in the constructor signature. Spring 4.3+ auto-injects if there's a single constructor — no annotation needed. Fields can be `final`."* 2. *"Setter injection: setter methods for each dependency, annotated with `@Autowired`. Useful for optional dependencies or when you need to reconfigure at runtime."* 3. *"Field injection: `@Autowired` directly on a field. Spring uses reflection to set it. Tempting because it's terse — but considered an anti-pattern."* 4. *"Why field is bad: invisible deps (no constructor signature), can't make field final, requires Spring (or a mocking framework with reflection) to test, encourages large unfocused classes."* 5. *"Modern preference: constructor injection everywhere. If the class is becoming hard to construct, that's a code-smell — split the class."*

Edge cases to mention: - Spring 4.3+: single constructor doesn't need `@Autowired`. Lombok's `@RequiredArgsConstructor` pairs with this for boilerplate-free constructor injection - Setter injection still valid for genuinely optional deps (e.g. an optional metrics recorder) - Avoid mixing styles within a single class — confusing

What NOT to say: - "Field injection is fine, it's less code" — code smell, will get downvoted in code review - "Constructor injection causes circular dependency issues" — that's the POINT; circular deps are a design problem

Common follow-up: *"How do you handle optional dependencies with constructor injection?"*

Q6 · Circular dependencies — what happens and how do you fix them?

Asked at: Mid-level Spring rounds · **Difficulty:** Medium · **Topic:** DI internals

The question: "What happens when two beans depend on each other? How does Spring handle it, and what should YOU do about it?"

The 30-second answer: With setter or field injection, Spring resolves circular deps using a three-stage cache (singleton beans). With constructor injection, you get `BeanCurrentlyInCreationException` at startup. The "fix" is almost always to refactor — circular deps usually signal that you should extract a third bean or use an event.

Step-by-step thought process: 1. "Spring's three-stage cache for singletons: `singletonObjects` (fully initialized), `earlySingletonObjects` (partially constructed), `singletonFactories` (factory references). Lets two beans complete construction even when they reference each other (via setters/fields)." 2. "Constructor injection breaks this — both beans need each other `CONSTRUCTED` before they can be constructed. Spring detects the cycle and throws `BeanCurrentlyInCreationException`." 3. "The 'lazy' workaround: annotate one side with `@Lazy`. Spring injects a proxy that resolves the real bean on first use. Works but masks a design problem." 4. "Proper fix: extract a third bean both depend on, OR use Spring's `ApplicationEventPublisher` to decouple, OR split responsibilities so the cycle is unnecessary." 5. "Spring Boot 2.6+ disables circular dependency resolution by default — you get the error even for setter injection unless you explicitly re-enable it (`spring.main.allow-circular-references=true`)."

Edge cases to mention: - Prototype beans + circular dep = always an error; no three-stage cache for prototypes - `@Lazy` is a code smell — use sparingly - Self-referencing beans (a bean injecting itself) usually mean you should use `@Lookup` or restructure

What NOT to say: - "Just set allow-circular-references to true" — defers the problem, doesn't solve it - "Circular deps are fine" — they aren't; they signal coupling

Common follow-up: "Show me how an event-based refactor would remove a circular dependency."

Q7 · ApplicationContext vs BeanFactory

Asked at: Senior rounds · **Difficulty:** Medium · **Topic:** Container internals

The question: "What's the difference between `BeanFactory` and `ApplicationContext`? When does it matter?"

The 30-second answer: `BeanFactory` is the basic IoC container — lazy, minimal. `ApplicationContext` extends it with enterprise features: event publication, internationalization, AOP support, environment abstraction, resource loading. In Spring Boot you always work with `ApplicationContext`; `BeanFactory` matters mostly for understanding internals.

Step-by-step thought process: 1. " `BeanFactory` is the core interface — `getBean(name/class)`, `containsBean`, etc. Defaults to lazy initialization." 2. " `ApplicationContext` extends

BeanFactory and adds: `ApplicationEventPublisher` (publish/subscribe events), `MessageSource` (i18n), `ResourceLoader` (load classpath / file / URL resources), `Environment` (profiles + properties)." 3. " `ApplicationContext` eagerly initializes singletons at startup by default — catches config errors immediately rather than on first use." 4. "In modern Spring you almost never deal with BeanFactory directly. `ApplicationContext` (specifically `ConfigurableApplicationContext` and `AnnotationConfigApplicationContext` in tests) is what you'll inject and use."

Edge cases to mention: - For tooling / programmatic bean inspection, you sometimes downcast `ApplicationContext` → `ConfigurableListableBeanFactory` - BeanFactory has historical relevance — it was the original Spring container before `ApplicationContext` layered features on top - `WebApplicationContext` is a further extension for web contexts (adds `ServletContext` access)

What NOT to say: - "They're the same thing" — wrong; `ApplicationContext` is a superset - "Use BeanFactory for performance" — micro-optimization, rarely matters; `ApplicationContext` is the right default

Common follow-up: "How does `ApplicationContext` support events? Show me an example."

Q8 · @Configuration vs @Component for defining beans

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Configuration

The question: "What's the difference between using `@Configuration` with `@Bean` methods vs `@Component` for defining beans?"

The 30-second answer: `@Component` (and stereotypes) means "this CLASS is a bean — please instantiate me." `@Configuration` means "this class CONFIGURES beans — call its `@Bean` methods to produce beans." Configuration classes also get a CGLIB proxy so multiple calls to the same `@Bean` method return the same singleton (avoids accidental duplicates).

Step-by-step thought process: 1. " `@Component` -annotated classes are auto-discovered via component scanning. The class itself becomes a managed bean." 2. " `@Configuration` -annotated classes are ALSO managed beans, but their job is to define other beans via `@Bean` methods. Use them for third-party class wiring, or when the constructor needs custom logic." 3. "Spring proxies `@Configuration` classes with CGLIB. When one `@Bean` method calls another within the same class, the proxy intercepts and returns the cached singleton instead of executing the method twice." 4. " `@Configuration(proxyBeanMethods = false)` (lite mode) skips the proxy — faster startup, but inter-method calls don't share state. Useful when methods don't call each other or when you want native-image friendliness." 5. "Choose `@Component` for your own service classes. Choose `@Configuration` for grouping bean factory methods (e.g. wiring a third-party `RestTemplate` + interceptors)."

Edge cases to mention: - AOT compilation (Spring 6) and native images prefer

`proxyBeanMethods=false` configurations - Configuration classes can themselves be injected — useful for fetching grouped beans - `@Import` lets you compose multiple configuration classes

What NOT to say: - "@Component and @Configuration are interchangeable" — they aren't, especially around `@Bean` method semantics - "Always use @Configuration for everything" — overkill for plain service classes

Common follow-up: *"What does proxyBeanMethods=false actually change?"*

Q9 · @Autowired — what does it actually do?

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Injection

The question: *"Walk me through what @Autowired actually does at runtime."*

The 30-second answer: `@Autowired` marks an injection point. At bean construction time, Spring's `AutowiredAnnotationBeanPostProcessor` looks at each `@Autowired` field/setter/constructor parameter, finds matching beans by type (then by name if multiple match), and injects. Constructor injection no longer needs `@Autowired` since Spring 4.3+ if there's a single constructor.

Step-by-step thought process: 1. *"During bean post-processing, `AutowiredAnnotationBeanPostProcessor` scans the bean class for `@Autowired` annotations on fields, setters, constructors."* 2. *"For each injection point, Spring resolves a matching bean: primary match by TYPE. If multiple beans match, narrow by NAME (matching the field/parameter name)."* 3. *"If no match, throws `NoSuchBeanDefinitionException` unless `@Autowired(required = false)` — then leaves the field null."* 4. *"Multiple matches with no clear winner → `NoUniqueBeanDefinitionException`. Resolve with `@Primary` (one preferred bean), `@Qualifier("name")` (specific name), or `@Order` (preference ranking)."* 5. *"Spring 4.3+ removed the need for `@Autowired` on single-constructor classes — implicit constructor injection. Less noise in your code."*

Edge cases to mention: - Injecting `List<MyInterface>` gets ALL beans of that type — useful for plugin patterns - Injecting `Map<String, MyInterface>` gets a map of bean name → bean — useful for strategy patterns - `@Inject` (JSR-330) is mostly equivalent to `@Autowired` — Spring supports both

What NOT to say: - "@Autowired is required on every dependency" — Spring 4.3+ removed that need for single constructors - "It uses reflection slowly" — Spring caches the resolution after first init; runtime cost is negligible

Common follow-up: *"How would you inject all implementations of an interface, or pick one by qualifier?"*

Q10 · @Qualifier and @Primary — disambiguating beans

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Injection disambiguation

The question: "You have two beans implementing the same interface. How does Spring decide which one to inject?"

The 30-second answer: With nothing else: `NoUniqueBeanDefinitionException`. To disambiguate: mark one with `@Primary` (preferred default), or use `@Qualifier("name")` at the injection point to pick a specific one. If neither, Spring falls back to matching by parameter/field name.

Step-by-step thought process: 1. "Two beans of the same type → ambiguous injection. Spring throws `NoUniqueBeanDefinitionException` at startup." 2. " `@Primary` on one bean tells Spring 'this is the default if there's a tie.' All injection points get this one unless they ask for a different one." 3. " `@Qualifier("specificName")` at the injection point picks a bean by name. Useful when you have multiple of the same type and need to be explicit." 4. "Fallback: if no `@Primary` and no `@Qualifier`, Spring tries to match by name — `@Autowired private UserService primaryUserService` would match a bean named `primaryUserService`." 5. "Best practice: prefer `@Primary` for the 'default' implementation, `@Qualifier` for explicit selection in edge cases. Names should be intention-revealing."

Edge cases to mention: - `@Qualifier` can be custom — create your own annotation meta-annotated with `@Qualifier` - For collections (`List<MyInterface>`), `@Order` controls the order of injected beans - Spring Boot's `@ConditionalOnMissingBean` is a common pattern for "use this if user hasn't provided one"

What NOT to say: - "Use `@Resource` for naming" — works (JSR-250) but less common in modern Spring; `@Qualifier` is the idiom - "Multiple `@Primary` is fine" — actually fails at startup with `NoUniqueBeanDefinitionException`

Common follow-up: "How would you have different `DataSource` beans for read replicas vs write masters?"

Q11 · AOP — what is it, how does Spring implement it?

Asked at: Mid-to-senior · **Difficulty:** Medium-Hard · **Topic:** AOP

The question: "What is AOP, and how does Spring's AOP work under the hood?"

The 30-second answer: AOP = Aspect-Oriented Programming. Lets you weave cross-cutting concerns (logging, transactions, security, metrics) into your code without polluting business logic. Spring AOP uses runtime proxies — JDK dynamic proxies for interfaces, CGLIB subclassing for classes. Pointcuts express "where to weave"; advice expresses "what to do."

Step-by-step thought process: 1. "Aspect = a module of cross-cutting behavior (e.g. 'log every controller call'). Implemented as a class annotated with `@Aspect`." 2. "Pointcut = expression matching

method executions ('all public methods in com.foo.controller'). Spring uses AspectJ pointcut syntax." 3. "Advice = code to run at the matched join points. Types: `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`." 4. "Spring weaves at runtime via proxies. For interface-implementing beans, JDK dynamic proxies. For concrete classes, CGLIB subclassing." 5. "Common uses: `@Transactional` (proxied to start/commit/rollback transactions), `@Cacheable` (intercepts to check cache first), `@Async` (offloads to a `TaskExecutor`), custom logging, metrics, retry."

Edge cases to mention: - Self-invocation doesn't trigger AOP — calling `this.method()` bypasses the proxy. To fix: inject self, or refactor - `final` methods can't be CGLIB-proxied (no override possible) - Spring AOP is proxy-based; full AspectJ (compile-time weaving) is more powerful but separate

What NOT to say: - "AOP is magic" — it's just proxies; understandable mechanics - "AOP slows everything down" — overhead is one method indirection per advised call; negligible for most use cases

Common follow-up: "Why doesn't `@Transactional` work when a method calls another method in the same class?"

Q12 · @Transactional — how does it actually work?

Asked at: Almost universally in Spring backend interviews · **Difficulty:** Medium · **Topic:** Transactions + AOP

The question: "What does `@Transactional` actually do? When does it fail silently?"

The 30-second answer: `@Transactional` is AOP advice. The proxy intercepts the method call, opens a transaction (begins / suspends / joins existing depending on propagation), invokes the method, then commits on normal return or rolls back on `RuntimeException`. Silent failure modes: self-invocation (proxy bypassed), checked exceptions (don't roll back by default), private methods (proxy can't intercept).

Step-by-step thought process: 1. "On startup, Spring wraps `@Transactional` beans in a proxy. The proxy intercepts incoming calls — opens a transaction, calls the real method, commits or rolls back." 2. "Rollback rules: by default, rolls back on `RuntimeException` and `Error`. Does NOT roll back on checked exceptions unless you specify `rollbackFor`." 3. "Propagation: `REQUIRED` (default — join existing or create new), `REQUIRES_NEW` (suspend existing, start fresh), `NESTED` (savepoint within existing), `SUPPORTS`, `MANDATORY`, `NEVER`, `NOT_SUPPORTED`." 4. "Isolation level: `DEFAULT` (DB default), `READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`, `READ_UNCOMMITTED`." 5. "Silent failure modes (high frequency interview question): (1) calling a `@Transactional` method from another method in the SAME class — proxy is bypassed, no transaction; (2) `private` methods — proxy can't intercept; (3) checked exception — rolls back only if `rollbackFor` configured."

Edge cases to mention: - For `READ_ONLY` transactions, set `readOnly=true` — Hibernate skips dirty checking, JDBC drivers can optimize - `@Transactional` on interfaces or interface methods — works only with JDK dynamic proxies, NOT CGLIB. Best to put on concrete classes - Transaction

synchronizations: register callbacks for `afterCommit` / `afterCompletion` — useful for post-commit operations (publish events, send notifications)

What NOT to say: - "@Transactional always rolls back on exception" — wrong, only RuntimeException by default - "Just annotate everything @Transactional" — over-broad, hurts performance and complicates testing

Common follow-up: "How would you make a method roll back on a custom checked exception?"

Q13 · Profiles — how to manage environment-specific config

Asked at: Common in Spring Boot rounds · **Difficulty:** Easy-Medium · **Topic:** Configuration

The question: "How do Spring profiles work? Walk me through a typical use case."

The 30-second answer: Profiles let you activate different beans + configs per environment.

`@Profile("dev")` on a class makes it active only in dev. Set the active profile via `spring.profiles.active=dev` (property), env var (`SPRING_PROFILES_ACTIVE`), or CLI. `application-dev.properties` / `application-dev.yml` auto-load for the active profile.

Step-by-step thought process: 1. "A profile is just a tag. Beans tagged `@Profile("dev")` are registered only when 'dev' is active." 2. "Activate a profile via `spring.profiles.active=dev` in properties, env var `SPRING_PROFILES_ACTIVE=dev`, JVM arg `-Dspring.profiles.active=dev`, or CLI `--spring.profiles.active=dev`." 3. "Multiple profiles allowed: `spring.profiles.active=dev,with-mocks` — both activate. Bean tagged `@Profile("dev | qa")` activates if dev OR qa is active." 4. "Property file convention: `application.properties` always loads; `application-{profile}.properties` loads on top when that profile is active. Profile-specific values override base values." 5. "Common pattern: `dev` profile uses an H2 in-memory DB and a mock email service; `prod` uses real Postgres + real SES. Same code, swappable backends."

Edge cases to mention: - `@Profile("default")` — only active if NO profile is set - Spring Boot 2.4+ uses `spring.config.import` for more flexible config layering (e.g. importing from Vault, K8s configmaps) - `@Profile` works on `@Configuration` classes too — useful for grouping all dev-specific wiring

What NOT to say: - "Use system properties to override profile" — confused; profiles control which beans/configs apply - "Profiles are like environment variables" — partial; profiles control bean activation as well

Common follow-up: "How would you handle a profile-specific secret (like a real DB password)?"

Q14 · @ConditionalOn* — Spring Boot's autoconfig superpower

Asked at: Mid-to-senior · **Difficulty:** Medium · **Topic:** Conditional configuration

The question: "What's `@ConditionalOnMissingBean` and how does Spring Boot use it for autoconfiguration?"

The 30-second answer: `@Conditional` annotations gate bean registration.

`@ConditionalOnMissingBean` registers a bean only if no other bean of that type is already in the context. Spring Boot autoconfig uses this heavily — it provides defaults (like a `DataSource` bean) but yields if the user defines their own.

Step-by-step thought process: 1. "`@Conditional(SomeCondition.class)` is the general mechanism — Spring evaluates the condition at config-time, decides whether to register the bean." 2. "Spring Boot provides ready-made variants: `@ConditionalOnClass` (only if class on classpath), `@ConditionalOnBean` (only if another bean exists), `@ConditionalOnMissingBean` (only if absent), `@ConditionalOnProperty` (only if property set), `@ConditionalOnWebApplication`, etc." 3. "The autoconfiguration pattern: define a default bean wrapped in `@ConditionalOnMissingBean(DataSource.class)`. If the user has their own `DataSource`, the autoconfig backs off. If not, the default is used." 4. "This is why Spring Boot 'just works' but is also fully overridable — autoconfig assumes nothing; it yields to user config."

Edge cases to mention: - `@ConditionalOnProperty` supports `matchIfMissing` — useful for opt-in features - You can write your own `Condition` for complex logic — implement `Condition.matches(ConditionContext, AnnotatedTypeMetadata)` - Use Spring Boot's actuator `/conditions` endpoint in dev to see which autoconfig classes evaluated true/false and why

What NOT to say: - "Conditional annotations slow down startup" — they're evaluated once at config-time, negligible cost - "Use `@Profile` instead of `@ConditionalOnProperty`" — different tools; profiles for env, conditional for feature flags / classpath presence

Common follow-up: "How would you write a custom condition for 'this bean exists only on macOS'?"

Q15 · ApplicationEventPublisher — pub/sub within Spring

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Events

The question: "How would you decouple two services using Spring events?"

The 30-second answer: Inject `ApplicationEventPublisher` and call `publishEvent(new MyEvent(data))`. Any bean with an `@EventListener` method matching the event type receives it. Decouples publisher from listeners — neither knows about the other.

Step-by-step thought process: 1. "Define an event class — could be any object (since Spring 4.2, doesn't need to extend `ApplicationEvent`)." 2. "Publisher: inject `ApplicationEventPublisher`, call `publisher.publishEvent(event)`. Synchronous by default — caller blocks until all listeners

complete." 3. "Listener: any bean method annotated `@EventListener` with a parameter matching the event type. Multiple listeners for one event type allowed." 4. "Async listeners: annotate the method with `@Async` (requires `@EnableAsync` on a config). Runs on a separate thread, caller doesn't block." 5. "Transactional listeners: `@TransactionalEventListener` lets you defer until commit phase — `phase=AFTER_COMMIT` is common, prevents side effects on rollback."

Edge cases to mention: - Synchronous-by-default catches many people by surprise — heavy listener delays the publisher - `@EventListener(condition = "...")` supports SpEL filtering - Spring's own internal events (`ContextRefreshedEvent`, `ApplicationReadyEvent`) are useful for startup hooks

What NOT to say: - "Use events for everything to decouple" — over-using events makes flow opaque; reserve for genuinely independent concerns - "Events are like Kafka" — Spring events are in-process, not durable

Common follow-up: "How would you make sure a side effect only runs if the transaction commits successfully?"

Q16 · ApplicationContextAware and other Aware callbacks

Asked at: Mid-to-senior · **Difficulty:** Medium · **Topic:** Lifecycle hooks

The question: "What are 'Aware' interfaces in Spring? Give an example use case."

The 30-second answer: `*Aware` interfaces let beans receive references to framework infrastructure during initialization. `BeanNameAware` (gets its own name), `BeanFactoryAware`, `ApplicationContextAware`, `ResourceLoaderAware`, `EnvironmentAware`. Spring calls the setter during the bean lifecycle. Usually you don't need them — `@Autowired` `ApplicationContext` or `Environment` instead. But they exist for low-level framework code.

Step-by-step thought process: 1. "Implement `BeanNameAware.setBeanName(String)` to learn your bean's name from the container. Useful for self-identifying logs." 2. "`ApplicationContextAware.setApplicationContext(ApplicationContext)` — gives the bean a reference to the container itself. Used to programmatically look up other beans, publish events, etc." 3. "`EnvironmentAware`, `ResourceLoaderAware` — similar, narrower scope." 4. "Modern alternative: `@Autowired` the dependency directly. `@Autowired private ApplicationContext ctx;` works the same way as `ApplicationContextAware`, but is more idiomatic." 5. "Aware interfaces still matter for framework-level code (Spring's own internals, libraries that need to integrate with Spring without coupling to it via annotations)."

Edge cases to mention: - Aware callbacks fire after dependency injection, before init methods - They couple your bean to Spring — testing in isolation gets harder - Mixing field injection of `ApplicationContext` with Aware is duplicative

What NOT to say: - "Always use Aware to get `ApplicationContext`" — outdated; `@Autowired` is preferred - "Aware interfaces are deprecated" — they're not, just niche

Common follow-up: "When would you ever choose `ApplicationContextAware` over `@Autowired`?"

Q17 · @Value and SpEL — wiring config values

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Properties + SpEL

The question: "What does `@Value` do, and what's SpEL?"

The 30-second answer: `@Value` injects values from properties / env / SpEL expressions into bean fields or constructor params. SpEL = Spring Expression Language — a small expression language used inside `${...}` (property placeholders) or `#{...}` (SpEL expressions). Useful for simple injections; for grouped config prefer `@ConfigurationProperties`.

Step-by-step thought process: 1. "`@Value("${app.timeout:30}")` — inject property `app.timeout`, default to 30 if not set. Uses the property placeholder syntax `${...}`." 2. "`@Value("#{T(java.lang.Math).PI}")` — SpEL expression syntax `#{...}`. Can call static methods, do arithmetic, reference other beans (`#{otherBean.field}`)." 3. "Source order for `${...}`: command-line args → `SPRING_APPLICATION_JSON` → `application-{profile}.properties` → `application.properties` → defaults. Resolved by `PropertySourcesPlaceholderConfigurer`." 4. "For grouped config (a whole config object), prefer `@ConfigurationProperties(prefix = "app")` over scattered `@Value` annotations. Better testability, type-safe binding." 5. "Common pitfalls: `@Value` on field doesn't work in constructor; can't use SpEL to reference beans not yet constructed."

Edge cases to mention: - SpEL supports `safe navigation` (`?.`) — useful for null-safe property reads - Lists/maps via `${app.items:#{T(java.util.Collections).emptyList()}}` — clunky; use `@ConfigurationProperties` instead - `@Value` resolution happens at injection time — runtime updates to properties won't propagate

What NOT to say: - "Use `@Value` for every config" — verbose; `@ConfigurationProperties` scales better - "SpEL is the same as JSP EL" — different language, similar syntax

Common follow-up: "Why would you choose `@ConfigurationProperties` over `@Value`?"

Q18 · @ConfigurationProperties — type-safe config binding

Asked at: Mid-level Spring Boot · **Difficulty:** Easy-Medium · **Topic:** Properties

The question: "What's `@ConfigurationProperties` and why prefer it over `@Value`?"

The 30-second answer: `@ConfigurationProperties(prefix = "app.email")` binds an entire properties subtree into a typed POJO. Cleaner than scattering `@Value` annotations across the codebase, gives you validation via Bean Validation, IDE auto-completion, and grouped configuration that's easier to test.

Step-by-step thought process: 1. "Define a class with fields matching your properties: `class EmailProps { String host; int port; List<String> recipients; }` ." 2. "Annotate with `@ConfigurationProperties(prefix = "app.email")` . Spring binds `app.email.host` , `app.email.port` , `app.email.recipients[0]` etc. to the fields." 3. "Two ways to enable: (a) `@ConfigurationPropertiesScan` (Spring Boot 2.2+) on a config class — scans for `@ConfigurationProperties` classes; (b) explicit `@EnableConfigurationProperties(EmailProps.class)` ." 4. "Bonus: pair with `@Validated` and Bean Validation annotations (`@NotBlank` , `@Min` , etc.). App fails fast at startup with clear errors if config is missing or wrong." 5. "Constructor binding (`@ConfigurationProperties` + `@ConstructorBinding` , or just a `record`) makes the props class immutable — even cleaner."

Edge cases to mention: - Auto-binding from `kebab-case` or `snake_case` properties to `camelCase` Java fields — Spring handles this - Nested properties bind to nested objects automatically - Spring Boot's `spring-boot-configuration-processor` annotation processor generates IDE metadata for autocompletion

What NOT to say: - "Use `@Value` because it's simpler" — only for single values; loses on groups - "`@ConfigurationProperties` requires Lombok" — no; records or plain classes work fine

Common follow-up: "How would you validate that a required property is set?"

Q19 · BeanPostProcessor — what is it and when would you write one?

Asked at: Senior + Spring-internal rounds · **Difficulty:** Hard · **Topic:** Extension point

The question: "What's a `BeanPostProcessor`, and when would you implement your own?"

The 30-second answer: A `BeanPostProcessor` is a Spring extension point — `postProcessBeforeInitialization` and `postProcessAfterInitialization` run on EVERY bean during creation. Use it to wrap beans with proxies, mutate them, or install cross-cutting behavior. Rarely needed in application code — useful for libraries and framework extensions.

Step-by-step thought process: 1. "Implement `BeanPostProcessor` : two callbacks, both receive the bean instance + name." 2. " `postProcessBeforeInitialization` runs AFTER injection but BEFORE `@PostConstruct` / init methods. `postProcessAfterInitialization` runs AFTER init." 3. "Either method can return a different object — that's how AOP installs proxies (Spring's `AbstractAutoProxyCreator` is a `BeanPostProcessor`)." 4. "Use case: scan every bean for a custom annotation and decorate matching ones with a metric recorder, retry wrapper, etc." 5. "BFPP (`BeanFactoryPostProcessor`) is similar but operates on bean DEFINITIONS before instantiation. Useful for mutating config (e.g. property substitution)."

Edge cases to mention: - BPP itself is a Spring bean — but it must be registered as a `static @Bean` or it can't process beans that depend on it - High-priority BPPs (implement `Ordered`) run earlier — important when ordering matters for proxy installation - ApplicationContext-level: `BeanDefinitionRegistryPostProcessor` can ADD/REMOVE bean definitions before instantiation

What NOT to say: - "Use BPP for every cross-cutting concern" — AOP is usually cleaner - "BPP runs once" — it runs once per bean

Common follow-up: "Walk me through how Spring's `@Transactional` gets installed via BPP."

Q20 · @Lazy — when and why

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Lifecycle

The question: "What does `@Lazy` do, and when would you use it?"

The 30-second answer: `@Lazy` on a bean delays creation until first injection or first lookup. `@Lazy` on an injection point injects a proxy that resolves the real bean lazily. Useful for: heavy beans rarely used, breaking circular dependencies, optional features. Default is eager because eager catches config errors at startup.

Step-by-step thought process: 1. "By default, Spring eagerly creates singletons at startup. Catches missing-bean errors immediately; warms caches; ready to serve." 2. " `@Lazy` on a `@Component` / `@Bean` defers creation until first injection or `getBean()` call." 3. " `@Lazy` at the INJECTION POINT (parameter, field) is more interesting — Spring injects a proxy that doesn't resolve until first method call. Useful for breaking circular dependencies." 4. "Use cases: a heavy bean that's needed only on rare admin endpoints; a circular dependency where breaking the cycle requires deferring construction." 5. "Trade-off: lazy = faster startup, but errors deferred to runtime (a misconfigured bean will fail on first user request, not at deploy)."

Edge cases to mention: - Spring Boot 2.2+ has a global `spring.main.lazy-initialization=true` — turns all beans lazy by default. Good for dev startup speed; bad for prod (errors surface late) - `@Lazy` on prototype beans is meaningless — prototypes are always lazy - `@Lazy` interacts poorly with AOP if not careful — proxy-on-proxy can confuse advice ordering

What NOT to say: - "Always use `@Lazy` for faster startup" — masks startup errors; bad default for prod - "Lazy beans aren't real" — they are; just constructed on demand

Common follow-up: "How would `@Lazy` help you break a circular dependency?"

Q21 · @Scope("prototype") gotcha with singleton injection

Asked at: Mid-to-senior · **Difficulty:** Medium-Hard · **Topic:** Scope semantics

The question: "A singleton bean depends on a prototype bean. Will the singleton get a fresh prototype on each method call?"

The 30-second answer: NO — by default, singleton holds ONE prototype reference, captured at injection time. To get fresh prototypes per call: use `@Lookup` method override, inject

`ObjectFactory<T>` / `ObjectProvider<T>` / `Provider<T>` , or inject `ApplicationContext` and call `getBean()` .

Step-by-step thought process: 1. "Singleton is created once with its dependencies wired. When prototype is wired in, Spring resolves it ONCE — singleton holds that reference forever, even though the prototype is logically supposed to be 'fresh per use.'" 2. "Fix #1: `@Lookup` annotation on an abstract or empty method in the singleton. Spring overrides at runtime to call `getBean(prototype.class)` each call. Returns a fresh prototype every invocation." 3. "Fix #2: inject `ObjectProvider<MyPrototype>` (or `ObjectFactory<MyPrototype>` / JSR-330's `Provider<MyPrototype>`). Call `.getObject()` (or `.get()`) for a fresh instance." 4. "Fix #3: inject `ApplicationContext` and call `context.getBean(MyPrototype.class)` — works but couples to Spring." 5. "Common in: any factory-like pattern, per-request stateful processors, request-scoped beans in non-web contexts."

Edge cases to mention: - Web scopes (`@RequestScope` , `@SessionScope`) injected into singletons need the same treatment — Spring offers `proxyMode = TARGET_CLASS` on these scope annotations - `ObjectProvider` supports `getIfAvailable()` / `getIfUnique()` for safe optional handling

What NOT to say: - "Just declare both as singleton" — defeats the purpose - "Spring handles this automatically" — it doesn't; this is the classic gotcha

Common follow-up: "Show me how to inject a request-scoped bean into a singleton service."

Q22 · Properties resolution order in Spring Boot

Asked at: Common in ops-y rounds · **Difficulty:** Medium · **Topic:** Configuration

The question: "What's the order Spring Boot resolves property values from? Where does a command-line arg fit?"

The 30-second answer: Highest priority first: command-line args → `SPRING_APPLICATION_JSON` → `ServletConfig` / `ServletContext` init params (web only) → JNDI → Java system properties (`-D...`) → OS environment variables → `RandomValuePropertySource` → profile-specific `application-{profile}.yml,properties` → base `application.yml,properties` → `@PropertySource` → `SpringApplication.setDefaultProperties()` . Higher wins. (Common interview slip: Java system properties beat env vars, not the other way around.)

Step-by-step thought process: 1. "Command-line args (`--server.port=9000`) trump everything. Useful for one-off overrides." 2. " `SPRING_APPLICATION_JSON` env var — JSON-encoded properties, useful for containerized environments where you set one env var with all overrides." 3. "In web apps: `ServletConfig` / `ServletContext` init params come next (rarely used in Spring Boot but on the official order)." 4. "JNDI lookups (rare today; relevant for app-server deployments)." 5. "Java system properties (`-Dserver.port=9000`) — ABOVE OS environment variables in priority." 6. "OS environment variables — `SERVER_PORT=9000` maps to `server.port` . Spring's relaxed binding converts

SCREAMING_SNAKE → kebab-case → camelCase." 7. " `RandomValuePropertySource` (`random.int` , `random.uuid`) — useful for tests / placeholder secrets." 8. "Then properties files: profile-specific files override the base. `application-prod.yml` beats `application.yml` ." 9. " `@PropertySource` annotation imports a specific file — lower priority than the standard ones."

Edge cases to mention: - Spring Boot 2.4+ adds `spring.config.import` — flexible imports including Vault, ConfigMap, AWS Secrets Manager - Custom property sources via `ConfigDataLocationResolver` (advanced extension point) - For secrets, use `spring.config.import=optional:secret://...` patterns from cloud providers

What NOT to say: - "Config is loaded once at startup, period" — true for static config, but cloud sources can refresh - "application.yml always wins" — wrong; it's at the bottom of the priority stack

Common follow-up: "How would you load secrets from AWS Secrets Manager into Spring properties?"

Q23 · Spring's resource abstraction (Resource, ResourceLoader)

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Resources

The question: "How does Spring abstract resource loading? When does it matter?"

The 30-second answer: `Resource` interface abstracts a file-like thing — could be on classpath, filesystem, URL, in-memory. `ResourceLoader` resolves a string location (e.g. `classpath:config.json` , `file:/etc/app/config.json` , `https://example.com/cfg.json`) into a `Resource`. `ApplicationContext` IS a `ResourceLoader`, so any Spring bean can load resources cleanly.

Step-by-step thought process: 1. " `Resource` interface: methods like `getInputStream()` , `exists()` , `getURL()` , `getFilename()` . Spring provides `ClassPathResource` , `FileSystemResource` , `UrlResource` , `ByteArrayResource` , `ServletContextResource` ." 2. " `ResourceLoader` resolves location strings. Prefixes: `classpath:` (default), `file:` , `https:` / `http:` , `classpath*:` (matches across multiple JARs)." 3. "Inject `ResourceLoader` (or just `ApplicationContext`) and call `getResource("classpath:templates/email.html")` . Use the `InputStream` to read content." 4. " `ResourcePatternResolver` extends `ResourceLoader` with `getResources(pattern)` — wildcards (`classpath*:META-INF/services/*`). Used by component scanning, JAR-merging libraries." 5. "Use case: loading static templates (email bodies, SQL scripts, JSON schemas) without hardcoding file paths or worrying about classpath vs filesystem semantics."

Edge cases to mention: - `classpath*` (with asterisk) finds resources from ALL classpath entries — useful for plugin-style configs - `Resource.exists()` doesn't throw if the resource is missing — friendlier than `File` API - Spring uses `Resource` internally for things like XML config, SQL script execution

What NOT to say: - "Just use `new File(...)` " — breaks classpath resources, breaks in JAR-packaged apps - "ResourceLoader is for files only" — wrong, handles classpath / URL / in-memory

Common follow-up: "How would you load an SQL migration script from the classpath?"

Q24 · @Async — async method execution

Asked at: Common in async-heavy interviews · **Difficulty:** Medium · **Topic:** Async

The question: "How does @Async work in Spring? What do I need to configure for it?"

The 30-second answer: `@Async` on a method tells Spring to invoke it on a different thread. Requires `@EnableAsync` on a config class to activate. Method returns `void`, `Future<T>`, or `CompletableFuture<T>`. Caller doesn't block. By default uses `SimpleAsyncTaskExecutor` (new thread per call — bad for production); configure a `ThreadPoolTaskExecutor` bean named `taskExecutor` to fix.

Step-by-step thought process: 1. "`@EnableAsync` on a `@Configuration` class enables async proxy generation." 2. "Methods annotated `@Async` get proxied. Caller's thread submits to an executor and returns immediately." 3. "Return type matters: `void` (fire-and-forget, no result), `Future<T>` (caller can `.get()` later — blocking), `CompletableFuture<T>` (caller can chain `.thenApply` etc. — non-blocking)." 4. "Default executor is `SimpleAsyncTaskExecutor` — spawns a new thread per task. Bad for production; you'll exhaust resources under load." 5. "Production fix: define a `ThreadPoolTaskExecutor` bean named `taskExecutor` (or override the `AsyncConfigurer`). Set core pool size, max pool size, queue capacity, thread name prefix."

Edge cases to mention: - `@Async` is proxy-based — self-invocation doesn't go async (same trap as `@Transactional`) - Combining `@Async` with virtual threads (Spring Boot 3.2+): `Executors.newVirtualThreadPerTaskExecutor()` as the TaskExecutor — millions of cheap concurrent tasks - Uncaught exceptions in async `void` methods are silently swallowed unless you configure an `AsyncUncaughtExceptionHandler`

What NOT to say: - "@Async is the same as `CompletableFuture`" — async is a Spring proxy mechanism; `CompletableFuture` is the result type - "Async methods are always faster" — only if the work is offloadable AND you're not bottlenecked elsewhere

Common follow-up: "How would you handle exceptions thrown by an @Async void method?"

Q25 · @Scheduled — task scheduling

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Scheduling

The question: "How does @Scheduled work? When would you NOT use it?"

The 30-second answer: `@Scheduled` on a method runs it on a schedule via `TaskScheduler`. Requires `@EnableScheduling`. Variants: `fixedRate`, `fixedDelay`, `cron`. Single-instance-

only — in a multi-pod deployment, every pod runs the schedule unless you add coordination (DB lock, Redis lock, leader election).

Step-by-step thought process: 1. "`@EnableScheduling` on a config class activates scheduling." 2. "Method annotated `@Scheduled(fixedRate = 60000)` runs every 60 seconds, regardless of previous run's duration. `fixedDelay` waits for the previous run to finish + delay. `cron = "0 0 * * * *"` for cron-syntax schedules." 3. "Default `TaskScheduler` is single-threaded — if scheduled work overlaps or takes long, other tasks queue. Configure a `ThreadPoolTaskScheduler` for multi-task parallelism." 4. "Multi-pod trap: every pod's Spring context has its own scheduler. Without coordination, every pod runs the task — duplicated work, possibly broken side effects." 5. "Solutions: (a) leader election (ShedLock library is popular — uses DB/Redis lock to ensure only one pod runs at a time); (b) external scheduler (Kubernetes CronJob, AWS EventBridge); (c) single dedicated pod with `spring.profiles.active=scheduler` only."

Edge cases to mention: - ShedLock library is the de-facto standard for "`@Scheduled` but only on one node" — uses DB lock with TTL - `@Scheduled(initialDelay = 30000)` for one-time delayed init - Spring's cron syntax has 6 fields (seconds first) — different from Unix cron's 5

What NOT to say: - "Just use `@Scheduled` in production" — works, but multi-pod = duplicated runs - "Scheduled methods must return void" — true, but it's worth saying explicitly

Common follow-up: "How would you ensure a scheduled job runs on only ONE pod in a Kubernetes deployment?"

Q26 · Spring Environment abstraction

Asked at: Senior · **Difficulty:** Medium · **Topic:** Configuration

The question: "What's the Environment in Spring? How is it different from `System.getenv()`?"

The 30-second answer: `Environment` is Spring's unified abstraction over property sources (env vars, properties files, system props, command-line, custom sources). Gives uniform access via `env.getProperty("name")`. Also tracks active profiles (`env.getActiveProfiles()`). Inject it instead of calling `System.getenv()` — wins on consistency, testability, profile-awareness.

Step-by-step thought process: 1. "Inject `Environment` directly: `@Autowired private Environment env;`" 2. "Access properties: `env.getProperty("server.port")` (returns `String` or `null`), or `env.getProperty("server.port", Integer.class, 8080)` (typed with default)." 3. "Profile checking: `env.getActiveProfiles()` returns the array, `env.acceptsProfiles(Profiles.of("dev"))` for boolean check." 4. "Why not `System.getenv()` : skips Spring's property source layering (no `.properties` file resolution, no relaxed binding, no defaults). Inconsistent behavior between code paths." 5. "In rare cases, you might use `Environment` programmatically — typically you'd use `@Value` or `@ConfigurationProperties` instead, which are wired by `Environment` under the hood."

Edge cases to mention: - Custom property sources: add via

`ConfigurableEnvironment.getPropertySources().addFirst(myPropertySource)` — useful for loading config from a database - `Environment` is itself a Spring bean — can be substituted in tests - `MutablePropertySources` lets you reorder source precedence at runtime (rarely needed)

What NOT to say: - "System.getenv is fine for env vars" — works but bypasses Spring's layered config - "Environment is just a Map" — has profiles + ordering semantics on top

Common follow-up: "How would you load properties from a database table at startup?"

Q27 · @ControllerAdvice — global exception handling

Asked at: Common in MVC interviews · **Difficulty:** Easy-Medium · **Topic:** Cross-cutting

The question: "How would you handle exceptions across all controllers in one place?"

The 30-second answer: `@ControllerAdvice` (or `@RestControllerAdvice` for REST) on a class makes its `@ExceptionHandler` methods apply globally. Match by exception type. Returns whatever the response should be (`ResponseEntity`, error DTO, view name). Clean separation of business code and error mapping.

Step-by-step thought process: 1. "`@RestControllerAdvice public class GlobalExceptionHandler` is the standard pattern." 2. "Define handlers: `@ExceptionHandler(EntityNotFoundException.class) public ResponseEntity<ErrorDto> handle(EntityNotFoundException ex) { return ResponseEntity.status(404).body(new ErrorDto(ex.getMessage())); }`." 3. "Multiple handlers for different exception types. Spring picks the most specific match (e.g. `IllegalArgumentException` handler called for both itself AND `NumberFormatException` unless you handle `NumberFormatException` explicitly)." 4. "`@ResponseStatus(HttpStatus.NOT_FOUND)` annotation on your custom exception type also works for simple cases — Spring maps that to the HTTP status automatically." 5. "Scope advice by package: `@ControllerAdvice(basePackages = "com.foo.api.v1")` — useful for v1 vs v2 different error formats."

Edge cases to mention: - `ResponseEntityExceptionHandler` is a Spring-provided base class for handling MVC framework exceptions (validation, missing params, etc.) — extend it to standardize error responses - Order between multiple `@ControllerAdvice` classes via `@Order` or `Ordered` interface - For reactive (WebFlux): use `@ControllerAdvice` similarly with `Mono<ResponseEntity<...>>` return types

What NOT to say: - "Handle exceptions in each controller" — duplicated code, inconsistent error formats - "Use try-catch everywhere" — boilerplate, misses cross-cutting concerns

Common follow-up: "How would you standardize error responses across a multi-team API?"

Q28 · @Validated and Bean Validation

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Validation

The question: "How does Spring integrate with Bean Validation (JSR 380)?"

The 30-second answer: Annotate DTO fields with `@NotNull`, `@Size`, `@Email`, `@Min`, etc. In controllers, annotate the parameter with `@Valid` or `@Validated` — Spring validates and throws `MethodArgumentNotValidException` on failure. For service-layer validation, annotate the CLASS with `@Validated` and method args with constraint annotations.

Step-by-step thought process: 1. "Add `spring-boot-starter-validation` — pulls in Hibernate Validator (reference impl)." 2. "Annotate DTO fields:

`@NotBlank String name; @Min(0) int age; @Email String email; ."` 3. "In controllers: `@PostMapping public Response create(@Valid @RequestBody UserDto user)` — validation runs before the method body." 4. "On failure: Spring throws `MethodArgumentNotValidException`. Handle in `@ControllerAdvice`. Return a structured error response with field-level messages." 5. "For service-layer methods: Spring Framework 6.1+ has built-in method-level validation that throws `HandlerMethodValidationException` (handle in `@ControllerAdvice`). The older `@Validated` on class + constraint annotations on parameters still works (throws `ConstraintViolationException`) but opts you out of the modern path — pick one and be consistent."

Edge cases to mention: - Custom validation: implement `ConstraintValidator` + define a custom annotation. Useful for cross-field validation (e.g. password confirmation) - Groups:

`@Validated(CreateGroup.class)` runs only validators in that group — useful for different validation rules on create vs update - Method-level validation via `@Validated` on classes uses AOP — same proxy caveats apply

What NOT to say: - "Use `@Valid` for everything" — `@Valid` is JSR-303 standard; `@Validated` is Spring's extension supporting groups + method-level - "Just check fields manually" — re-implementing what Bean Validation already does cleanly

Common follow-up: "How would you validate that 'startDate < endDate' across two fields?"

Q29 · WebApplicationContext vs ApplicationContext

Asked at: Senior + web-internals rounds · **Difficulty:** Medium · **Topic:** Web context

The question: "What's a `WebApplicationContext` and how does it relate to `ApplicationContext`?"

The 30-second answer: `WebApplicationContext` extends `ApplicationContext` with web-specific features: access to `ServletContext`, support for `request` / `session` / `application` scopes, hooks for SpringMVC's `DispatcherServlet`. In Spring Boot you get it automatically; the `WebApplicationContext` is the runtime context for web apps.

Step-by-step thought process: 1. "`WebApplicationContext` adds `getServletContext()` to `ApplicationContext`. Enables web-aware scopes." 2. "There's typically ONE root `WebApplicationContext` (shared across the app) and OPTIONALLY one child context per `DispatcherServlet` (in old XML-config apps; rare in Spring Boot)." 3. "Spring Boot embeds the servlet container (Tomcat/Jetty/Undertow) and creates a single `WebApplicationContext` at startup." 4. "Reactive variant: `ReactiveWebApplicationContext` for WebFlux apps. Same idea, no servlet." 5. "You rarely touch it directly. It's the substrate under everything in a web app."

Edge cases to mention: - Pre-Spring-Boot pattern: `ContextLoaderListener` (root context) + `DispatcherServlet` (child context) — historical, mostly gone - `AnnotationConfigWebApplicationContext` is the modern programmatic factory - For testing web layers: `@WebMvcTest` and `@WebFluxTest` create slice-only `WebApplicationContexts`

What NOT to say: - "It's just `ApplicationContext` for web" — accurate but misses scopes + `servletcontext` - "Use it for non-web apps" — defeats the point

Common follow-up: "What's the difference between `WebApplicationContext` and `ReactiveWebApplicationContext`?"

Q30 · BeanFactory vs FactoryBean — different things

Asked at: Senior + tricky-Spring rounds · **Difficulty:** Hard · **Topic:** Confusing names

The question: "What's the difference between `BeanFactory` and `FactoryBean`? Why do both exist?"

The 30-second answer: `BeanFactory` is the container — it produces beans. `FactoryBean<T>` is a special kind of BEAN whose JOB is to produce another bean. When you inject `MyService` but the container has a `MyServiceFactoryBean`, Spring calls `factoryBean.getObject()` to produce the actual `MyService` instance. Useful for complex bean construction logic.

Step-by-step thought process: 1. "`BeanFactory` = the IoC container. The thing that manages beans. Discussed in Q7." 2. "`FactoryBean<T>` = a bean that PRODUCES other beans. Implement `getObject()` returning the `T` instance, `getObjectType()` returning the type, `isSingleton()` ." 3. "When you `@Autowired MyService myService`, if the registered bean is `MyServiceFactoryBean`, Spring calls `factoryBean.getObject()` and gives you the `T` result, not the factory." 4. "Use cases: third-party class construction that needs builder/factory logic, JPA `EntityManager`, JMS `ConnectionFactory` wrappers, complex objects with optional config." 5. "To get the FACTORY itself (not its product): prefix the bean name with `&` — `getBean("&myService")` . Rarely needed."

Edge cases to mention: - `SmartFactoryBean` adds eager/prototype hinting - In modern Spring, `@Bean` methods replace most `FactoryBean` usage — much cleaner - Spring uses `FactoryBean` internally for things like `LocalContainerEntityManagerFactoryBean` (JPA `EntityManagerFactory`)

What NOT to say: - "BeanFactory and FactoryBean are the same" — opposite roles; classic confusion - "FactoryBean is deprecated" — it's not; just rarely needed in user code

Common follow-up: "When would you write a FactoryBean instead of a @Bean method?"

Section 2 — Spring Boot (30 Qs)

What makes Spring Boot the default for new Java apps. Topics: autoconfiguration, starters, actuator, embedded server, Spring Boot 3 migration, virtual threads, AOT / native.

Q31 · Spring Boot autoconfiguration — what is it?

Asked at: Almost every Spring Boot interview · **Difficulty:** Easy-Medium · **Topic:** Autoconfig

The question: "Walk me through how Spring Boot autoconfiguration works."

The 30-second answer: Spring Boot ships with auto-configuration classes that detect classpath + property + existing-bean conditions and register reasonable defaults. `@SpringBootApplication` triggers `@EnableAutoConfiguration` which loads classes listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. Each class is gated by `@ConditionalOn*` annotations.

Step-by-step thought process: 1. "`@SpringBootApplication` is `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`." 2. "`@EnableAutoConfiguration` triggers the auto-config import process. Spring Boot 2.7+ reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` from every JAR on the classpath (was `META-INF/spring.factories` pre-2.7)." 3. "Each listed class is a `@Configuration` gated by `@ConditionalOn*` (see Q14). E.g. `DataSourceAutoConfiguration` is gated by `@ConditionalOnClass(DataSource.class)` + `@ConditionalOnMissingBean(DataSource.class)`." 4. "At startup, Spring evaluates each candidate's conditions. If all true, the @Beans inside register. If any fail, the config is skipped." 5. "Net effect: add `spring-boot-starter-data-jpa` to your pom, get a fully-configured DataSource + EntityManager + transaction manager + Spring Data repositories — all without writing any of it."

Edge cases to mention: - Disable a specific auto-config: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` - See evaluated conditions: enable actuator's `/conditions` endpoint or `--debug` flag at startup - Order autoconfig classes via `@AutoConfigureBefore` / `@AutoConfigureAfter`

What NOT to say: - "Autoconfig is magic" — it's just `@Conditional` evaluation; debuggable - "Spring Boot replaces Spring" — wrong; Spring Boot is conventions + autoconfig on top of plain Spring

Common follow-up: "How would you write your own autoconfiguration?"

Q32 · Starters — what are they really?

Asked at: Common · **Difficulty:** Easy · **Topic:** Dependency management

The question: "What's a Spring Boot starter? Walk me through what `spring-boot-starter-web` actually does."

The 30-second answer: A starter is a Maven/Gradle artifact that pulls in a curated set of dependencies for a use case — usually as a transitive-only POM. `spring-boot-starter-web` pulls Spring MVC + embedded Tomcat + Jackson + validation. No code of its own; just dependency aggregation.

Step-by-step thought process: 1. "Starters are Maven/Gradle artifacts (usually `spring-boot-starter-{thing}`) that bring in a coherent set of dependencies." 2.

"`spring-boot-starter-web` declares (transitively): `spring-boot-starter` (core), `spring-web`, `spring-webmvc`, `spring-boot-starter-tomcat` (embedded server), `spring-boot-starter-json` (Jackson), `spring-boot-starter-validation`. Add the starter — get the full stack." 3. "Most starters have a paired auto-configuration. Adding the starter brings in JARs; the autoconfig classes inside those JARs do the wiring." 4. "For non-web apps: use `spring-boot-starter` (just the core) and pick specific deps. Avoid pulling Tomcat if you don't need it." 5. "Custom starters: a published artifact + an autoconfig class + condition annotations. Useful for internal teams shipping reusable infrastructure (a 'company-starter-logging' kind of thing)."

Edge cases to mention: - `spring-boot-starter-tomcat` vs `spring-boot-starter-jetty` vs `spring-boot-starter-undertow` — pick one if you want a different embedded server - `<scope>provided</scope>` for war-deployment scenarios (don't bundle the server) - Spring Boot's BOM (`spring-boot-dependencies`) manages versions — you usually don't specify versions in your POM

What NOT to say: - "Starters contain code" — they don't, just POMs - "spring-boot-starter-web is required for any Spring Boot app" — false, lots of apps are CLI / batch / no-web

Common follow-up: "How would you build your own starter?"

Q33 · Spring Boot Actuator — production-ready endpoints

Asked at: Common in ops-y rounds · **Difficulty:** Easy-Medium · **Topic:** Observability

The question: "What's Spring Boot Actuator? Which endpoints would you turn on for a production service?"

The 30-second answer: Actuator exposes built-in endpoints for ops — health, metrics, info, env, beans, mappings, threaddump, heapdump, loggers, conditions. Add `spring-boot-starter-actuator` to enable. In prod: expose `health` + `info` + `prometheus` (for scraping) + `loggers`. Keep `heapdump` / `threaddump` available but secured.

Step-by-step thought process: 1. "`spring-boot-starter-actuator` brings in the actuator endpoints. By default only `/actuator/health` is exposed over HTTP." 2. "Expose more:

`management.endpoints.web.exposure.include=health,info,metrics,prometheus,loggers` . Comma-separated names, or `*` for all." 3. "Health: pluggable health indicators auto-detect DB, Redis, message brokers, etc. Returns UP/DOWN/UNKNOWN per component. Used by load balancers / K8s liveness+readiness probes." 4. "Metrics: Micrometer-backed. Tagged metrics for JVM, HTTP, datasource, cache, etc. Expose via `/actuator/prometheus` for scraping into Grafana." 5. "Loggers: GET to see current log levels, POST to change at runtime — incredibly useful for prod debugging without redeploy." 6. "Security: actuator endpoints leak info — secure them. Spring Security can require auth on `/actuator/**` while keeping `/actuator/health` public."

Edge cases to mention: - `management.server.port=9090` puts actuator on a different port — useful to keep ops endpoints off the public-facing port - Custom health indicators: implement `HealthIndicator` , your health check appears in `/actuator/health` - `management.endpoint.health.show-details=when-authorized` — show component-level health only to authenticated users

What NOT to say: - "Expose all endpoints in prod" — leak risk (env vars in `/env` , bean names in `/beans`) - "Use it only for dev" — production is exactly where it shines

Common follow-up: "How would you build a custom health indicator?"

Q34 · Embedded server — Tomcat vs Jetty vs Undertow vs Netty

Asked at: Architect / senior rounds · **Difficulty:** Medium · **Topic:** Server selection

The question: "What embedded server does Spring Boot use by default, and when would you switch?"

The 30-second answer: Tomcat is the default (matches industry-standard production deployments). Jetty and Undertow are alternative servlet containers. Netty is the non-servlet, reactive default (used with WebFlux). Switch reasons: lower memory (Undertow), reactive (Netty), better WebSocket (Jetty in some cases).

Step-by-step thought process: 1. "`spring-boot-starter-web` brings Tomcat by default. Familiar, battle-tested, the safest pick." 2. "Switch to Jetty: exclude Tomcat from `spring-boot-starter-web`, add `spring-boot-starter-jetty` . Slightly different threading model, sometimes faster startup, similar throughput." 3. "Undertow: lower memory footprint than Tomcat, async-first design. Used by Red Hat / Quarkus folks. Pick for memory-constrained envs." 4. "Netty: not a servlet container; used by WebFlux for reactive non-blocking IO. `spring-boot-starter-webflux` brings Netty by default." 5. "For most teams, stick with Tomcat unless you have a specific reason. Operational familiarity > micro-optimizations."

Edge cases to mention: - Spring Boot 3.2+: enable virtual threads on Tomcat with `spring.threads.virtual.enabled=true` — usually a bigger win than switching servers - AOT/native compilation: GraalVM native image works with all four; some have better support than others - For WebSockets: Tomcat and Jetty both support; Undertow has stronger XNIO-based async support

What NOT to say: - "Switch to Undertow for performance" — only marginal in most workloads - "Netty is faster than Tomcat" — different programming model; not directly comparable

Common follow-up: "What's the difference between Tomcat + virtual threads and Netty + reactive?"

Q35 · application.yml vs application.properties

Asked at: Easy · **Difficulty:** Easy · **Topic:** Config format

The question: "YAML or properties — which would you pick and why?"

The 30-second answer: YAML for nested config, multi-env files, lists. Properties for flat key-value, easier diff in PRs, no indentation pitfalls. Either works; team preference matters more than the format. YAML is more common in modern Spring Boot apps.

Step-by-step thought process: 1. ".properties" is flat:

`app.email.smtp.host=smtp.gmail.com` . Easy to grep, easy to diff. " 2. "YAML is nested:

`app:\n email:\n smtp:\n host: smtp.gmail.com` . More compact for deeply-nested config.

Supports lists and maps natively. " 3. "Both bind to `@ConfigurationProperties` the same way. Bind

order is identical. " 4. "For lists: properties uses index syntax (`app.servers[0]=...`) — clunky. YAML

lists are cleaner. " 5. "Watch out in YAML: indentation matters (always spaces, never tabs); strings with `:` need quotes; multi-document with `---` separators for profile-specific sections in one file. "

Edge cases to mention: - Comments differ: `#` in both, but YAML supports inline comments after values - YAML supports anchors / references (rarely useful in app config) - Encrypted properties (Jasypt) — same approach works for both

What NOT to say: - "YAML is always better" — flat configs are cleaner in properties - "Use both" — split-brain, confusing

Common follow-up: "How would you have profile-specific config in one YAML file?"

Q36 · Spring Boot 3 migration — what breaks?

Asked at: Common in 2024-26 interviews · **Difficulty:** Medium · **Topic:** Migration

The question: "You're moving an app from Spring Boot 2.7 to 3.0. What breaks?"

The 30-second answer: Five major shifts: (1) Java 17 baseline (was 8). (2) `javax.*` → `jakarta.*` package rename across servlet, JPA, validation, mail. (3) Spring Security 6's new lambda DSL replaces fluent builder. (4) Micrometer Observation replaces Sleuth. (5) Hibernate 6 with stricter SQL generation. Use openrewrite recipes for the mechanical parts; test the rest carefully.

Step-by-step thought process: 1. "Java baseline 17+. If you're on 8/11, that's the first hill. Look at all your dependencies — many also dropped older Java around the same time." 2. "Jakarta migration: every

`javax.persistence`, `javax.servlet`, `javax.validation`, `javax.mail` import → `jakarta.*`. Mechanical but pervasive. IDE refactor handles most; openrewrite recipe `org.openrewrite.java.migrate.UpgradeToJava17` + Jakarta recipes automate it." 3. "Spring Security 6: HttpSecurity DSL is lambda-only — `.authorizeHttpRequests(auth -> auth...)` instead of `.authorizeHttpRequests().anyRequest()....`. `WebSecurityConfigurerAdapter` is gone — use `SecurityFilterChain @Bean`." 4. "Observability: Spring Sleuth (distributed tracing) is replaced. New stack: Micrometer Tracing + Micrometer Observation API. Different metric / trace naming." 5. "Hibernate 6: stricter SQL generation, new ID generators, JPQL behavior changes. Test all queries; some implicit casts no longer work."

Edge cases to mention: - Spring Boot 3.2 (Nov 2023) added virtual thread support — bonus capability after migration - AOT compilation for native images — additional capability you can adopt incrementally - Many Spring projects also bumped at the same time (Spring Cloud, Spring Data, Spring Integration) — watch their compat matrices

What NOT to say: - "Just bump the version" — Jakarta alone is a multi-day refactor on big codebases - "Native compilation is mandatory" — it's not; just available

Common follow-up: "What's openrewrite and how does it help with the migration?"

Q37 · Externalized config in Spring Boot

Asked at: Common ops rounds · **Difficulty:** Easy-Medium · **Topic:** Config

The question: "How would you keep prod secrets out of your `application.yml`?"

The 30-second answer: Don't put them in. Use environment variables for secrets — Spring Boot's relaxed binding maps `SPRING_DATASOURCE_PASSWORD` to `spring.datasource.password` automatically. For more: Vault (`spring-cloud-starter-vault-config`), AWS Secrets Manager (`spring-cloud-starter-aws-secrets-manager-config`), Kubernetes Secrets as env vars. Never commit secrets to git.

Step-by-step thought process: 1. "Default for simple cases: env vars.

`SPRING_DATASOURCE_PASSWORD=...` in deployment. Spring Boot binds automatically." 2. "For richer secret management: Vault — declare `spring.config.import=vault://secret/myapp`, Spring fetches at startup. Auto-rotation supported." 3. "AWS: Secrets Manager or Systems Manager Parameter Store. `spring.config.import=aws-secretsmanager:/myapp/prod` (Spring Cloud AWS)." 4. "Kubernetes: mount Secrets as env vars or files. Spring Boot's `spring.config.import=optional:file:/etc/secrets/` reads them." 5. "NEVER: secrets in `application.yml` committed to git. NEVER: secrets in Docker image envs (baked in). NEVER: secrets in command-line args (visible in `ps`)."

Edge cases to mention: - For dev env: `application-local.yml` with `.gitignore` entry, OR `direnv` / `.env` files NOT committed - For testing: `@TestPropertySource` or

`@DynamicPropertySource` to inject test secrets - Auditability: log access to secrets, rotate regularly, use short-lived credentials when possible (Vault dynamic secrets, IAM roles for AWS)

What NOT to say: - "Just encrypt the yml file" — adds key management problem on top of secret problem - "Use a config server" — Spring Cloud Config Server doesn't solve the secret-storage problem on its own

Common follow-up: "How does Spring Boot 2.4+'s `spring.config.import` work with Vault?"

Q38 · Embedded H2 for tests vs production DB

Asked at: Common · **Difficulty:** Easy · **Topic:** Test setup

The question: "Why would you use H2 for tests when prod is Postgres?"

The 30-second answer: H2 in-memory is fast and isolated — each test class spins up a fresh DB, no shared state. BUT: SQL dialect differences leak (sequences, JSON types, window functions). Best practice: use Testcontainers to run real Postgres for tests. H2 is a holdover.

Step-by-step thought process: 1. "H2 in-memory: zero infrastructure, fast startup (milliseconds), automatic schema management. Easy." 2. "Problem: H2's SQL dialect $\neq$ Postgres. JSON column type, window function syntax, sequence behavior, `LIMIT ... OFFSET` semantics — all subtly different." 3. "Result: tests pass on H2, prod breaks. False confidence is worse than no test." 4. "Better: Testcontainers (Java library that launches Docker containers in tests). `@Container PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:15");` . Real Postgres, isolated per test class." 5. "Cost: Docker required in CI (most CIs support it). Test startup is slower (seconds vs ms). For most projects, the dialect parity is worth it."

Edge cases to mention: - For repository-layer tests, `@DataJpaTest` slice is fast — combined with Testcontainers gives speed + parity - `@DynamicPropertySource` to wire the container's connection details into Spring's environment - Shared containers across tests: Testcontainers supports reusable containers — speeds up dev loops

What NOT to say: - "H2 is fine for any test" — fine until prod breaks - "Just use prod DB for tests" — flaky, slow, not isolated

Common follow-up: "How does Testcontainers wire its connection details into Spring's properties?"

Q39 · @SpringBootTest vs slice tests (@WebMvcTest, @DataJpaTest)

Asked at: Common · **Difficulty:** Medium · **Topic:** Testing

The question: "When would you use @SpringBootTest vs @WebMvcTest vs @DataJpaTest?"

The 30-second answer: `@SpringBootTest` loads the full ApplicationContext — slow but realistic, use for end-to-end / integration tests. `@WebMvcTest` loads only MVC infrastructure + named controllers — fast unit-of-controller tests with MockMvc. `@DataJpaTest` loads only JPA/Hibernate + repositories — fast tests with H2 (or Testcontainers).

Step-by-step thought process: 1. " `@SpringBootTest` boots the full app. Use for tests that exercise the system end-to-end. Slow (seconds per test), but realistic." 2.

" `@WebMvcTest(MyController.class)` loads MVC: controllers, JSON converters, validation, security. Does NOT load services / repos — mock them with `@MockitoBean` (Spring Boot 3.4+) or the older `@MockBean` (deprecated since 3.4). Fast." 3. " `@DataJpaTest` loads JPA: entities, repositories, transaction manager. Tests repository methods against a real (or H2) DB. Transactions auto-rollback between tests." 4. "Other slices: `@WebFluxTest` , `@JsonTest` , `@RestClientTest` , `@DataMongoTest` ." 5. "Pattern: slice tests for fast unit-style coverage of each layer. A handful of `@SpringBootTest` integration tests for end-to-end happy paths + critical flows."

Edge cases to mention: - `@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)` boots a real server — for TestRestTemplate / WebTestClient against it - `@MockitoBean` (Spring Boot 3.4+) is the modern replacement for `@MockBean` — replaces a Spring bean with a Mockito mock. `@MockBean` still works but is deprecated. - Test slice tests can be FASTER than `@SpringBootTest` by an order of magnitude

What NOT to say: - "Use @SpringBootTest for everything" — slow, prevents fast feedback loops - "Slice tests are unit tests" — they involve Spring; integration-y

Common follow-up: "How does @MockBean differ from @Mock?"

Q40 · Spring Boot's banner — can it actually be customized?

Asked at: Soft / personality question · **Difficulty:** Easy · **Topic:** Trivia

The question: "How would you change the Spring Boot banner?"

The 30-second answer: Put a `banner.txt` file in `src/main/resources/` . Spring Boot prints it at startup. Variables like `${spring-boot.version}` interpolate. Or disable with `spring.main.banner-mode=off` . Useful trivia question — shows you actually deploy and watch logs.

Step-by-step thought process: 1. " `src/main/resources/banner.txt` — replaces the default banner." 2. "Variables: `${application.title}` (from MANIFEST.MF Implementation-Title), `${application.version}` , `${spring-boot.version}` ." 3. "Color: ANSI color codes work — `${AnsiColor.BLUE}TEXT${AnsiColor.DEFAULT}` ." 4. "Disable: `spring.main.banner-mode=off` . `=log` prints to log instead of stdout." 5. "Common use: team identifier ('Razorpay Order Service v3.2.1'), env tag (DEV / STAGING / PROD)."

Edge cases to mention: - `banner.gif` / `banner.jpg` also supported (rendered as ASCII art) - For ops: stamp the build SHA in the banner via Maven/Gradle filtering — visible in logs - Banner is the only thing printed before logging is configured — useful sanity check

What NOT to say: - "Why would you care about the banner?" — interviewer is checking you've actually run a Spring Boot app - "Banner is unchangeable" — it isn't

Common follow-up: *"What would you put in a banner for a production service?"*

Q41 · `spring.factories` vs `AutoConfiguration.imports` (Spring Boot 2.7+)

Asked at: Senior + Spring-internals · **Difficulty:** Medium · **Topic:** Autoconfig internals

The question: *"What changed in how Spring Boot discovers autoconfigurations in 2.7+?"*

The 30-second answer: Before 2.7, autoconfig classes were listed under

`EnableAutoConfiguration` key in `META-INF/spring.factories`. Spring Boot 2.7 introduced `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (one class name per line). The old key is deprecated since 2.7, removed in 3.0.

Step-by-step thought process: 1. "Pre-2.7: `META-INF/spring.factories` with key `org.springframework.boot.autoconfigure.EnableAutoConfiguration=com.foo.MyAutoConfiguration`" 2. "2.7+: `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` — plain text file, one class name per line. Cleaner, easier to maintain." 3. "`spring.factories` still loads OTHER SPI entries (`EnvironmentPostProcessor`, `FailureAnalyzer`, etc.). Just the autoconfig entry moved." 4. "For your own starters / autoconfig: switch to the new file format. Old format still works but generates deprecation warnings in 2.7+, removed in 3.0." 5. "Trivia question, but shows you've kept up with releases."

Edge cases to mention: - `@AutoConfiguration` annotation (Spring Boot 2.7+) replaces `@Configuration` for autoconfig classes — sets `proxyBeanMethods=false` and applies stricter semantics - Auto-imports respect `@AutoConfigureBefore` / `@AutoConfigureAfter` for ordering - The Spring Boot starter team uses Gradle Imports plugin to auto-generate this file from `@AutoConfiguration` classes

What NOT to say: - "spring.factories is gone" — partial truth; just the autoconfig key - "It's all the same now" — different file path matters for your own starters

Common follow-up: *"How would you migrate a Spring Boot 2.6 custom starter to 3.x?"*

Q42 · Spring Boot's `@SpringBootApplication` composition

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Application bootstrap

The question: "What's `@SpringBootApplication` composed of?"

The 30-second answer: `@SpringBootApplication` = `@SpringBootConfiguration` (a specialized `@Configuration`) + `@EnableAutoConfiguration` + `@ComponentScan`. Convenience meta-annotation — saves writing all three.

Step-by-step thought process: 1. "`@SpringBootConfiguration` : marks the class as the application's primary config. Internally a `@Configuration`. Used by Spring Boot's test slices to locate the config." 2. "`@EnableAutoConfiguration` : triggers the autoconfig import process (Q31)." 3. "`@ComponentScan` : starts scanning from this class's package and below. That's why you typically put `@SpringBootApplication` in the root package — it scans the whole codebase." 4. "Customizing scan: `@SpringBootApplication(scanBasePackages = "com.foo", excludeFilters = ...)` ." 5. "Excluding autoconfig: `@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})` ."

Edge cases to mention: - You CAN'T easily extend `@SpringBootApplication` (annotations don't inherit cleanly) — meta-annotation helps but with caveats - Multiple `@SpringBootApplication` classes confuse component scanning — keep one per app - For modular apps: use `@Configuration` + `@ComponentScan` per module, one `@SpringBootApplication` at the entry point

What NOT to say: - "@SpringBootApplication does magic" — it's just three annotations rolled into one - "Use `@Configuration` instead" — `@Configuration` alone doesn't trigger autoconfig

Common follow-up: "Why does `@SpringBootApplication` usually sit in the root package?"

Q43 · CommandLineRunner and ApplicationRunner

Asked at: Common · **Difficulty:** Easy · **Topic:** Startup hooks

The question: "How do you run code at Spring Boot startup, after the context is ready?"

The 30-second answer: Implement `CommandLineRunner.run(String... args)` or `ApplicationRunner.run(ApplicationArguments args)`. Spring Boot calls them after the `ApplicationContext` is fully initialized, before `SpringApplication.run()` returns. `ApplicationRunner` gives structured access to parsed args.

Step-by-step thought process: 1. "`CommandLineRunner` interface: one method `void run(String... args)`. Spring Boot invokes every `CommandLineRunner` bean at startup, after bean creation." 2. "`ApplicationRunner` : same idea, but the parameter is `ApplicationArguments` — gives `getOptionNames()`, `getOptionValues(name)`, `getNonOptionArgs()`. Parsed view of args." 3. "Order multiple runners with `@Order` or `Ordered` interface." 4. "Use cases: load initial data, warm caches, sanity-check config, register with service discovery." 5. "For more lifecycle events: listen to `ApplicationReadyEvent` (after all runners complete; the app is serving requests)."

Edge cases to mention: - Runners run in the main thread. If they throw, app startup fails (configurable via SpringApplication settings) - Heavy work in runners delays the "app ready" signal — bad for K8s readiness probes - For background warming: use `@Async` + `@EventListener(ApplicationReadyEvent.class)` — let app become ready, warm in background

What NOT to say: - "Use static initializer" — bypasses Spring lifecycle, can't depend on beans - "Put init in @PostConstruct" — bean lifecycle, runs DURING context creation (bean might depend on others not ready)

Common follow-up: "What's the difference between CommandLineRunner and ApplicationReadyEvent?"

Q44 · @SpringBootApplication's main method

Asked at: Easy · **Difficulty:** Easy · **Topic:** Bootstrap

The question: "Walk me through what `SpringApplication.run(MyApp.class, args)` actually does."

The 30-second answer: Creates a `SpringApplication` instance, configures it (logging, ApplicationContextFactory, listeners), and runs it. Run does: start timer, prepare environment, print banner, create ApplicationContext, refresh context (instantiate all beans, fire `ApplicationStartedEvent`), call runners, fire `ApplicationReadyEvent`. Returns the configured ApplicationContext.

Step-by-step thought process: 1. "Constructs `SpringApplication` : detects web vs non-web vs reactive based on classpath." 2. "Prepares environment: loads config from all sources (Q22), determines active profiles." 3. "Prints banner." 4. "Creates ApplicationContext (Annotation-based by default for Spring Boot)." 5. "Refreshes context: bean creation, dependency injection, BeanPostProcessor application, init methods. Fires `ApplicationStartedEvent` ." 6. "Calls `CommandLineRunner` / `ApplicationRunner` beans." 7. "Fires `ApplicationReadyEvent` — app is ready to serve." 8. "Returns ConfigurableApplicationContext. You CAN use this for programmatic shutdown or post-init wiring."

Edge cases to mention: - For headless / batch apps that should exit after running: use `SpringApplication.exit(ctx, ...)` to compute a status code - ApplicationFailedEvent fires if startup fails — register a listener for telemetry - Spring Boot 3+ supports AOT / native compilation; the same flow with AOT-processed beans

What NOT to say: - "It's just JVM startup" — Spring does a lot - "main() is unimportant" — it's the bootstrap; understand what it triggers

Common follow-up: "How would you intercept startup to run a sanity check before any beans are created?"

Q45 · spring-boot-devtools

Asked at: Easy / dev productivity · **Difficulty:** Easy · **Topic:** Dev tooling

The question: "What's *spring-boot-devtools* and what does it actually do?"

The 30-second answer: Dev-only dependency (`spring-boot-devtools`) that auto-restarts the app when classpath files change, reloads templates without restart, disables caching of templates, enables remote debug. Excluded from production JARs automatically — `spring-boot-maven-plugin` strips it.

Step-by-step thought process: 1. "Add `spring-boot-devtools` to dev classpath. Spring Boot detects it and enables developer features." 2. "Auto-restart: watches classpath for changes. When you save a `.java` file (and IDE recompiles), devtools restarts the app — partial restart via two classloaders is faster than a full restart." 3. "LiveReload: integrates with browser plugins to auto-refresh on resource changes." 4. "Disables Thymeleaf / Mustache / FreeMarker template caching." 5. "Production safety: `spring-boot-maven-plugin` package goal excludes devtools from the fat JAR. Won't be active in production deployments."

Edge cases to mention: - Trigger restart manually: touch any classpath file — useful for forcing restart from CLI - Disable restart per-app: `spring.devtools.restart.enabled=false` - Remote dev (uncommon): devtools can connect to a remote app for live restart over network

What NOT to say: - "Use in production" — explicitly disabled there - "Devtools is the same as JRebel" — JRebel does class-reload without restart; devtools does fast restart

Common follow-up: "What's the difference between devtools restart and JRebel?"

Q46 · Spring Boot AOT and native image

Asked at: Forward-looking senior · **Difficulty:** Medium-Hard · **Topic:** Spring 6 / native

The question: "What's Spring AOT processing, and when would you use GraalVM native image?"

The 30-second answer: AOT (Ahead-Of-Time) processing analyzes the Spring app at build time and generates reflection / proxy hints. Combined with GraalVM, you get a native binary: sub-second startup, lower memory, no JVM. Useful for serverless, scale-to-zero, CLI tools. Trade-off: slower build, runtime constraints (reflection, dynamic proxies need hints).

Step-by-step thought process: 1. "Standard Spring uses reflection heavily — at runtime. Slow startup (seconds to load + introspect)." 2. "AOT processing: a build step (Spring Boot's `spring-boot-maven-plugin` `process-aot` goal / Gradle equivalent) analyzes your code, determines what reflection / proxies are needed, generates hints + pre-computed bean definitions." 3. "Native image (GraalVM): with the AOT hints, GraalVM compiles a standalone binary. JVM stripped out. Startup goes from 3s to 50ms. Memory 200MB to 30MB." 4. "Use cases: serverless (Lambda cold starts), scale-to-zero deployments"

(K8s with zero-replica autoscalers), CLI tools that need instant startup." 5. "Cost: build is much slower (minutes), reflection requires explicit hints (`@RegisterReflectionForBinding`), dynamic features (loading a class by name at runtime) need extra config."

Edge cases to mention: - Throughput at peak load is typically slightly LOWER than JIT-compiled JVM — no profile-guided optimizations - Spring Native (the standalone project) was merged into Spring 6 as official AOT support - `spring-boot-maven-plugin`'s `build-image` goal uses Paketo buildpacks (via the `pack` CLI under the hood) to produce a container image, native or JVM

What NOT to say: - "Native image is just faster" — different tradeoffs (startup vs peak throughput) - "GraalVM is plug-and-play" — needs careful config, especially for reflection / classpath scanning

Common follow-up: "What's the trade-off between native image and a JIT-compiled long-running JVM?"

Q47 · Spring Boot's @Conditional ecosystem

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Conditional config

The question: "Beyond `ConditionalOnMissingBean`, what other conditional annotations does Spring Boot ship?"

The 30-second answer: `@ConditionalOnClass`, `@ConditionalOnMissingClass`, `@ConditionalOnBean`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`, `@ConditionalOnExpression` (SpEL), `@ConditionalOnWebApplication`, `@ConditionalOnNotWebApplication`, `@ConditionalOnJava` (version check), `@ConditionalOnResource` (classpath resource exists), `@ConditionalOnSingleCandidate`. All evaluated at config-time.

Step-by-step thought process: 1. "All implemented as `@Conditional` with specific `Condition` classes under the hood." 2. " `@ConditionalOnClass` : only register if the named class is on classpath. Used to gate auto-configs (e.g. `DataSource` auto-config gated on `DataSource` class being present)." 3. " `@ConditionalOnProperty(name = \"feature.x.enabled\", havingValue = \"true\", matchIfMissing = false)` — feature flags via config." 4. " `@ConditionalOnSingleCandidate(MyType.class)` — only if exactly one bean of `MyType` exists. Avoids ambiguity." 5. " `@ConditionalOnExpression(\"${env} == 'prod'\")` — SpEL-based; flexible but harder to debug." 6. "Combine: multiple `@ConditionalOn*` on one class. All must match."

Edge cases to mention: - Custom conditions: implement `Condition.matches(...)`; use `ConditionContext` to inspect environment, classpath, beans - Negative conditions: `@ConditionalOnMissingClass`, `@ConditionalOnMissingBean` - Order matters when combined with `@AutoConfigureBefore` / `@AutoConfigureAfter` — earlier autoconfigs influence what later ones see

What NOT to say: - "Profiles do the same thing" — different tools; profiles for env, conditionals for feature presence / classpath - "All evaluated at runtime" — wrong, config-time

Common follow-up: "Show me how you'd gate a bean on 'feature flag X is true'."

Q48 · Spring Boot test slices vs @SpringBootTest

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Testing

The question: "What's a Spring Boot test slice? Name three and explain when each is useful."

The 30-second answer: Test slices boot only a slice of the app context — faster than

`@SpringBootTest` . `@WebMvcTest` (controllers + MVC infra), `@DataJpaTest` (JPA + repos + transactional rollback), `@WebFluxTest` (reactive controllers), `@JsonTest` (Jackson + GsonAutoConfiguration), `@RestClientTest` (RestTemplate mock setup), `@DataMongoTest` , `@MockitoBean` ergonomics.

Step-by-step thought process: 1. " `@WebMvcTest(MyController.class)` : loads MVC infrastructure + the specified controller(s). Does NOT load services / repos. Mock them with `@MockBean` . Test with `MockMvc` ." 2. " `@DataJpaTest` : loads JPA + Hibernate + repositories. Wraps each test in a transaction that rolls back on exit. Use with H2 or Testcontainers." 3. " `@JsonTest` : loads Jackson auto-config. Useful for round-trip serialization tests on DTOs." 4. " `@RestClientTest(MyClient.class)` : mocks the HTTP server side, configures RestTemplate / WebClient with a mock. Test API integration without hitting real network." 5. "Reactive: `@WebFluxTest` , `@DataR2dbcTest` , etc."

Edge cases to mention: - Slice tests pick up `@SpringBootApplication` via package scan — keep your test classes in subpackages of the main app - `@AutoConfigureMockMvc` adds MockMvc to a `@SpringBootTest` if you want full context + MockMvc - Slice tests are faster but less realistic — combine with a few full integration tests

What NOT to say: - "Always use slice tests" — full integration tests cover what slices can't (real wiring) - "Slice tests are unit tests" — they involve Spring; closer to integration

Common follow-up: "How would you test only the controller layer without booting the whole app?"

Q49 · Spring Boot's logging configuration

Asked at: Easy / ops rounds · **Difficulty:** Easy · **Topic:** Logging

The question: "How is logging configured in Spring Boot? What does Spring Boot use by default?"

The 30-second answer: Default = Logback. SLF4J as the facade (you log via SLF4J's `Logger`). Configure via `application.properties` (`logging.level.com.foo=DEBUG`) or via `logback-spring.xml` for full control. Out of the box: ANSI-colored console output, sensible defaults.

Step-by-step thought process: 1. "Spring Boot brings Logback (default) + SLF4J via `spring-boot-starter-logging` (transitively included)." 2. "Programmatic logging: `private static final Logger log = LoggerFactory.getLogger(MyClass.class);` then `log.info("hi")` . Use SLF4J's interface; Logback is the impl." 3. "Configure via properties: `logging.level.root=INFO` , `logging.level.com.foo=DEBUG` , `logging.pattern.console=...` . Profile-specific via `application-{profile}.properties` ." 4. "For full control: `logback-spring.xml` in resources. `-spring` suffix lets Spring's profile support work (`<springProfile name="prod">...</springProfile>`)." 5. "Switching to Log4j2: exclude `spring-boot-starter-logging` , include `spring-boot-starter-log4j2` . Useful for high-throughput scenarios."

Edge cases to mention: - Actuator's `/loggers` endpoint lets you change log levels at runtime per package — gold for prod debugging - Lombok's `@Slf4j` annotation generates the `log` field — saves boilerplate - For structured JSON logging (ELK / Datadog): `logstash-logback-encoder` is the standard

What NOT to say: - "Use System.out.println" — never in real code - "Spring Boot uses Log4j" — wrong, Logback by default

Common follow-up: "How would you log JSON structured logs for ingestion into Elasticsearch?"

Q50 · Spring Boot 3.2 virtual threads — enable + caveats

Asked at: Forward-looking 2024-26 rounds · **Difficulty:** Medium · **Topic:** Java 21 + Spring

The question: "How do you enable virtual threads in Spring Boot 3.2+? What breaks?"

The 30-second answer: Set `spring.threads.virtual.enabled=true` (requires Java 21+). Tomcat now uses a virtual-thread-per-request executor. `@Async` and `@Scheduled` also use virtual threads. Caveats: `synchronized` blocks PIN the virtual thread (defeating the purpose); ThreadLocal still leaks across requests in a pool — neither of which the flag fixes for you.

Step-by-step thought process: 1. "Prereqs: Java 21+, Spring Boot 3.2+." 2. "Set `spring.threads.virtual.enabled=true` . Tomcat (default embedded server) switches to virtual-thread-per-request mode. Each HTTP request runs on its own virtual thread." 3. " `@Async` and `@Scheduled` also pick up virtual-thread executors automatically when the flag is set." 4. "Win: massive concurrency for IO-bound workloads. A thousand parallel slow DB calls don't block your Tomcat's request pool anymore." 5. "Caveats: `synchronized` blocks PIN the virtual thread to its carrier (can't unmount on blocking IO inside synchronized). Use `ReentrantLock` instead. ThreadLocal cleanup is still your responsibility (Q21 in `01 Top 50`)."

Edge cases to mention: - For CPU-bound work: virtual threads don't help. Stick with bounded pools. - WebFlux + virtual threads: less synergy — WebFlux is already non-blocking via Netty. Virtual threads benefit Tomcat (blocking-stack) more. - For libraries: check if they use `synchronized` internally — JDBC drivers, for instance, varied widely in pinning behavior in 2024-25; mostly cleared up by 2026.

What NOT to say: - "Virtual threads are a free upgrade" — pinning matters; libraries need to be audited - "WebFlux is now obsolete" — both have valid use cases; virtual threads make blocking-stack viable at scale, not necessarily better than reactive

Common follow-up: "How would you detect virtual-thread pinning in production?"

Q51 · Spring Boot's @Async configuration

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Async config

The question: "How would you configure @Async to use a bounded thread pool with custom naming?"

The 30-second answer: Define a `ThreadPoolTaskExecutor` bean named `taskExecutor` (or implement `AsyncConfigurer` and return it from `getAsyncExecutor()`). Set core pool size, max pool size, queue capacity, thread name prefix. Spring's `@Async` methods will use it.

Step-by-step thought process: 1. "Add `@EnableAsync` to a `@Configuration` class." 2. "Define the executor as a bean: `@Bean(name = \"taskExecutor\") public Executor taskExecutor() { ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor(); e.setCorePoolSize(10); e.setMaxPoolSize(50); e.setQueueCapacity(200); e.setThreadNamePrefix(\"async-\"); e.initialize(); return e; }` ." 3. "For multiple executors: name them and reference via `@Async(\"specificExecutorName\")` ." 4. "Bounded queue + sensible max pool = backpressure. When queue fills, default rejection policy is `AbortPolicy` — caller gets exception." 5. "Alternative rejection policies: `CallerRunsPolicy` (run on caller's thread — natural backpressure), `DiscardPolicy` (silent drop — be careful)."

Edge cases to mention: - For virtual threads (Spring Boot 3.2+): no pool config needed; one virtual thread per task - For exception handling on `@Async void` : implement `AsyncUncaughtExceptionHandler` - Don't share one executor across very different workloads — separate them so one slow workload doesn't starve another

What NOT to say: - "Just use the default" — `SimpleAsyncTaskExecutor` (spawns thread per task) is dangerous in prod - "Make it unbounded for safety" — opposite; bounded enforces backpressure

Common follow-up: "How would you handle exceptions in async void methods?"

Q52 · Spring Boot's @ConfigurationProperties + relaxed binding

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Config binding

The question: "What's 'relaxed binding' in Spring Boot config?"

The 30-second answer: Spring Boot binds property names flexibly — `app.email-config.smtp-host`, `app.emailConfig.smtpHost`, `app.email_config.smtp_host`, `APP_EMAIL_CONFIG_SMTP_HOST` all bind to the same `appEmailConfig.smtpHost` field. Lets you use OS-friendly env vars (SCREAMING_SNAKE_CASE) and Java-friendly camelCase from the same config.

Step-by-step thought process: 1. "Property names in different formats all bind to the same target Java property. Kebab-case is canonical in `application.yml`. Camel-case works too. Underscores work. SCREAMING_SNAKE for env vars." 2. "Example: a field `serverPort` accepts `server.port`, `server-port`, `SERVER_PORT`, `server_port`." 3. "Why: enables uniform config across env vars (uppercase + underscores), system properties (typically dots), and properties files (any of the above)." 4. "Particularly important in containerized deployments — set Docker env `MY_APP_DB_URL=...`, your `@ConfigurationProperties(prefix = "my-app.db") class { String url; }` binds correctly." 5. "Documented in Spring Boot's reference: 'Relaxed Binding'."

Edge cases to mention: - Lists: `app.servers[0]=host1` (indexed) or `app.servers=host1,host2,host3` (CSV) - Maps: `app.metadata.key1=value1` binds to `Map<String, String> metadata` - Enum binding: case-insensitive, kebab-case matching to enum names (`enum-value` matches `ENUM_VALUE`)

What NOT to say: - "Property names must match exactly" — they don't - "Use environment variables with dots" — won't work; use underscores

Common follow-up: "How does Spring bind a list of objects from properties?"

Q53 · Spring Boot's external configuration files (application.yml location)

Asked at: Ops rounds · **Difficulty:** Easy-Medium · **Topic:** Config locations

The question: "Where does Spring Boot look for application.yml? Can I override the location?"

The 30-second answer: Default search order: `file:./config/`, `file:./`, `classpath:/config/`, `classpath:/`. First match wins per file. Override with `spring.config.location=...` (replaces defaults) or `spring.config.additional-location=...` (adds to defaults). Useful for `--spring.config.location=file:/etc/app/`.

Step-by-step thought process: 1. "Default locations searched in order. Earlier overrides later." 2. "Containerized apps usually rely on `file:./` (mount config as a file next to the JAR) or env vars." 3. "`spring.config.location=file:/etc/myapp/application.yml` — replaces ALL defaults with just this location. Tightest control but risky (might miss a profile file)." 4. "`spring.config.additional-location=file:/etc/myapp/` — ADDS this location with higher priority than the defaults. Safer." 5. "Spring Boot 2.4+ `spring.config.import=...` — modern way to pull in external sources, including Vault, K8s ConfigMaps, AWS Secrets Manager."

Edge cases to mention: - `spring.config.name=myapp` — search for `myapp.yml` instead of `application.yml` - For multi-tenant / multi-deployment configs: use `spring.config.import` with conditional imports - Some PaaS (Heroku, Cloud Foundry) inject config via env vars — relaxed binding handles them

What NOT to say: - "Spring only looks at application.yml in resources" — wrong; multiple locations searched - "Use a single location always" — needs flexibility for profiles + overrides

Common follow-up: "How would you override a single property without modifying application.yml?"

Q54 · Spring Boot Maven plugin vs Gradle plugin

Asked at: Build / ops rounds · **Difficulty:** Easy-Medium · **Topic:** Build

The question: "What does *spring-boot-maven-plugin* do that plain Maven doesn't?"

The 30-second answer: Repackages your JAR as an executable "fat JAR" with all deps inside, with a custom JarLauncher class that loads nested JARs at runtime. Adds goals: `bootRun` (run app from build), `bootJar` (build the fat JAR), `bootBuildImage` (build Docker image via buildpacks). Gradle plugin (`org.springframework.boot`) does equivalent.

Step-by-step thought process: 1. "Plain `mvn package` produces a JAR with your code but NOT dependencies. Java's classpath needs all the JARs together to run — not portable." 2. " `spring-boot-maven-plugin` `repackage` goal: bundles your JAR + all dependency JARs into one 'fat' JAR. Adds a JarLauncher main class that knows how to load nested JARs." 3. "Run with `java -jar myapp.jar` . No external classpath needed." 4. "Maven goals: `mvn spring-boot:run` (run from source without packaging), `mvn package` (build fat JAR), `mvn spring-boot:build-image` (build OCI image via Paketo buildpacks)." 5. "Gradle equivalent: `bootRun` , `bootJar` , `bootBuildImage` ."

Edge cases to mention: - Layered JARs (Spring Boot 2.3+): split deps from code into separate layers — Docker images cache base layers - For Maven shade plugin users: shade fights with Spring Boot's layered loader; use Spring Boot's plugin instead - Class data sharing (CDS) — Spring Boot 3.3+ supports `-XX:ArchiveClassesAtExit` for faster startup

What NOT to say: - "Use uber-jar / shade plugin" — wrong; Spring Boot's nested JAR loader is different - "Fat JAR is bad practice" — for Spring Boot it's standard; the loader handles it

Common follow-up: "What's a layered JAR and why does it help Docker builds?"

Q55 · Spring Boot's `application.properties` profile-specific files

Asked at: Easy · **Difficulty:** Easy · **Topic:** Profile config

The question: "How do you have profile-specific properties files in Spring Boot?"

The 30-second answer: Name them `application-{profile}.properties` or `application-{profile}.yml`. Active profile is set via `spring.profiles.active`. When that profile is active, `application-{profile}.properties` is loaded ON TOP of `application.properties`. Profile-specific values win.

Step-by-step thought process: 1. "`application.properties` — always loaded. Defaults for all profiles." 2. "`application-dev.properties` — loaded only when `spring.profiles.active=dev`. Values here override `application.properties`." 3. "`application-prod.properties` for prod, `application-staging.properties` for staging, etc." 4. "Multiple profiles active: `spring.profiles.active=prod,with-feature-x` — both files load; later in the list wins on conflict." 5. "For YAML multi-document: one file with `---` separators and `spring:\n config:\n activate:\n on-profile: prod` per section. Useful for keeping all profile configs in one place."

Edge cases to mention: - Spring Boot 2.4+ syntax for in-document profile sections changed; use `spring.config.activate.on-profile` instead of older `spring.profiles` - Profile groups (Spring Boot 2.4+): `spring.profiles.group.prod=prod-db,prod-cache` — activating `prod` activates the group - Default profile (no active profile set) auto-activates beans with `@Profile("default")`

What NOT to say: - "Profiles only affect bean activation" — they affect property files too - "All profile files load together" — only the active one(s)

Common follow-up: "What's a profile group?"

Q56 · Spring Boot's failure analyzers

Asked at: Senior · **Difficulty:** Medium · **Topic:** Startup diagnostics

The question: "What's a FailureAnalyzer in Spring Boot?"

The 30-second answer: A FailureAnalyzer turns startup exceptions into human-readable error messages. When startup fails, Spring Boot dispatches the exception to all registered analyzers — first that handles it generates the friendly output ("Port 8080 already in use. Identify and stop the process..."). Custom analyzers help diagnose app-specific config errors.

Step-by-step thought process: 1. "Default analyzers handle common failures: port-in-use, bean creation errors, missing properties, validation failures." 2. "Output is the friendly red box at startup — `*****\nAPPLICATION FAILED TO START\n*****\n\nDescription: ... \n\nAction: ...`." 3. "Custom: implement `FailureAnalyzer<MyException>`. Register via `META-INF/spring.factories` (key: `org.springframework.boot.diagnostics.FailureAnalyzer`)." 4. "Useful for: app-specific config errors ('Configuration property X is required but missing'), domain

validation ('Database schema X doesn't exist — run migrations')." 5. "Skip a chunk of stack-trace noise for actionable diagnosis."

Edge cases to mention: - See active analyzers via debug logging — `--debug` flag at startup - Spring Cloud and various starters bring their own analyzers for their domain - `AbstractFailureAnalyzer` base class makes implementing one easy

What NOT to say: - "Failure analyzers are bloat" — they save dev time - "Use Splunk" — that's after the fact; `FailureAnalyzer` catches at the source

Common follow-up: "Show me how you'd write a `FailureAnalyzer` for a missing required config property."

Q57 · Spring Boot Web Client (WebClient vs RestTemplate)

Asked at: Common in 2024+ · **Difficulty:** Medium · **Topic:** HTTP client

The question: "Should you use `RestTemplate` or `WebClient` in a new Spring Boot 3 app?"

The 30-second answer: `WebClient`. `RestTemplate` is in maintenance mode (no new features; Spring docs explicitly recommend `WebClient` since 5.0). `WebClient` is reactive but works fine in blocking code via `.block()`. Use for new code. For existing code, no rush to migrate but new endpoints should use `WebClient`.

Step-by-step thought process: 1. "`RestTemplate`: synchronous HTTP client. Simple `restTemplate.getForObject(url, String.class)`. Mature, ubiquitous." 2. "`WebClient`: non-blocking, reactive. From `spring-webflux` (yes, in non-reactive apps too). Returns `Mono` / `Flux`. Call `.block()` if you need blocking." 3. "Spring docs (since 5.0): '`RestTemplate` is in maintenance mode... we recommend `WebClient`.'" 4. "`WebClient` API: `webClient.get().uri(\"/api/foo\").retrieve().bodyToMono(MyDto.class).block()`. More fluent, more features (streaming, backpressure)." 5. "For new code: `WebClient`. For old code: leave `RestTemplate` alone, migrate gradually if you need new features (streaming, async)."

Edge cases to mention: - Spring 6 added `RestClient` — a synchronous fluent API that wraps `WebClient`. Best of both worlds. Preferred for new SYNCHRONOUS code. - For reactive apps (`WebFlux`): `WebClient` is the only choice - Mock with `MockWebServer` (OkHttp) for tests — works with both clients

What NOT to say: - "`RestTemplate` is deprecated" — maintenance mode, not deprecated - "`WebClient` is slower" — not true; comparable or better, depending on workload

Common follow-up: "What's `RestClient` in Spring 6?"

Q58 · Spring Boot's CORS configuration

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Web

The question: "How do you configure CORS in a Spring Boot app?"

The 30-second answer: For all endpoints: `@Configuration` class with a `WebMvcConfigurer` overriding `addCorsMappings`. For specific controllers/methods: `@CrossOrigin` annotation. Spring Security adds another layer — `http.cors()` enables Spring MVC's CORS config; otherwise security filter blocks requests.

Step-by-step thought process: 1. "Define a global CORS config: `@Configuration class WebConfig implements WebMvcConfigurer { @Override public void addCorsMappings(CorsRegistry registry) { registry.addMapping("/api/**").allowedOrigins("https://app.example.com").allowedMethods("GET", "POST"); } }`". 2. "Per-endpoint: `@CrossOrigin(origins = "https://app.example.com") @GetMapping("/foo")`". 3. "With Spring Security: must enable CORS in SecurityFilterChain — `http.cors(Customizer.withDefaults())`. Otherwise security blocks the preflight OPTIONS request." 4. "Watch for `*` in allowed origins + `allowCredentials=true` — these are MUTUALLY EXCLUSIVE per CORS spec. Spring will throw at config time." 5. "Common gotcha: missing OPTIONS preflight handling. Spring's CorsFilter handles this automatically when CORS is configured."

Edge cases to mention: - For reactive (WebFlux): `WebFluxConfigurer.addCorsMappings(...)` — similar API - CORS is browser-enforced; doesn't help against server-to-server attacks - For prod: explicit origins, not `*`. Use multiple origins via list or method config

What NOT to say: - "Just allow `*`" — too permissive; use specific origins - "CORS works automatically" — needs explicit config

Common follow-up: "What's a CORS preflight request?"

Q59 · Spring Boot's Jackson configuration

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Serialization

The question: "How would you customize Spring Boot's JSON serialization (e.g. dates, naming convention)?"

The 30-second answer: Configure via `application.properties` (`spring.jackson.date-format=...`, `spring.jackson.property-naming-strategy=SNAKE_CASE`) for simple cases. For deeper customization: provide an `ObjectMapper` bean or `Jackson2ObjectMapperBuilderCustomizer`. Spring Boot auto-detects.

Step-by-step thought process: 1. "Spring Boot auto-configures Jackson via `JacksonAutoConfiguration`. `ObjectMapper` is built with sensible defaults + your `spring.jackson.*` overrides." 2. "Quick customization: `application.properties`. `spring.jackson.date-format=yyyy-MM-dd`,

`spring.jackson.property-naming-strategy=SNAKE_CASE`, `spring.jackson.default-property-inclusion=NON_NULL`." 3. "For more: `@Bean public Jackson20objectMapperBuilderCustomizer customizer() { return b -> b.modulesToInstall(new JavaTimeModule()).serializerByType(...); }`." 4. "Or define a full `@Bean ObjectMapper` — replaces the auto-configured one entirely." 5. "Java 8 dates (`LocalDate`, `Instant`): `JavaTimeModule` is auto-registered if `jackson-datatype-jsr310` is on classpath (it is by default in Spring Boot)."

Edge cases to mention: - For Kotlin: register `KotlinModule` for proper Kotlin data class / nullability support - For different ObjectMappers per use case (REST vs internal serialization): use `@Qualifier` and define multiple - Failing on unknown properties:

`spring.jackson.deserialization.fail-on-unknown-properties=false` (default true)

What NOT to say: - "Always use the default ObjectMapper" — customization is common and well-supported - "Use Gson instead" — works but breaks the autoconfig assumption; usually not worth it

Common follow-up: "How would you serialize all `BigDecimal` fields as strings to avoid JS precision issues?"

Q60 · @SpringBootTest's webEnvironment options

Asked at: Common in testing rounds · **Difficulty:** Easy-Medium · **Topic:** Testing

The question: "What's the difference between @SpringBootTest webEnvironment options?"

The 30-second answer: `MOCK` (default — mock servlet env, no port), `RANDOM_PORT` (boots real server on random port), `DEFINED_PORT` (real server on the configured port), `NONE` (no web env at all). Use `MOCK` with `MockMvc` for fast tests; `RANDOM_PORT` for full HTTP integration with `TestRestTemplate` or `WebTestClient`.

Step-by-step thought process: 1. " `MOCK` (default): no real servlet container; uses `MockServletContext`. Use with `MockMvc` — `@AutoConfigureMockMvc` provides `MockMvc` bean." 2. " `RANDOM_PORT` : boots full Tomcat (or other) on a random open port. `@LocalServerPort int port` to inject it. Use with `TestRestTemplate` or `WebTestClient` for real HTTP." 3. " `DEFINED_PORT` : boots on the configured `server.port` . Useful for tests that need a predictable port (rare)." 4. " `NONE` : no web env. For non-web app tests where you don't want to even prepare a web context." 5. "Pattern: `MOCK` + `MockMvc` for controller-level tests. `RANDOM_PORT` + `TestRestTemplate` for full HTTP-and-back integration tests. `NONE` for pure business-logic tests."

Edge cases to mention: - `MOCK` is dramatically faster (no socket, no thread pool startup) but doesn't catch infra-level issues - `TestRestTemplate` auto-configures with the right port when `RANDOM_PORT` is used - For `WebFlux`: `WebTestClient` is the equivalent of `MockMvc`

What NOT to say: - "Always use `RANDOM_PORT`" — slow, often unnecessary for unit-of-controller tests - "MOCK and NONE are the same" — `MOCK` gives `MockMvc`; `NONE` doesn't prepare web infra at all

Common follow-up: "When would you choose *MockMvc* over a real HTTP call?"

Section 3 — Spring MVC + REST (30 Qs)

How requests flow through your Spring Boot web app — from *DispatcherServlet* to your *@Controller*, through validation, exception handling, and back as JSON. The 30 below cover the questions that decide whether you've actually built REST endpoints or just glued frameworks together.

Q61 · *DispatcherServlet* — the entry point

Asked at: Common in mid-level Spring rounds · **Difficulty:** Medium · **Topic:** MVC internals

The question: "Walk me through what happens when a GET request hits a Spring MVC endpoint."

The 30-second answer: Request enters *DispatcherServlet* (the front controller). It consults *HandlerMapping* to find the matching *@RequestMapping* method. *HandlerAdapter* invokes the method. Return value is processed by *HandlerMethodReturnValueHandlers* (e.g. *@ResponseBody* → message converter → JSON). Exception thrown → *HandlerExceptionResolvers* (e.g. *@ControllerAdvice*) → error response.

Step-by-step thought process: 1. "*DispatcherServlet* is registered as the front-controller servlet (Spring Boot auto-configures this at `/`)." 2. "On request, it consults *HandlerMapping* beans (*RequestMappingHandlerMapping* for *@RequestMapping*) to find the right handler method." 3. "*HandlerAdapter.handle()* invokes the method — argument resolvers handle *@PathVariable*, *@RequestParam*, *@RequestBody*, etc. Validation runs here if *@Valid* is present." 4. "Return value goes through *HandlerMethodReturnValueHandler* — *@ResponseBody* triggers *HttpMessageConverter* (Jackson) to serialize to JSON." 5. "If anything throws: *HandlerExceptionResolver* chain runs. *@ExceptionHandler* methods in *@ControllerAdvice* beans get the exception." 6. "Response is written back through *HttpServletResponse*."

Edge cases to mention: - *ContentNegotiationManager* picks the response format (JSON vs XML vs HTML) based on Accept header / file extension / parameter - Interceptors (*HandlerInterceptor*) sit between mapping resolution and method invocation — useful for cross-cutting logic before / after - Filters operate at servlet level, before *DispatcherServlet* sees the request — different scope than interceptors

What NOT to say: - "DispatcherServlet does magic" — it's a well-defined pipeline; learn the stages - "Spring MVC and Spring WebFlux are the same" — different stacks; WebFlux uses reactive pipeline, not *DispatcherServlet*

Common follow-up: "What's the difference between a Filter and an Interceptor?"

Q62 · @PathVariable vs @RequestParam

Asked at: Almost every Spring web interview · **Difficulty:** Easy · **Topic:** Request binding

The question: "When would you use @PathVariable vs @RequestParam?"

The 30-second answer: `@PathVariable` extracts a value from the URI path template (`/users/{id}`). `@RequestParam` extracts from the query string (`?status=active`). Path variables identify a resource; query params filter or modify the response. Use either, never both for the same data.

Step-by-step thought process: 1. " `@GetMapping("/users/{id}") public User get(@PathVariable Long id)` — the `id` comes from the URI segment." 2. " `@GetMapping("/users") public List<User> list(@RequestParam(defaultValue="active") String status)` — the `status` comes from `?status=...`." 3. "REST conventions: path variables for resource identity (`/orders/123`), query params for filtering or pagination (`?page=2&size=20`)." 4. "Both support default values, type conversion, and optional/required." 5. "Mismatch in name? `@PathVariable("id") Long userId` lets you bind path `{id}` to a method parameter named differently."

Edge cases to mention: - Optional path variables aren't great REST — use two endpoints instead - Encoding gotchas: `@PathVariable` doesn't decode `+` to space (query params do); reserved characters need explicit encoding - For lists in query params: `?ids=1,2,3` vs `?ids=1&ids=2&ids=3` — Spring handles both, but pick one convention

What NOT to say: - "Just use @RequestParam for everything" — breaks REST convention - "Path variables can't be optional" — they can with multiple `@RequestMapping` paths, but it's cleaner to use separate endpoints

Common follow-up: "How would you handle a query param that's a list of IDs?"

Q63 · @RequestBody and message converters

Asked at: Common in REST-heavy interviews · **Difficulty:** Easy-Medium · **Topic:** Request body

The question: "How does Spring deserialize a JSON request body into a Java object?"

The 30-second answer: `@RequestBody` triggers Spring's `HttpMessageConverter` chain to deserialize the request body. For JSON, `MappingJackson2HttpMessageConverter` (Jackson) handles it. The chosen converter depends on the request's `Content-Type` header. Validation kicks in if `@Valid` is also present.

Step-by-step thought process: 1. "Method signature: `@PostMapping("/users") public ResponseEntity<User> create(@RequestBody UserDto dto)` . Spring sees `@RequestBody` ." 2. "Spring iterates `HttpMessageConverter` beans, finds the first that supports the request's `Content-Type` AND the target type. For JSON requests with a Jackson-compatible target, that's `MappingJackson2HttpMessageConverter` ." 3. "The converter calls Jackson's

`ObjectMapper.readValue()` under the hood." 4. "If the JSON has unknown fields and `FAIL_ON_UNKNOWN_PROPERTIES` is true (default), 400 Bad Request." 5. "Add `@Valid` next to `@RequestBody` to also trigger Bean Validation. `MethodArgumentNotValidException` thrown on validation failure."

Edge cases to mention: - For non-JSON content types (XML, form-encoded), different converters are used — Spring picks via `Content-Type` - Content-Type vs Accept: request body uses Content-Type to deserialize; response uses Accept to pick the response format - Custom converters: implement `HttpMessageConverter` and register via `WebMvcConfigurer.configureMessageConverters`

What NOT to say: - "Use Jackson directly with `ObjectMapper.readValue` in the controller" — defeats Spring's pipeline, harder to test - "@RequestBody only works with JSON" — depends on registered converters

Common follow-up: "How do you customize Jackson behaviour for a single endpoint?"

Q64 · ResponseEntity vs @ResponseBody

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Response handling

The question: "Why use `ResponseEntity` instead of just returning a POJO from a `@RestController`?"

The 30-second answer: A POJO return becomes a 200 OK with the JSON body — fine for happy paths. `ResponseEntity<T>` lets you control status code, headers, AND body — necessary for 201 Created with Location, 404 Not Found, 422 Unprocessable Entity, custom headers, etc. Use `ResponseEntity` when you need anything beyond 200 OK.

Step-by-step thought process: 1. "`@RestController` = `@Controller` + `@ResponseBody` . Every return value is serialized to the body." 2. "Return a POJO: Spring wraps it in 200 OK + JSON body. Simple." 3. "Return `ResponseEntity<T>` : you control status (`ResponseEntity.status(201).body(user)`), headers (`.header("Location", uri)`), or both." 4. "For not-found patterns: `return repo.findById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());` — chained cleanly." 5. "Mix: most methods return POJO (200 OK happy path), exception types are mapped to status codes via `@ControllerAdvice` ."

Edge cases to mention: - For Spring 6: `ProblemDetail` (RFC 7807) is the new standard for error response bodies — pair with `ResponseEntity<ProblemDetail>` - For 204 No Content: `return ResponseEntity.noContent().build();` - For streaming: return `ResponseEntity<StreamingResponseBody>` or `SseEmitter`

What NOT to say: - "Always wrap in `ResponseEntity`" — adds boilerplate for 200 OK responses - "Use `HttpServletResponse` directly" — couples to servlet API, harder to test

Common follow-up: "How would you return a 201 Created with the new resource's URL in the Location header?"

Q65 · @ControllerAdvice and global exception handling

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Exception handling

The question: "Show me how you'd return a consistent error JSON across all controllers."

The 30-second answer: One `@RestControllerAdvice` class with `@ExceptionHandler` methods for each exception type. Return a standard error DTO. Spring picks the most specific handler match. Extend `ResponseBodyExceptionHandler` to also handle MVC framework exceptions (validation, missing params, type mismatches) in the same shape.

Step-by-step thought process: 1. "`@RestControllerAdvice public class GlobalExceptionHandler extends ResponseExceptionHandler` — extend the base to override framework exception handling." 2. "For domain exceptions: `@ExceptionHandler(EntityNotFoundException.class) public ResponseEntity<ErrorDto> handle(EntityNotFoundException ex) { ... }`." 3. "For validation: override `handleMethodArgumentNotValid` (from `ResponseBodyExceptionHandler`) to format `BindingResult.getFieldErrors()` into your error DTO." 4. "For everything else: `@ExceptionHandler(Exception.class) public ResponseEntity<ErrorDto> handleCatchAll(Exception ex) { ... }` — last-resort handler returning 500." 5. "For Spring 6: prefer returning `ProblemDetail` (RFC 7807) — standard error format that clients can parse uniformly."

Edge cases to mention: - Order between multiple advice classes via `@Order` - Scope by package: `@ControllerAdvice(basePackages = "com.foo.api.v2")` — useful for v1 vs v2 different error formats - For reactive (WebFlux): same `@ControllerAdvice` pattern with `Mono<ResponseEntity<...>>` return types

What NOT to say: - "Catch exceptions in each controller method" — duplicated, inconsistent - "Just let Spring's default error page show" — leaks internals to clients

Common follow-up: "What's RFC 7807 ProblemDetail and how do you use it in Spring 6?"

Q66 · Content negotiation

Asked at: Common in API-heavy interviews · **Difficulty:** Medium · **Topic:** REST

The question: "How does Spring decide whether to return JSON, XML, or HTML for the same endpoint?"

The 30-second answer: `ContentNegotiationManager`. Strategies in order: URL extension (`.json` / `.xml`) → query parameter (`format=json`) → Accept header. The matched MIME type

drives which `HttpMessageConverter` serializes the response. In Spring Boot 2.5+, extension and query-param strategies are off by default; only Accept header is used.

Step-by-step thought process: 1. "`ContentNegotiationManager` resolves the request's preferred response type based on configured strategies." 2. "Default strategy in modern Spring Boot: Accept header. `Accept: application/json` → JSON response; `Accept: application/xml` → XML if `MappingJackson2XmlHttpMessageConverter` is on classpath." 3. "Spring iterates registered `HttpMessageConverter` beans, picks the first that supports both the negotiated type AND the return value's class." 4. "To re-enable old behaviours: `spring.mvc.contentnegotiation.favor-parameter=true` + `spring.mvc.contentnegotiation.media-types.json=application/json` to support `?format=json`." 5. "For most APIs in 2026: Accept header only. Cleaner, REST-idiomatic, fewer attack vectors."

Edge cases to mention: - Spring Boot 2.6+ disabled path-extension matching by default (`mvc.pathmatch.matching-strategy=path-pattern` is the new default) - Wildcard support: `Accept: */*` matches anything; `Accept: application/*` matches any JSON/XML/etc. - For multi-format APIs (legacy XML clients + JSON clients), enable both converters; clients self-select via Accept

What NOT to say: - "Use the file extension always" — disabled by default in newer Spring Boot - "Content negotiation is automatic, ignore it" — silently picks wrong format on misconfigured clients

Common follow-up: "What happens if the Accept header doesn't match any registered converter?"

Q67 · Bean Validation in controllers

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Validation

The question: "How do you validate a request body in Spring? What gets thrown on failure?"

The 30-second answer: Annotate the DTO fields with JSR-380 constraints (`@NotBlank`, `@Email`, `@Min`, `@Size`). In the controller, annotate the param with `@Valid` (or `@Validated` for group support). Spring runs validation before the method body — on failure throws `MethodArgumentNotValidException` (request body) or `ConstraintViolationException` (path/query params with `@Validated` on class).

Step-by-step thought process: 1. "Add `spring-boot-starter-validation` — brings Hibernate Validator (JSR-380 reference impl)." 2. "DTO: `class UserCreatedDto { @NotBlank @Size(max=80) String name; @Email String email; @Min(18) int age; }`." 3. "Controller: `@PostMapping public ResponseEntity<User> create(@Valid @RequestBody UserCreatedDto dto)`. Validation runs before method body executes." 4. "On failure: `MethodArgumentNotValidException`. Handle in `@ControllerAdvice` — extract `getBindingResult().getFieldErrors()` for field-level messages." 5. "For path/query validation: `@Validated` on the controller CLASS + constraints inline: `public User get(@PathVariable`

`@Min(1) Long id)`. Throws `ConstraintViolationException` (different exception type — needs its own handler)."

Edge cases to mention: - Custom validators: implement `ConstraintValidator<MyAnnotation, MyType>` + define `@MyConstraint` annotation - Validation groups: `@Validated(CreateGroup.class)` runs only group-tagged constraints — useful for different rules on create vs update - Cross-field validation: class-level constraint (e.g. `@PasswordsMatch`) with a `ConstraintValidator` at the class level

What NOT to say: - "Validate manually in each method" — boilerplate; defeats Bean Validation - "Skip validation if it slows things down" — micro-optimization; constraints are cheap

Common follow-up: "How would you validate that `startDate < endDate` across two fields?"

Q68 · @RestController vs @Controller

Asked at: Common in fresher rounds · **Difficulty:** Easy · **Topic:** Stereotypes

The question: "What's the difference between `@RestController` and `@Controller`?"

The 30-second answer: `@RestController = @Controller + @ResponseBody`. With `@Controller`, return value is interpreted as a view name (looked up by `ViewResolver`). With `@RestController`, return value is serialized via `HttpMessageConverter` (typically to JSON). For REST APIs always use `@RestController`. For server-rendered HTML use `@Controller`.

Step-by-step thought process: 1. "`@Controller` : returned `String` → looked up as a view name (Thymeleaf template, JSP, etc.). Method returning `Model` is enriched and rendered by the view." 2. "`@RestController` : every return value passed through `HttpMessageConverter` → serialized directly. No view layer." 3. "Mixing in one app: legacy MVC + new REST endpoints? Use both annotations as appropriate. Not all methods of a `@Controller` need to render views — annotate specific methods with `@ResponseBody` to opt them out." 4. "For REST-only APIs: always `@RestController`. Saves typing `@ResponseBody` on every method."

Edge cases to mention: - `@RestControllerAdvice` is the rest-flavoured `@ControllerAdvice` - Server-side rendering still has its place (admin panels, simple internal tools) — Thymeleaf + `@Controller` - For mixed apps: separate REST controllers (`/api/**`) from view controllers (`/admin/**`) by package

What NOT to say: - "@RestController doesn't exist; it's just @Controller" — wrong, it's a real annotation with `@ResponseBody` composed in - "Use @Controller for everything" — wrong default for REST APIs

Common follow-up: "How would you have one app serve both HTML and JSON from different endpoints?"

Q69 · HandlerInterceptor vs Filter

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Cross-cutting

The question: "When would you use a Spring HandlerInterceptor vs a Servlet Filter?"

The 30-second answer: Filter operates at the servlet level — sees raw request/response, runs before DispatcherServlet, used for auth, CORS, logging, gzip. HandlerInterceptor is Spring MVC-specific, runs inside DispatcherServlet, has access to the resolved handler method + ModelAndView. Use Filter for app-level concerns, Interceptor for MVC-specific logic.

Step-by-step thought process: 1. "Filter: servlet-level. Implement `Filter.doFilter()`. Configured in web.xml or via `FilterRegistrationBean` in Spring Boot. Runs BEFORE Spring's DispatcherServlet." 2. "Use Filter for: authentication (Spring Security uses filters), CORS, request/response logging, compression, character encoding." 3. "HandlerInterceptor: Spring MVC-specific. Implement `preHandle`, `postHandle`, `afterCompletion`. Configured via `WebMvcConfigurer.addInterceptors`." 4. "Use Interceptor for: MVC-specific logic that needs the resolved handler method — e.g. per-handler rate limiting based on method annotations, logging the controller method name, attaching request-scoped attributes." 5. "Order: Filters → DispatcherServlet → Interceptor `preHandle` → handler method → Interceptor `postHandle` → render view → Interceptor `afterCompletion`."

Edge cases to mention: - For reactive (WebFlux): WebFilter (not the servlet Filter) and WebFluxConfigurer - Spring Security 6 prefers `SecurityFilterChain` configuration — still servlet filters under the hood - Interceptor `postHandle` doesn't run on exceptions; use `afterCompletion` for cleanup that must always run

What NOT to say: - "Filter and Interceptor are the same" — different layers, different capabilities - "Use Filter for everything Spring MVC related" — Interceptor has MVC-specific hooks Filter doesn't

Common follow-up: "How would you implement per-endpoint rate limiting?"

Q70 · Async controllers (Callable, DeferredResult, virtual threads)

Asked at: Senior backend rounds · **Difficulty:** Medium-Hard · **Topic:** Async

The question: "How do you make a Spring MVC controller async? When does it actually help?"

The 30-second answer: Three options. (1) Return `Callable<T>` — Spring executes on a TaskExecutor; releases the servlet thread immediately. (2) Return `DeferredResult<T>` — caller-managed; set the result from another thread when ready. (3) (Modern best) Enable Spring Boot 3.2+ virtual threads (`spring.threads.virtual.enabled=true`) — every request gets a virtual thread, no async syntax needed.

Step-by-step thought process: 1. "Default: servlet thread blocks until the handler returns. With a small thread pool, blocking IO under load causes queueing." 2. "Async option 1: `return Callable<T>` —

Spring's `WebAsyncManager` submits to a `TaskExecutor`; servlet thread is released. Result rejoins the response when ready." 3. "Async option 2: `return DeferredResult<T>` — your code sets `.setResult(value)` from elsewhere (e.g. a message listener). Good for long polling, server-side push patterns." 4. "Modern alternative (Spring Boot 3.2+ + Java 21): virtual threads. Set `spring.threads.virtual.enabled=true`. Every request runs on its own virtual thread; blocking IO unmounts the carrier. Massive concurrency without async syntax." 5. "For reactive: Spring WebFlux + `Mono<T>` / `Flux<T>` returns — different stack, fully non-blocking."

Edge cases to mention: - `Callable` async still uses Spring's thread pool — bound it; default `SimpleAsyncTaskExecutor` spawns new threads per task - `DeferredResult` supports timeout via constructor - Virtual threads + `synchronized` blocks PIN the thread; use `ReentrantLock` to avoid

What NOT to say: - "Async is always faster" — overhead matters; for fast endpoints async is slower - "Use WebFlux for everything" — WebFlux is a different paradigm; virtual threads often a simpler answer

Common follow-up: "What's the difference between virtual threads and Project Reactor?"

Q71 · CORS configuration

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Web

The question: "How would you configure CORS for a Spring Boot app whose frontend is on a different origin?"

The 30-second answer: For all endpoints: implement `WebMvcConfigurer.addCorsMappings` and register origins, methods, headers. For specific endpoints: `@CrossOrigin(origins = "https://app.example.com")` annotation. If Spring Security is enabled, must also enable CORS in the `SecurityFilterChain` or the security filter blocks preflight requests.

Step-by-step thought process: 1. "Global config: `@Configuration class WebConfig implements WebMvcConfigurer { public void addCorsMappings(CorsRegistry r) { r.addMapping("/api/**").allowedOrigins("https://app.example.com").allowedMethods("GET", "POST", "PUT", "DELETE").allowCredentials(true); } }`" 2. "Per-endpoint: `@CrossOrigin(origins = "https://app.example.com") @GetMapping("/foo")`" 3. "With Spring Security: `http.cors(Customizer.withDefaults())` — picks up the `WebMvcConfigurer`'s CORS config and applies before the security filter." 4. "Browser does preflight OPTIONS for non-simple requests (custom headers, non-simple methods). Spring's CORS filter responds automatically." 5. " * for allowed origins is NOT allowed with `allowCredentials=true` — CORS spec rejects this combo."

Edge cases to mention: - For multiple specific origins: `allowedOriginPatterns("https://*.example.com")` since Spring 5.3 (supports wildcards safely) - CORS doesn't help server-to-server; only browser-enforced - Exposed headers must be explicitly listed (`exposedHeaders`) — clients won't see them otherwise

What NOT to say: - "Allow * for safety" — opposite of safe - "Disable Spring Security to fix CORS" — broken; configure CORS properly instead

Common follow-up: "Why does CORS need a preflight request?"

Q72 · WebClient vs RestTemplate vs RestClient

Asked at: Common in 2024-26 · **Difficulty:** Medium · **Topic:** HTTP clients

The question: "For a new Spring 6 app, which HTTP client would you pick — RestTemplate, WebClient, or RestClient?"

The 30-second answer: Spring Framework 6.1 / Spring Boot 3.2 introduced **RestClient** — a synchronous fluent client built on top of WebClient. For new SYNC code: RestClient. For reactive code: WebClient. RestTemplate is in maintenance mode since 2018; don't pick it for new code.

Step-by-step thought process: 1. "RestTemplate: synchronous, classic Spring HTTP client.

`restTemplate.getForObject(url, User.class)` . Maintenance mode since Spring 5.0; no new features." 2. "WebClient: non-blocking, reactive.

`webClient.get().uri(url).retrieve().bodyToMono(User.class).block()` — can be used synchronously via `.block()` ." 3. "RestClient (Spring Framework 6.1 / Spring Boot 3.2+): synchronous fluent API like WebClient but blocking.

`restClient.get().uri(url).retrieve().body(User.class)` . Best of both worlds for sync code." 4. "New code: pick RestClient for sync, WebClient for reactive." 5. "Existing code on RestTemplate: leave it; migrate gradually."

Edge cases to mention: - All three support customization via filters, interceptors, error handling - For testing: MockRestServiceServer (RestTemplate), MockWebServer (WebClient/RestClient), or @RestClientTest slice - For high-throughput async work in non-reactive apps: WebClient + virtual threads is a powerful combo

What NOT to say: - "RestTemplate is deprecated" — maintenance mode, not deprecated - "WebClient is always better" — for blocking call sites in sync code, RestClient is more ergonomic

Common follow-up: "Show me how you'd add a timeout to a RestClient call."

Q73 · @PostMapping vs @RequestMapping

Asked at: Easy / fresher · **Difficulty:** Easy · **Topic:** Routing

The question: "What's the difference between @PostMapping and @RequestMapping?"

The 30-second answer: `@RequestMapping` is the general-purpose annotation — supports any HTTP method via `method = RequestMethod.POST` . `@PostMapping` is a composed annotation:

`@RequestMapping(method = POST)` . Same for `@GetMapping` , `@PutMapping` , `@DeleteMapping` , `@PatchMapping` . Composed annotations are more readable.

Step-by-step thought process: 1. "`@RequestMapping(value = "/users", method = RequestMethod.POST)` — verbose but works for any HTTP verb." 2. "`@PostMapping("/users")` — same thing, half the code. Spring 4.3+ added the composed annotations." 3. "Use `@RequestMapping` at the class level (`@RequestMapping("/api/v1/users")`) for base paths, then composed annotations on methods." 4. "Use `@RequestMapping` directly when you need to handle multiple verbs in one method — rare in REST design."

Edge cases to mention: - Spring 6 added pattern matching via `path-pattern` strategy — slightly different semantics for `{*path}` patterns - For consumes/produces: `@PostMapping(value = "/users", consumes = "application/json", produces = "application/json")` - Method routes by path + method; conflicts at startup → error

What NOT to say: - "PostMapping is faster" — same routing under the hood; no perf difference - "Use RequestMapping everywhere for consistency" — verbose, harder to read

Common follow-up: "How would you handle the same URL for POST and PUT in one method?"

Q74 · Spring HATEOAS — when does it help?

Asked at: Architect / API-design rounds · **Difficulty:** Medium · **Topic:** REST maturity

The question: "What's HATEOAS? Do you use Spring HATEOAS in practice?"

The 30-second answer: HATEOAS = Hypermedia as the Engine of Application State. The idea: clients discover available actions via links in the response, not by hardcoded knowledge of URL structures. Spring HATEOAS makes it easy to add `_links` to your response DTOs. In practice, rarely used outside truly hypermedia-driven APIs.

Step-by-step thought process: 1. "REST maturity model (Richardson): Level 0 (single endpoint), Level 1 (resources), Level 2 (HTTP verbs), Level 3 (hypermedia). Most real-world REST stops at Level 2." 2. "HATEOAS adds links to responses telling clients what they can do next: `{ \"id\": 123, \"_links\": { \"self\": \"/users/123\", \"delete\": \"/users/123\" } }` ." 3. "Spring HATEOAS: extend `EntityModel` or `RepresentationModel`; add links via `linkTo(methodOn(MyController.class).someMethod(id)).withRel(\"self\")` ." 4. "In practice: most APIs use OpenAPI/Swagger docs instead. Frontend clients know their endpoints; HATEOAS adds bytes and complexity for marginal benefit." 5. "When it shines: truly evolvable APIs (HAL, JSON:API), media-rich clients, browsable APIs. Rarely the right call for internal microservices."

Edge cases to mention: - HAL (Hypertext Application Language) is the common HATEOAS media type — `application/hal+json` - For GraphQL-style needs, JSON:API or direct GraphQL is often a better fit than HATEOAS - Spring HATEOAS works fine but the cost/benefit rarely justifies it

What NOT to say: - "HATEOAS is required for REST" — Roy Fielding might argue, but real-world doesn't agree - "Always add HATEOAS for future flexibility" — premature, adds noise

Common follow-up: "What's the alternative to HATEOAS for evolving APIs?"

Q75 · @RestControllerAdvice scope (basePackages, basePackageClasses)

Asked at: Senior · **Difficulty:** Easy-Medium · **Topic:** Exception handling scope

The question: "How would you have v1 and v2 of an API return different error formats?"

The 30-second answer: Two `@RestControllerAdvice` classes, each scoped via `basePackages` or `basePackageClasses` to its API version's controller package. Each defines its own error DTO + handlers. Spring routes exceptions to the right advice based on which controller package threw.

Step-by-step thought process: 1. "`@RestControllerAdvice(basePackages = \"com.foo.api.v1\") class V1ErrorHandler { ... }\"` 2. "`@RestControllerAdvice(basePackages = \"com.foo.api.v2\") class V2ErrorHandler { ... }\"` 3. "Each defines its own `@ExceptionHandler` methods returning the version-specific error DTO." 4. "When v1 controller throws, V1ErrorHandler picks it up. v2 → V2ErrorHandler." 5. "Alternative scoping: `assignableTypes = {V1BaseController.class}` (any controller extending that base), or `annotations = {V1Api.class}` (any controller annotated)."

Edge cases to mention: - For shared exception types (NullPointerException, etc.): add a global advice without scope as fallback - Order between advice classes via `@Order` or implementing `Ordered` - Reactive equivalent: same approach with `@RestControllerAdvice` + `Mono<ResponseEntity<...>>` returns in WebFlux

What NOT to say: - "Put both v1 and v2 in one advice class" — works but harder to maintain when versions diverge - "Hardcode error format per controller method" — defeats the `@ControllerAdvice` abstraction

Common follow-up: "How would you keep one shared catch-all handler across both versions?"

Q76 · MultipartFile upload

Asked at: Common in file-handling rounds · **Difficulty:** Easy-Medium · **Topic:** File upload

The question: "How do you handle file uploads in a Spring Boot REST API?"

The 30-second answer: Parameter type `@RequestParam("file") MultipartFile file`. Request must be `Content-Type: multipart/form-data`. Configure size limits via

`spring.servlet.multipart.max-file-size` and `max-request-size`. Save the file using `file.transferTo(Path)` or stream directly to S3 / cloud storage.

Step-by-step thought process: 1. "Controller: `@PostMapping("/upload") public ResponseEntity<...> upload(@RequestParam("file") MultipartFile file)` ." 2. " `MultipartFile` has `getName()` , `getOriginalFilename()` , `getContentType()` , `getSize()` , `getInputStream()` , `transferTo(Path)` ." 3. "Size limits: `application.properties` — `spring.servlet.multipart.max-file-size=10MB` , `spring.servlet.multipart.max-request-size=50MB` . Default is 1MB / 10MB; too small for many use cases." 4. "For large uploads: stream directly via `getInputStream()` to S3 / GCS — don't load the whole file in memory or disk." 5. "Validation: check content type (`MimeType` parsing), size, filename safety (no path traversal — strip `../`)."

Edge cases to mention: - For multiple files: `@RequestParam("files") MultipartFile[] files` or `List<MultipartFile>` - For metadata fields alongside files: `@RequestPart("metadata") UserDto dto` + `@RequestPart("file") MultipartFile` - Beware: `MultipartFile` content lives in memory or temp disk; clean up if not transferring elsewhere

What NOT to say: - "Load the file into a byte array" — OOM for large files - "Don't bother with size limits in dev" — denial-of-service vector

Common follow-up: "How would you stream a 1 GB upload directly to S3 without buffering?"

Q77 · ServerSentEvents (SseEmitter) and streaming responses

Asked at: Streaming-heavy rounds · **Difficulty:** Medium · **Topic:** Streaming

The question: "How would you push real-time updates from server to browser using REST?"

The 30-second answer: Server-Sent Events (SSE). Controller returns `SseEmitter` — long-lived connection, client subscribes, server sends events via `emitter.send(...)` . Spring keeps the response open until you close or timeout. Lighter than WebSocket for one-way push.

Step-by-step thought process: 1. "Endpoint: `@GetMapping("/stream") public SseEmitter stream() { var emitter = new SseEmitter(60_000L); ... return emitter; }` ." 2. "Server pushes events: `emitter.send(SseEmitter.event().name(\"update\").data(payload));` ." 3. "Client (browser EventSource API): `new EventSource('/stream')` . Receives events as they're sent." 4. "Close: `emitter.complete()` ends cleanly; `emitter.completeWithError(ex)` ends with error. Spring handles timeout." 5. "Send from a separate thread (publisher pattern) — the controller returns immediately; the emitter is held alive until done."

Edge cases to mention: - For reactive: return `Flux<ServerSentEvent<MyDto>>` from a WebFlux controller — cleaner - SSE = one-way (server → client). For bidirectional: WebSocket - Connection limits:

each open SSE holds a thread (unless using async / virtual threads). Don't open thousands without virtual threads

What NOT to say: - "Use WebSocket for everything streaming" — SSE is simpler for one-way push - "Sleep + return chunks" — actually use the async response API, not blocking

Common follow-up: "What's the difference between SSE and WebSocket?"

Q78 · @Validated vs @Valid

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Validation

The question: "When would you use @Validated instead of @Valid?"

The 30-second answer: `@Valid` is JSR-380 standard — triggers validation on the annotated parameter. `@Validated` is Spring's extension — supports validation GROUPS (run only constraints tagged with a specific group) and (in pre-6.1 form) method-level AOP validation. Note: Spring Framework 6.1+ supports method-level validation natively on controllers, throwing `HandlerMethodValidationException` — `@Validated` on the class is no longer required there.

Step-by-step thought process: 1. "`@Valid @RequestBody UserDto dto` — validates all constraints. Throws `MethodArgumentNotValidException` on failure." 2. "`@Validated(CreateGroup.class) @RequestBody UserDto dto` — runs only constraints annotated with `groups = CreateGroup.class`. Useful when create rules differ from update rules." 3. "In DTO: `@NotBlank(groups = CreateGroup.class) String name;` — only validated when `CreateGroup` is the active group." 4. "`@Validated` on a class (controller) + constraints on method parameters enables method-level validation: `public User get(@PathVariable @Min(1) Long id)`. Throws `ConstraintViolationException`." 5. "Both work for request body; `@Validated` adds groups + method-level support."

Edge cases to mention: - Different exception types: `@Valid` → `MethodArgumentNotValidException`; `@Validated` on class → `ConstraintViolationException`. Handle both in `@ControllerAdvice` - For nested validation: `@Valid` on nested fields cascades; `@Validated` doesn't cascade as cleanly

What NOT to say: - "They're identical" — they aren't; group support is the differentiator - "Always use @Valid" — fine default, but `@Validated` adds capabilities

Common follow-up: "How would you have different validation rules for POST vs PUT?"

Q79 · Long-running endpoints — async vs thread blocking

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Scaling

The question: "Your endpoint takes 5 seconds because of a slow external API call. How do you stop blocking your Tomcat thread pool?"

The 30-second answer: Three options in order of modern preference. (1) Spring Boot 3.2+ virtual threads — set `spring.threads.virtual.enabled=true`, the slow blocking call unmounts its virtual thread from the carrier. (2) Async controller with `CompletableFuture` return — offloads to a separate executor. (3) WebFlux + WebClient — fully reactive, no threads waiting on IO.

Step-by-step thought process: 1. "Default Tomcat pool: ~200 threads. 200 concurrent slow requests = pool exhausted, new requests queue." 2. "Option 1 (modern): virtual threads.

`spring.threads.virtual.enabled=true` + Java 21. Each request gets a virtual thread; blocking IO unmounts the carrier; massively scales blocking-stack code." 3. "Option 2 (works on any version): return `CompletableFuture<T>` from controller. Spring runs it on a separate executor; servlet thread is released." 4. "Option 3 (paradigm shift): WebFlux + WebClient — reactive stack. Steeper learning curve; only when you're going all-in." 5. "For mostly-blocking codebases: virtual threads is the no-rewrite path. For greenfield: either virtual threads (simpler) or WebFlux (different paradigm)."

Edge cases to mention: - Virtual threads + `synchronized` pin the carrier — use `ReentrantLock` instead - For JDBC: most drivers had pinning issues in 2024; mostly resolved by 2026 - Mixing reactive and blocking causes problems; pick one stack per service

What NOT to say: - "Increase thread pool size to 1000" — band-aid; backpressure is the real fix - "Switch to WebFlux for any slow endpoint" — paradigm change for marginal benefit if virtual threads work

Common follow-up: "How does Spring Boot's virtual-thread flag interact with `@Async`?"

Q80 · ResourceHandler and static content

Asked at: Easy / ops · **Difficulty:** Easy · **Topic:** Static resources

The question: "How does Spring Boot serve static files like `/index.html` or `/css/app.css`?"

The 30-second answer: Spring Boot auto-configures `/static`, `/public`, `/resources`, and `/META-INF/resources` (in resources/) as static content roots. Anything under those is served at the URL path matching the file's relative path. Customize via `WebMvcConfigurer.addResourceHandlers`.

Step-by-step thought process: 1. "Default roots: `src/main/resources/static`, `/public`, `/resources`, `/META-INF/resources`. Files there are served at their relative URL." 2. " `src/main/resources/static/index.html` is served at `/index.html`. `src/main/resources/static/css/app.css` at `/css/app.css`." 3. "Override: `WebMvcConfigurer.addResourceHandlers(ResourceHandlerRegistry r)` { `r.addResourceHandler("/assets/**").addResourceLocations("classpath:/my-assets/");` } ." 4. "Cache headers: `addResourceHandler(...).setCachePeriod(3600)`

for client caching." 5. "For SPAs: typically all unknown paths route to `/index.html` (so React Router / Vue Router can take over). Configure via a controller that maps `/ {anyPath:[^\\.|.]*}` to `forward:/index.html`."

Edge cases to mention: - Spring Boot's WebJars support: dependencies on classpath like `webjars/bootstrap` are served at `/webjars/bootstrap/...` - Compression: `server.compression.enabled=true` gzips text responses including static content - For large static asset bundles: serve via CDN, not Spring — put static behind CloudFront / Fastly

What NOT to say: - "Spring can't serve static files" — wrong, well-supported - "Use a Tomcat external instance for static" — embedded Tomcat handles it fine

Common follow-up: "How would you serve a React SPA from a Spring Boot app?"

Q81 · OpenAPI / Swagger integration

Asked at: Common in API-heavy rounds · **Difficulty:** Easy-Medium · **Topic:** Documentation

The question: "How would you add OpenAPI documentation to a Spring Boot REST API?"

The 30-second answer: Add `springdoc-openapi-starter-webmvc-ui` dependency. Springdoc scans your `@RestController` annotations and generates OpenAPI 3.0 spec at `/v3/api-docs` + Swagger UI at `/swagger-ui.html`. Annotate methods/DTOs with `@Operation`, `@Schema`, `@Parameter` for richer docs.

Step-by-step thought process: 1. "Add `springdoc-openapi-starter-webmvc-ui` (for Spring Boot 3.x). Brings springdoc-openapi + Swagger UI." 2. "Out of the box: visit `/v3/api-docs` for the JSON spec, `/swagger-ui.html` for the interactive UI." 3. "Springdoc auto-detects: `@RestController`, `@RequestMapping` paths, parameter types, return types, `@Valid` constraints." 4. "Annotate for richer docs: `@Operation(summary = \"Create user\")`, `@Schema(description = \"User name, 1-80 chars\")`, `@Parameter`." 5. "For multi-version APIs: configure multiple `GroupedOpenApi` beans, one per version."

Edge cases to mention: - For reactive: use `springdoc-openapi-starter-webflux-ui` instead - Authentication for Swagger UI: secure via Spring Security; or `springdoc.swagger-ui.enabled=false` in prod - For client SDK generation: `openapi-generator-cli` reads the JSON spec to generate clients in any language

What NOT to say: - "Springfox" — old / unmaintained library for Spring Boot 2.x. For Spring Boot 3.x use springdoc. - "Write OpenAPI yaml by hand" — error-prone, drifts from code; auto-generation is the standard

Common follow-up: "How would you protect Swagger UI in production?"

Q82 · Response compression

Asked at: Ops · **Difficulty:** Easy · **Topic:** Performance

The question: "How would you enable gzip compression for Spring Boot responses?"

The 30-second answer: `server.compression.enabled=true` . Set `server.compression.mime-types` to specify which types to compress (default includes JSON, XML, HTML, JavaScript). Set `server.compression.min-response-size` for the byte threshold (default 2 KB).

Step-by-step thought process: 1. "In `application.properties`: `server.compression.enabled=true` ." 2. "By default, Tomcat compresses responses larger than 2 KB matching common text MIME types." 3. "Configurable: `server.compression.mime-types=application/json,application/xml,text/html` , `server.compression.min-response-size=1024` ." 4. "Browser sends `Accept-Encoding: gzip, deflate, br` ; server picks the highest-preference algorithm it supports." 5. "For static assets, often better to pre-compress and serve `.gz` files directly. For dynamic JSON, on-the-fly gzip works fine."

Edge cases to mention: - Brotli (`br`) is more efficient than gzip but requires plugin; not on by default in Tomcat - For very large responses, streaming compression matters more; for small responses, overhead exceeds savings - Behind a CDN/proxy: compression usually happens at the CDN; don't double-compress

What NOT to say: - "Compression doesn't help APIs" — JSON compresses 70-90%; significant bandwidth savings - "Always compress everything" — small responses (< 2 KB) often net out negative

Common follow-up: "What's the difference between gzip and Brotli?"

Q83 · Spring's RestExceptionHandler hierarchy

Asked at: Senior · **Difficulty:** Medium · **Topic:** Exception design

The question: "Show me a maintainable exception-handling structure for a large Spring app."

The 30-second answer: Define a domain exception hierarchy (e.g. `DomainException` → `NotFoundException`, `ConflictException`, `ValidationException`). Map each leaf type to an HTTP status in a `@RestControllerAdvice` . Use RFC 9457 `ProblemDetail` (Spring 6+) for response bodies. Log + trace at the advice layer, not in controllers.

Step-by-step thought process: 1. "Domain exceptions: `abstract class DomainException` extends `RuntimeException` . Specific: `NotFoundException` , `ConflictException` , `ValidationException` , `AuthorizationException` ." 2. "Centralize mapping: `@RestControllerAdvice` class `ApiExceptionHandler` with one `@ExceptionHandler` per domain exception." 3. "Use Spring 6's `ProblemDetail` : `ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage())` ."

Standardizes error format across endpoints." 4. "Add tracing: log structured (correlationId, exception class, stack trace at WARN/ERROR depending on severity)." 5. "For validation: handle `MethodArgumentNotValidException` separately — extract field errors into a structured response."

Edge cases to mention: - Don't expose stack traces in production responses — log them, return only error code + message - For multi-tenant: include tenant ID in error log - For client-friendly errors: include a stable error CODE (e.g. `\\"ERR_USER_NOT_FOUND\\"`) in the body so clients can switch on it

What NOT to say: - "Let exceptions bubble — Spring returns 500" — leaks internals - "Catch in every controller method" — bloat, inconsistent

Common follow-up: "What's RFC 7807 ProblemDetail format?"

Q84 · WebMvcConfigurer customization

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Customization

The question: "What is WebMvcConfigurer and what would you commonly customize through it?"

The 30-second answer: `WebMvcConfigurer` is the SPI for tweaking Spring MVC defaults — add interceptors, configure CORS, register message converters, customize content negotiation, add resource handlers, configure view resolvers. Implement the interface; Spring picks it up via component scanning.

Step-by-step thought process: 1. "`@Configuration class WebConfig implements WebMvcConfigurer { ... }` — override only the methods you need." 2.

"`addInterceptors(InterceptorRegistry)` — register HandlerInterceptor beans, with path patterns + ordering." 3. "`addCorsMappings(CorsRegistry)` — global CORS config." 4.

"`configureMessageConverters(List<HttpMessageConverter<?>>)` — replace OR `extendMessageConverters` to add to defaults." 5.

"`configureContentNegotiation(ContentNegotiationConfigurer)` — change negotiation strategies." 6. "`addResourceHandlers(ResourceHandlerRegistry)` — additional static resource locations." 7. "`addArgumentResolvers(List<HandlerMethodArgumentResolver>)` — custom argument resolution (e.g. parse a custom annotation into a typed value)."

Edge cases to mention: - For WebFlux: `WebFluxConfigurer` instead — similar API for reactive stack - Annotation-based ENABLE: previously `@EnableWebMvc` was used; in Spring Boot, doing so disables auto-configuration. Avoid unless you really mean to opt out - For most apps: `WebMvcConfigurer` is the right way to extend; `@EnableWebMvc` replaces auto-config entirely

What NOT to say: - "Use `@EnableWebMvc` to extend defaults" — opposite; it replaces them - "WebMvcConfigurer doesn't work in Spring Boot" — it does, and is the recommended extension point

Common follow-up: "What's the difference between `WebMvcConfigurer` and `@EnableWebMvc`?"

Q85 · @RequestMapping path patterns (PathPatternParser)

Asked at: Senior · **Difficulty:** Medium · **Topic:** Routing

The question: "What changed in Spring 5.3+ about path matching?"

The 30-second answer: Spring 5.3+ introduced `PathPatternParser` as the new default path-matching engine, replacing `AntPathMatcher`. `PathPattern` supports a richer syntax (`{*path}` for catch-all suffix), faster matching, and clearer semantics. Spring Boot 2.6+ has it as default.

Step-by-step thought process: 1. "`AntPathMatcher` (legacy): supports `*`, `**`, `{var}`. Some semantic ambiguities (e.g. trailing slash) caused subtle bugs." 2. "`PathPatternParser` (Spring 5.3+): cleaner syntax with `{var}` (single segment), `{*path}` (anything across segments including slashes, at end of pattern), `**` (anything across segments)." 3. "Default since Spring Boot 2.6. Configure: `spring.mvc.pathmatch.matching-strategy=path-pattern` (or `=ant-path-matcher` for legacy)." 4. "Trailing slash matching: `PathPattern` is stricter — `/users` ≠ `/users/`. Configure with `WebMvcConfigurer.configurePathMatch` if you need legacy behavior." 5. "Performance: `PathPattern` is ~30% faster on average."

Edge cases to mention: - Trailing slash strictness can break clients used to legacy permissive behaviour — migrate carefully - Suffix extension matching (`/file.{ext}`) is disabled by default in modern Spring Boot - For Spring Cloud Gateway and Spring Security: `PathPattern` is the canonical syntax

What NOT to say: - "`AntPathMatcher` is removed" — still available; just not the default - "Path matching changes don't break apps" — they can; trailing slash differences are real

Common follow-up: "How do you migrate from `AntPathMatcher` to `PathPattern` safely?"

Q86 · @Component vs @Bean for registering a third-party class

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Bean definition

The question: "You want to register a third-party class (e.g. a `Redisson RedissonClient`) as a Spring bean. `@Component` or `@Bean`?"

The 30-second answer: `@Bean` — because you don't control the class (can't annotate it with `@Component`). Define a `@Bean` method in a `@Configuration` class that constructs and returns the third-party instance. Spring manages its lifecycle.

Step-by-step thought process: 1. "`@Component` requires modifying the class — impossible for third-party JARs." 2. "`@Bean` is the factory pattern: write a method in a `@Configuration` class that returns the instance. Spring calls it once (for singleton scope) and caches." 3. "Example: `@Bean public RedissonClient redissonClient(@Value("${redis.url}") String url) { Config c = new Config(); c.useSingleServer().setAddress(url); return Redisson.create(c); }`" 4. "Spring resolves the method's parameters via DI (so `@Value` for properties, type-based injection for other beans)." 5. "For close on shutdown: implement the

AutoCloseable contract OR use `@Bean(destroyMethod = \"shutdown\")` to call a method on context close.

Edge cases to mention: - For complex construction: `FactoryBean` is older alternative — usually `@Bean` is cleaner - Multiple beans of same type: name them (`@Bean public RedissonClient cacheRedisson()`) and inject by qualifier - Auto-configure check: use `@ConditionalOnMissingBean(RedissonClient.class)` for libraries that should defer to user-defined beans

What NOT to say: - "Make a wrapper class and annotate it with `@Component`" — works but adds indirection - "Just use new in the method that needs it" — bypasses Spring, no lifecycle management

Common follow-up: *"How would you close a third-party resource (e.g. a Kafka producer) on context shutdown?"*

Q87 · Filter chain order

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Filters

The question: *"You have three custom Filters: `AuthFilter`, `RequestLogFilter`, and `RateLimitFilter`. How do you control the order they run in?"*

The 30-second answer: `FilterRegistrationBean.setOrder(int)` — lower number = earlier in the chain. Default order for Spring-managed Filter beans is `Ordered.LOWEST_PRECEDENCE`. For Spring Security filters: they sit at a specific known position (`SecurityProperties.DEFAULT_FILTER_ORDER`).

Step-by-step thought process: 1. "Each Filter bean is wrapped in a `FilterRegistrationBean` for explicit control." 2. " `@Bean public FilterRegistrationBean<AuthFilter> authFilter() { var bean = new FilterRegistrationBean<>(new AuthFilter()); bean.setOrder(1); return bean; }` ." 3. "Lower order runs first. So `AuthFilter` (1) → `RequestLogFilter` (2) → `RateLimitFilter` (3)." 4. "Spring Security's chain runs at order -100 by default. To run before Spring Security, set order < -100. To run after, set > -100." 5. "For `OncePerRequestFilter` (Spring helper): same registration pattern; just extend the base class for the once-per-request guarantee."

Edge cases to mention: - For Spring Boot's `WebSecurityConfiguration`: filters from Spring Security come in via the `SecurityFilterChain`, not as individual beans - For async dispatching: configure `setAsyncSupported(true)` if your filter needs to run on async dispatches too - For path-specific filters: `setUrlPatterns(\"/api/*\")` to limit scope

What NOT to say: - "Order doesn't matter" — auth before logging vs logging before auth changes leak surface - "Use `@Order` annotation only" — works for some cases but `FilterRegistrationBean` is more explicit

Common follow-up: *"How would you make sure your filter runs before Spring Security's filter chain?"*

Q88 · Conditional REST routes

Asked at: Senior · **Difficulty:** Medium · **Topic:** Conditional config

The question: "You want a debug endpoint that's only available in dev. How do you set that up?"

The 30-second answer: Annotate the `@RestController` class with `@Profile("dev")` — only registered when the dev profile is active. Or use `@ConditionalOnProperty(name = "debug.endpoints.enabled", havingValue = "true")` for property-based gating. Pair with secure access in case it slips into another env.

Step-by-step thought process: 1. "`@Profile(\"dev\") @RestController class DebugController { ... }` — controller bean only exists when dev profile is active. In other profiles, endpoints don't even exist." 2. "`@ConditionalOnProperty` lets you gate via config: `@ConditionalOnProperty(name = \"debug.enabled\", havingValue = \"true\")`. Useful if you want a feature flag rather than env-tied behavior." 3. "For safety, even if a debug controller IS active, restrict via Spring Security: `http.authorizeHttpRequests(a -> a.requestMatchers(\"/debug/**\").hasRole(\"ADMIN\"))`." 4. "For actuator-like endpoints: extend `@Endpoint` (Spring Boot 2.0+) — gets actuator's exposure + security model for free."

Edge cases to mention: - `@Profile(\"!prod\")` for "any profile except prod" — useful but rely on having `prod` always set - For per-environment property: use Spring config import + secrets backends - Spring Boot 2.6+ profile groups let you compose: `dev` profile pulls in `dev-db`, `dev-cache`, etc.

What NOT to say: - "Just delete the controller before deploying" — fragile; profiles are the proper abstraction - "Use static methods for debugging" — defeats Spring lifecycle management

Common follow-up: "How would you set a profile for an integration test that activates only that one debug endpoint?"

Q89 · Spring Boot's request/response logging

Asked at: Ops rounds · **Difficulty:** Easy-Medium · **Topic:** Observability

The question: "How would you log every incoming HTTP request + response in a Spring Boot app?"

The 30-second answer: Use Spring's built-in `CommonsRequestLoggingFilter` or `OncePerRequestFilter` subclass. Configure log level to DEBUG for the filter's class. Be careful: don't log body content for endpoints that accept secrets/PII; respect logging-policy.

Step-by-step thought process: 1. "`@Bean public CommonsRequestLoggingFilter requestLoggingFilter() { var f = new CommonsRequestLoggingFilter(); f.setIncludeQueryString(true); f.setIncludePayload(true); f.setMaxPayloadLength(1000); f.setBeforeMessagePrefix(\"REQUEST: \"); return`

`f; }` . " 2. "Set log level: `logging.level.org.springframework.web.filter.CommonsRequestLoggingFilter=DEBUG` ."
 3. "For response logging: implement `OncePerRequestFilter` ; wrap response in `ContentCachingResponseWrapper` ; read body after the chain completes; log." 4. "For structured logging (JSON for Elasticsearch): use Logstash encoder for Logback; include request ID / trace ID / user ID." 5. "Production: log to a structured pipeline (Datadog, Splunk, ELK); don't dump raw bodies — redact PII."

Edge cases to mention: - For high-throughput: log selectively (e.g. errors only, or sample 1% of OK responses) to avoid log volume - Request body logging requires reading the `InputStream`; once read, it's gone unless you wrap in `ContentCachingRequestWrapper` - Distributed tracing: use Micrometer + OpenTelemetry instead of log scraping

What NOT to say: - "Print everything to stdout" — unstructured, expensive at scale - "Don't log requests in production" — you need them for debugging incidents

Common follow-up: "How would you add a request ID that's logged on every line for a given request?"

Q90 · @ModelAttribute (the OTHER way to bind data)

Asked at: Common in legacy / forms rounds · **Difficulty:** Medium · **Topic:** Form binding

The question: "What's `@ModelAttribute` and when would you use it vs `@RequestBody`?"

The 30-second answer: `@ModelAttribute` binds form-data or query-string params to a Java object (field by field). `@RequestBody` deserializes a JSON / XML body. For HTML form submissions (`application/x-www-form-urlencoded` or `multipart/form-data`): `@ModelAttribute` . For REST JSON APIs: `@RequestBody` .

Step-by-step thought process: 1. " `@PostMapping("/submit") public String submit(@ModelAttribute UserDto dto)` — Spring binds form fields to dto fields by name." 2. "For JSON: `@RequestBody` — deserializes the whole body via Jackson." 3. " `@ModelAttribute` also fills the model for the view (in `@Controller` returning a view name)." 4. " `@ModelAttribute` on a method (not parameter): pre-populates model with that method's return value before any handler runs. Useful for dropdowns / lookup data common across views." 5. "For REST APIs: rarely use `@ModelAttribute` . For server-rendered forms: standard."

Edge cases to mention: - File uploads use multipart with `@ModelAttribute` + `MultipartFile` field on the DTO - For "object" query-string binding (no `@RequestBody`): use `@ModelAttribute` to bind multiple query params into one DTO - Binding result: add `BindingResult result` as the next parameter to inspect binding errors

What NOT to say: - "@ModelAttribute is deprecated" — it isn't; just less common in REST APIs - "Use `@RequestBody` for forms" — wrong content type; bodies are typically `application/json`

Common follow-up: "How would you handle a form upload with a file + text fields?"

Section 4 — Spring Security (15 Qs)

Authentication, authorization, JWT, OAuth2, method-level security, Spring Security 6's new DSL. The most-asked-and-most-misunderstood Spring component.

Q91 · Spring Security 6's new DSL (replacing WebSecurityConfigurerAdapter)

Asked at: Universal in Spring backend interviews since 2024 · **Difficulty:** Medium · **Topic:** Security config

The question: "WebSecurityConfigurerAdapter is gone in Spring Security 6. What replaces it?"

The 30-second answer: Component-based config. Define a `@Bean SecurityFilterChain` that takes an `HttpSecurity` and configures via lambdas: `http.authorizeHttpRequests(auth -> auth.requestMatchers("/public/**").permitAll().anyRequest().authenticated()).formLogin(Customizer.withDefaults()).build()`. No more extending `WebSecurityConfigurerAdapter`.

Step-by-step thought process: 1. "Pre-Spring-Security-5.7: extend `WebSecurityConfigurerAdapter` + override `configure(HttpSecurity)`. Deprecated in 5.7, REMOVED in 6.0." 2. "Replacement: define `@Bean SecurityFilterChain` directly. `@Bean public SecurityFilterChain api(HttpSecurity http) throws Exception { http.authorizeHttpRequests(auth -> ...).csrf(AbstractHttpConfigurer::disable).build(); }`" 3. "Lambda DSL is the modern way. The legacy fluent `.and()` chain is deprecated in Spring Security 6 (removed in 7.0) — migrate to lambdas during 6.x." 4. "For `UserDetailsService`: just define a `@Bean UserDetailsService` — Spring picks it up automatically." 5. "For `PasswordEncoder`: define a `@Bean PasswordEncoder` (default factory: `PasswordEncoderFactories.createDelegatingPasswordEncoder()` — `DelegatingPasswordEncoder` using BCrypt for new hashes)." 6. "For `csrf()`: `csrf().disable()` or `csrf(AbstractHttpConfigurer::disable)`." 7. "For `formLogin()`: `formLogin(Customizer.withDefaults())`." 8. "For `logout()`: `logout().logoutUrl("/logout")`." 9. "For `oauth2()`: `oauth2Login()` or `oauth2ResourceServer()`." 10. "For `jwt()`: `oauth2ResourceServer().jwt()`." 11. "For `session()`: `session().sessionFixate().cookie()`." 12. "For `headers()`: `headers().frameOptions().disable()`." 13. "For `cors()`: `cors().allowedOrigins()`." 14. "For `security()`: `security().authorize()`." 15. "For `requestMatchers()`: `requestMatchers().requestMatchers()`." 16. "For `permitAll()`: `permitAll()`." 17. "For `authenticated()`: `authenticated()`." 18. "For `build()`: `build()`." 19. "For `csrf(AbstractHttpConfigurer::disable)`: `csrf(AbstractHttpConfigurer::disable)`." 20. "For `csrf().disable()`: `csrf().disable()`." 21. "For `csrf().disable().build()`: `csrf().disable().build()`." 22. "For `csrf().disable().build().build()`: `csrf().disable().build().build()`." 23. "For `csrf().disable().build().build().build()`: `csrf().disable().build().build().build()`." 24. "For `csrf().disable().build().build().build().build()`: `csrf().disable().build().build().build().build()`." 25. "For `csrf().disable().build().build().build().build().build()`: `csrf().disable().build().build().build().build().build()`." 26. "For `csrf().disable().build().build().build().build().build().build()`: `csrf().disable().build().build().build().build().build().build()`." 27. "For `csrf().disable().build().build().build().build().build().build().build()`: `csrf().disable().build().build().build().build().build().build().build()`." 28. "For `csrf().disable().build().build().build().build().build().build().build().build()`: `csrf().disable().build().build().build().build().build().build().build().build()`." 29. "For `csrf().disable().build().build().build().build().build().build().build().build().build()`: `csrf().disable().build().build().build().build().build().build().build().build().build()`." 30. "For `csrf().disable().build().build().build().build().build().build().build().build().build().build()`: `csrf().disable().build().build().build().build().build().build().build().build().build().build()`."

Edge cases to mention: - Multiple `SecurityFilterChain` beans: order them with `@Order` for different URL patterns (e.g. one for `/api/**` JWT-based, another for `/admin/**` form-based) - `csrf(...)` must be explicitly disabled for stateless REST APIs (default is enabled — fine for browser forms, broken for JWT APIs) - For test: `@WithMockUser` annotation + `@WebMvcTest` slice tests

What NOT to say: - "Extend `WebSecurityConfigurerAdapter`" — removed in Spring Security 6 - "Use XML config" — XML config still exists but lambda DSL is the modern way

Common follow-up: "How would you have one auth method (JWT) for `/api/` and another (form) for `/admin/`?"

Q92 · Authentication vs Authorization

Asked at: Common, all levels · **Difficulty:** Easy · **Topic:** Security fundamentals

The question: "What's the difference between authentication and authorization in Spring Security?"

The 30-second answer: Authentication = "who are you?" — verifying identity via credentials (password, token). Authorization = "what can you do?" — deciding whether the authenticated user has permission for the requested action. AuthN comes first; AuthZ uses the AuthN result to decide.

Step-by-step thought process: 1. "Authentication: receive credentials (login form, JWT, API key), verify via `AuthenticationManager`, populate `SecurityContext` with an `Authentication` object containing principal + granted authorities." 2. "Authorization: on each request, Spring checks if the `SecurityContext`'s authorities satisfy the access rules. Configured via `authorizeHttpRequests` (URL-based) or `@PreAuthorize` (method-based)." 3. "They're separated cleanly: AuthN is a filter step; AuthZ happens via `AuthorizationManager` (or legacy `AccessDecisionManager`) at access time." 4. "Spring Security's flow: `SecurityFilterChain` → AuthN filter (JWT, Basic, Form) → AuthZ at the resource."

Edge cases to mention: - Anonymous access is a special kind of AuthN —

`AnonymousAuthenticationToken` is set even for unauthenticated users - Method-level `@PreAuthorize(\"hasRole('ADMIN')\")` evaluates against the `SecurityContext` at method invocation - For multi-tenant authZ: include tenant ID in the Authentication's details + custom `AuthorizationManager`

What NOT to say: - "Same thing" — they're distinct concerns; conflating leads to bad security design - "Authorization happens at login" — wrong; it happens per-request

Common follow-up: "How does `@PreAuthorize` check authorities?"

Q93 · JWT vs session-based auth

Asked at: Universal · **Difficulty:** Medium · **Topic:** Auth tokens

The question: "When would you pick JWT over session-based authentication?"

The 30-second answer: JWT for stateless distributed services where session storage adds latency / coordination problems. Session for web apps where you control browser cookies + need easy invalidation. JWT is harder to revoke (need a deny-list); sessions are easy. Pick JWT for APIs, session for web UIs.

Step-by-step thought process: 1. "Session: server keeps session state (in-memory, Redis, DB). Browser cookie → session ID → looks up state. Easy revocation: delete the session." 2. "JWT: signed token containing claims. Server doesn't need state — verifies signature + reads claims. Stateless." 3. "JWT pros: no shared session store, horizontally scalable, works across services. Cons: hard to revoke (need

deny-list of revoked tokens), tokens carry data so size matters." 4. "Session pros: easy revocation, smaller bandwidth, well-tested for browser-driven UIs. Cons: shared session store (sticky sessions or distributed cache)." 5. "For mobile/SPA REST APIs: JWT (often paired with refresh tokens). For server-rendered web apps: session. For microservices internal: mutual TLS or short-lived JWT."

Edge cases to mention: - For JWT revocation before expiry: deny-list in Redis with TTL = token's remaining lifetime - Don't put sensitive data in JWT claims (visible to anyone who has the token) — it's signed, not encrypted - Refresh tokens + short-lived access tokens is the modern pattern

What NOT to say: - "JWT is more secure" — different threat model; both can be secure - "Session doesn't scale" — Redis-backed sessions scale to millions of users fine

Common follow-up: "How would you handle JWT revocation before the token expires?"

Q94 · OAuth2 Resource Server in Spring Boot 3

Asked at: Common in API-heavy senior rounds · **Difficulty:** Medium-Hard · **Topic:** OAuth2

The question: "How would you set up a Spring Boot 3 app as an OAuth2 Resource Server with JWT validation?"

The 30-second answer: Add `spring-boot-starter-oauth2-resource-server`. Configure issuer URI: `spring.security.oauth2.resourceserver.jwt.issuer-uri=https://auth.example.com/realms/main`. Spring auto-configures JWT decoder + signature verification + token validation. Customize authority extraction with a `JwtAuthenticationConverter` bean.

Step-by-step thought process: 1. "Dependency: `spring-boot-starter-oauth2-resource-server`." 2. "In `application.properties`: `spring.security.oauth2.resourceserver.jwt.issuer-uri=https://auth.example.com/.well-known/openid-configuration`. Spring fetches OIDC discovery → JWKS URL → public keys." 3. "In `SecurityFilterChain`: `http.oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()))`." 4. "Token sent as `Authorization: Bearer <jwt>` header. Spring decodes, verifies signature against JWKS, validates exp/iss/aud, populates `SecurityContext` with `JwtAuthenticationToken`." 5. "To map JWT claims to Spring authorities: define a `JwtAuthenticationConverter` bean: extract `roles` claim, prefix with `ROLE_`, return `GrantedAuthority` list."

Edge cases to mention: - For opaque tokens (introspection-based): use `opaquetoken` config instead of `jwt` - For multiple issuers: configure multiple decoders via `JwtIssuerAuthenticationManagerResolver` - JWKS is cached in memory; if your IdP rotates keys frequently, configure cache TTL appropriately

What NOT to say: - "Spring can't do OAuth2 resource server" — built-in, well-supported - "Verify JWT manually" — fragile; let Spring do it

Common follow-up: "How does Spring fetch the public keys to verify JWT signatures?"

Q95 · @PreAuthorize and @PostAuthorize

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Method security

The question: "What's @PreAuthorize and how does it differ from URL-based access rules?"

The 30-second answer: `@PreAuthorize` is method-level authorization — checks before the method runs. URL-based rules in `authorizeHttpRequests` are coarser (path-based). `@PreAuthorize` uses SpEL: `@PreAuthorize("hasRole('ADMIN') and #userId == authentication.principal.id")` — supports arguments, properties, method calls.

Step-by-step thought process: 1. "Enable: `@EnableMethodSecurity` on a `@Configuration` class (Spring Security 6 — replaces older `@EnableGlobalMethodSecurity`)." 2. "On a method: `@PreAuthorize(\"hasRole('ADMIN')\")` `public User get(Long id)` — only invoked if `SecurityContext` has `ROLE_ADMIN`." 3. "SpEL is powerful: access method arguments (`#id`), the authentication (`authentication.principal.username`), call methods (`@authz.canEdit(#userId)` invokes a bean named `authz`)." 4. " `@PostAuthorize` runs AFTER the method, with access to the return value via `returnObject` . Useful for filtering: \"only return this if owner == current user.\"" 5. "Combine with `@PreFilter` (filter collection input) and `@PostFilter` (filter collection output)."

Edge cases to mention: - Method security uses AOP — self-invocation bypasses it (same as `@Transactional`) - For high-perf paths: prefer URL-based rules; method security has per-call SpEL evaluation overhead - `@Secured` (older) is equivalent to `@PreAuthorize` for role-only checks but lacks SpEL

What NOT to say: - "Method security is slow" — overhead is real but trivial unless you're at 100k req/sec - "Use @PreAuthorize for everything" — for coarse path-based rules, URL config is cleaner

Common follow-up: "How would you authorize 'user can only edit their own profile'?"

Q96 · CSRF — when on, when off

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Security

The question: "When would you disable CSRF protection in Spring Security?"

The 30-second answer: CSRF protection helps against form-based attacks where a malicious site submits to your app using the user's cookie. Enable it for browser-driven UIs with cookies. Disable it for stateless REST APIs that use JWT/bearer tokens — they're immune to CSRF by design.

Step-by-step thought process: 1. "CSRF (Cross-Site Request Forgery): attacker tricks user's browser into making an authenticated request to your app via a cookie." 2. "Defense: a token Spring generates per session. Form must include it. Without the token, request is rejected." 3. "Spring Security enables CSRF by default for form-based auth (browser cookie sessions)." 4. "REST APIs using JWT (Authorization header, not cookie) are immune to CSRF — the attacker can't read the JWT from another origin, so they can't submit it." 5. "Disable for REST: `http.csrf(AbstractHttpConfigurer::disable)` in `SecurityFilterChain`."

Edge cases to mention: - If your REST API uses cookies (not JWT), don't disable CSRF — you ARE vulnerable - SameSite=Strict cookies are an additional defense (browser-level) - For mixed apps (browser UI + JWT API): use different `SecurityFilterChain` beans

What NOT to say: - "Always disable CSRF for convenience" — vulnerable for cookie-based auth - "JWT doesn't need CSRF protection" — only if JWT is in Authorization header, not in a cookie

Common follow-up: "Why does CSRF matter for cookie-based auth but not for header-based?"

Q97 · Password encoding (BCrypt)

Asked at: Common, all levels · **Difficulty:** Easy-Medium · **Topic:** Password storage

The question: "How should you store passwords in a Spring app?"

The 30-second answer: Never plain. Spring Security 5+ default is `DelegatingPasswordEncoder` — supports `{id}`-prefixed legacy hashes for migration — whose encode algorithm is BCrypt. For pure new-system code you can use `BCryptPasswordEncoder` directly. Inject `PasswordEncoder`; use `passwordEncoder.encode(plainPassword)` to store, `passwordEncoder.matches(plain, stored)` to verify.

Step-by-step thought process: 1. "`@Bean public PasswordEncoder passwordEncoder() { return PasswordEncoderFactories.createDelegatingPasswordEncoder(); }` — returns a `DelegatingPasswordEncoder` that encodes new passwords with BCrypt while still verifying legacy hashes via `{id}`-prefixed encoding. Or `new BCryptPasswordEncoder()` directly if you don't need backward compatibility." 2. "On registration: `repo.save(new User(username, encoder.encode(plainPassword)));`. BCrypt includes salt in the output; same input → different hash every time." 3. "On login: `encoder.matches(submittedPlain, storedHash)` — true if hash matches." 4. "Strength: higher = slower (more secure, more CPU). Default 10 hashes in ~100ms. For high-traffic apps, balance with brute-force defense (rate limiting)." 5. "For migrating from MD5/SHA1/ plain: use `DelegatingPasswordEncoder` — supports multiple encoders, identified by prefix in stored hash. Upgrade users on login."

Edge cases to mention: - BCrypt has a 72-byte input limit (rare in practice but real) - Argon2 is newer and considered slightly stronger; Spring Security supports it (`Argon2PasswordEncoder`) - Never log passwords, even plain ones during auth flow

What NOT to say: - "MD5 is fine" — broken in under a second on modern GPUs - "SHA256 is enough" — too fast; not a password hash. Use BCrypt/Argon2/PBKDF2

Common follow-up: "How would you migrate users from MD5-hashed passwords to BCrypt?"

Q98 · Custom UserDetailsService

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** User loading

The question: "How do you tell Spring Security where users come from (your own DB)?"

The 30-second answer: Implement `UserDetailsService.loadUserByUsername(String username)` — return a `UserDetails` (username, encoded password, granted authorities). Register as a `@Bean`. Spring picks it up; `AuthenticationManager` uses it to authenticate.

Step-by-step thought process: 1. "Define a `UserDetailsService` bean: `@Bean public UserDetailsService userDetailsService(UserRepository repo) { return username -> repo.findByUsername(username).map(this::mapToUserDetails).orElseThrow(() -> new UsernameNotFoundException(username)); }` ." 2. "Map your entity to Spring's `UserDetails` : `User.withUsername(u.username).password(u.encodedPassword).authorities(u.roles).build()` — `org.springframework.security.core.userdetails.User` is a builder for this." 3. "Spring's authentication providers (`DaoAuthenticationProvider`) use it: look up user → check password (encoded vs submitted) → if match, populate `SecurityContext`." 4. "For richer principal: extend `UserDetails` with additional fields (tenantId, locale, etc.) — accessible via `((MyUserDetails) authentication.getPrincipal()).getTenantId()` ."

Edge cases to mention: - `UsernameNotFoundException` is converted to `BadCredentialsException` by Spring Security to avoid leaking which usernames exist - For multi-tenant: include tenant in username lookup OR derive from token claims - Cache `loadUserByUsername` output for high-frequency auth (per-request lookup vs cached)

What NOT to say: - "Hardcode user list in code" — fine for demo, never production - "Use Spring's in-memory user store" — for demos only

Common follow-up: "How would you authenticate against an LDAP server instead?"

Q99 · Spring Security filter chain order

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Filter ordering

The question: "What's the filter chain order in Spring Security 6, and how would you inject a custom filter?"

The 30-second answer: Spring Security's `SecurityFilterChain` runs filters in a fixed order: `SecurityContext` → `CSRF` → `Header writers` → `Logout` → `Authentication filters (Username/Password, OAuth2, JWT)` → `ExceptionTranslationFilter` → `Authorization`. Inject custom filters via `http.addFilterBefore(myFilter, UsernamePasswordAuthenticationFilter.class)`.

Step-by-step thought process: 1. "Spring Security has a documented filter chain — about 15 filters in fixed order. `SecurityContextPersistenceFilter` → `CsrfFilter` → various authentication filters → `AuthorizationFilter`." 2. "To insert your own: `http.addFilterBefore(new MyTokenFilter(), UsernamePasswordAuthenticationFilter.class);` — runs before username/password handling." 3. " `addFilterAfter` runs after a specific filter." 4. " `addFilterAt` REPLACES at a position (use sparingly)." 5. "For JWT auth: typically a custom `OncePerRequestFilter` added before `UsernamePasswordAuthenticationFilter` — parses Authorization header, decodes JWT, populates `SecurityContext`."

Edge cases to mention: - The full filter list: see `FilterOrderRegistration` in Spring Security source - For diagnostics: enable DEBUG logging on `org.springframework.security` — Spring logs each filter's execution - `OncePerRequestFilter` ensures your filter runs once even on async dispatches — usually what you want

What NOT to say: - "Add filter wherever, order doesn't matter" — auth before authorization is critical - "Override the whole chain" — fragile; insert into the existing chain

Common follow-up: "Show me a JWT authentication filter you'd add to the chain."

Q100 · CORS + Spring Security interplay

Asked at: Common · **Difficulty:** Medium · **Topic:** Security + CORS

The question: "Why does CORS sometimes fail even though I configured it in `WebMvcConfigurer`?"

The 30-second answer: Spring Security's filter chain runs BEFORE MVC's CORS handling. The preflight OPTIONS request hits Spring Security first; if not configured, security rejects it (no CORS headers, 401). Fix: `http.cors(Customizer.withDefaults())` — Spring Security delegates to MVC's CORS config.

Step-by-step thought process: 1. "CORS preflight = browser sends OPTIONS request before the actual request for non-simple cross-origin requests." 2. "In Spring MVC alone: `WebMvcConfigurer.addCorsMappings` handles OPTIONS automatically." 3. "With Spring Security: the security filter chain runs BEFORE MVC. If security blocks OPTIONS (no auth header), browser never gets the CORS response, blocks the actual request." 4. "Fix: `http.cors(Customizer.withDefaults());` in `SecurityFilterChain`. This tells security to apply CORS BEFORE auth checks — preflight passes through." 5. "Spring Security delegates to MVC's CORS configuration — your `WebMvcConfigurer` settings still apply."

Edge cases to mention: - For OAuth2 / JWT-only APIs: explicit CORS config needed; default permissive policies don't apply - Browser may cache failed preflights; clear cache when testing CORS changes - For testing: curl with `-H 'Origin: ...' -X OPTIONS` to verify preflight responds correctly

What NOT to say: - "Disable Spring Security to fix CORS" — opposite of what's needed - "CORS doesn't matter for backend" — your frontend caring about CORS is your backend's problem

Common follow-up: "What's the difference between a simple and non-simple CORS request?"

Q101 · Session fixation protection

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Session security

The question: "What's session fixation, and what does Spring do about it?"

The 30-second answer: Session fixation = attacker plants a known session ID on the victim, then takes over once victim authenticates. Spring Security defends by ROTATING the session ID after successful login (default: `migrateSession` strategy — copies old session's attributes to new one).

Step-by-step thought process: 1. "Attack: attacker visits login page, gets sessionId X (assigned by app). Attacker tricks victim into using sessionId X. Victim logs in. App keeps using X. Attacker now controls the authenticated session." 2. "Defense: rotate the session ID at login. Even if attacker knew the pre-login ID, the post-login ID is new." 3. "Spring Security default: `migrateSession` — new session created, old session's attributes copied over." 4. "Alternatives: `newSession` (no copy), `none` (no rotation — vulnerable), `changeSessionId` (Servlet 3.1+ — rotates without creating new session object)." 5. "Configure via `http.sessionManagement(s -> s.sessionFixation(SessionFixationConfigurer::migrateSession));`."

Edge cases to mention: - For JWT auth: no sessions, so fixation isn't possible - `changeSessionId` is more efficient (Servlet 3.1+) — recommended for new apps - Cookie `SameSite=Strict` + `Secure` flags add defense in depth

What NOT to say: - "Sessions aren't vulnerable" — they are if not rotated - "Disable session security for speed" — security trade-off rarely worth it

Common follow-up: "How does Spring Security handle session timeout?"

Q102 · Method security for collections (PreFilter / PostFilter)

Asked at: Senior · **Difficulty:** Hard · **Topic:** Method security

The question: "You want a method that returns a list filtered to only items the current user can see. How?"

The 30-second answer: `@PostFilter("filterObject.owner == authentication.principal.username")` on the method. Spring iterates the return Collection and removes items that don't pass the SpEL predicate. Combine with `@PostAuthorize` for non-collection cases.

Step-by-step thought process: 1. "`@PostFilter` runs AFTER the method returns. Iterates the Collection/Stream/Array; removes items not matching the SpEL predicate." 2. "`filterObject` references each element in the iteration. SpEL: `@PostFilter(\"filterObject.owner == authentication.principal.username\")`." 3. "`@PreFilter` is the input version: filters a Collection parameter before the method runs. `filterObject` refers to each element of the param." 4. "For non-collection: `@PostAuthorize(\"returnObject.owner == authentication.principal.username\")`. Throws `AccessDenied` if predicate fails." 5. "For performance: better to filter in the DB query (`WHERE owner = ?`) than load and filter in app. Use `@PostFilter` only when DB-level filter isn't possible."

Edge cases to mention: - `@PostFilter` on huge collections is slow — filter at DB query whenever possible - For Stream return types: Spring handles it but the stream is materialized to filter; not lazy - Combining `@PreAuthorize` (entry) + `@PostFilter` (exit) for layered security

What NOT to say: - "Filter in the controller" — leaks behavior into transport layer - "Always use `@PostFilter`" — DB filtering is more efficient

Common follow-up: "Why is DB-level filtering preferred over `@PostFilter`?"

Q103 · Role hierarchy

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Authorization

The question: "You want ADMIN to automatically have all USER permissions without listing them everywhere. How?"

The 30-second answer: Define a `RoleHierarchy` bean. In Spring Security 6.3+: `RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER\nROLE_USER > ROLE_GUEST")`. Pre-6.3: instantiate then `setHierarchy(...)`. Then `hasRole('USER')` returns true for ADMIN too. Wire into method security via `MethodSecurityExpressionHandler`.

Step-by-step thought process: 1. "Without hierarchy: ADMIN role doesn't auto-include USER privileges. You'd write `hasAnyRole('ADMIN', 'USER')` everywhere — fragile." 2. "With hierarchy: `ROLE_ADMIN > ROLE_USER > ROLE_GUEST` means ADMIN has USER and GUEST permissions automatically." 3. "Define bean (Spring Security 6.3+): `@Bean public RoleHierarchy roleHierarchy() { return RoleHierarchyImpl.fromHierarchy(\"ROLE_ADMIN > ROLE_USER\nROLE_USER > ROLE_GUEST\"); }`. Pre-6.3: `var rh = new RoleHierarchyImpl(); rh.setHierarchy(\"ROLE_ADMIN > ROLE_USER\nROLE_USER > ROLE_GUEST\"); return rh;`." 4. "For method security (Spring Security 6): `@Bean MethodSecurityExpressionHandler msh(RoleHierarchy rh) { var h = new`

`DefaultMethodSecurityExpressionHandler(); h.setRoleHierarchy(rh); return h; }` 5. "For URL-based: also wire into `WebExpressionAuthorizationManager` configuration."

Edge cases to mention: - Hierarchy is for ROLE_ authorities only — doesn't extend to arbitrary granted authorities (e.g. PERMISSION_*) - Avoid deeply nested hierarchies — gets confusing fast - For complex permissions: use a permission system (PERMISSION_USER_EDIT, etc.) rather than role hierarchy

What NOT to say: - "Just include both roles in every user's authorities" — defeats single-source-of-truth
- "Check role inheritance manually in code" — error-prone

Common follow-up: "How would you implement permission-based authorization instead of role-based?"

Q104 · Remember-me authentication

Asked at: Easy / browser-app rounds · **Difficulty:** Easy-Medium · **Topic:** Browser auth

The question: "How would you implement 'Remember me' so users stay logged in across browser restarts?"

The 30-second answer: Enable Spring Security's remember-me: `http.rememberMe(rm -> rm.key("secret").tokenValiditySeconds(2592000))` . Issues a cookie containing a signed token; on session expiry, the cookie auto-authenticates. Default validity: 14 days.

Step-by-step thought process: 1. "In SecurityFilterChain: `http.rememberMe(rm -> rm.key(\"a-secret-string\").tokenValiditySeconds(2_592_000))` — 30 days." 2. "Login form: add a 'Remember me' checkbox with `name=\"remember-me\"` . Form login picks it up." 3. "On submit: if checked, Spring issues a cookie with a signed token containing username + expiry + signature." 4. "On subsequent visit (session expired): RememberMeAuthenticationFilter sees the cookie, verifies the signature, populates SecurityContext." 5. "Persistent variant: `rm.tokenRepository(myPersistentTokenRepository)` stores tokens in DB — allows revocation."

Edge cases to mention: - The `key` must be the same across deploys; otherwise existing remember-me cookies invalidate - For sensitive operations (password change, payment), require fresh login — `SpEL isFullyAuthenticated()` (not `rememberMe()`) in `authorizeHttpRequests` rules - Logout invalidates the remember-me cookie

What NOT to say: - "Just extend session timeout to 30 days" — hurts users who actually log out - "Don't bother — sessions are good enough" — UX-wise, remember-me is expected

Common follow-up: "What's the difference between `fullyAuthenticated` and `rememberMe` authentication?"

Q105 · Stateless session policy

Asked at: REST/JWT rounds · **Difficulty:** Easy-Medium · **Topic:** Session policy

The question: "You're building a stateless JWT API. How do you stop Spring from creating sessions?"

The 30-second answer: `http.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))`. Spring Security won't create or use sessions for that filter chain. Combined with disabling CSRF (which depends on session-based tokens), you get a truly stateless API.

Step-by-step thought process: 1. "By default, Spring Security creates a session on first authentication. For stateless APIs, you don't want that — every request should be self-contained." 2. "Configure: `http.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))`." 3. "STATELESS means: Spring Security won't create a session, won't store the SecurityContext in one. Each request must carry full auth (JWT in header)." 4. "Pair with: `http.csrf(c -> c.disable())` — CSRF protection is session-based, not needed for stateless APIs." 5. "For mixed apps: separate SecurityFilterChain beans — STATELESS for `/api/**`, normal for `/admin/**`."

Edge cases to mention: - Other policy options: `ALWAYS` (force session), `IF_REQUIRED` (default), `NEVER` (only use if already exists) - For OAuth2 Resource Server pattern: STATELESS is standard - Logout doesn't work the usual way for STATELESS — handle via token revocation / deny-list

What NOT to say: - "Sessions are fine for REST" — wastes memory, breaks horizontal scaling - "Just don't call request.getSession()" — Spring may still create sessions implicitly

Common follow-up: "How would you handle logout for a stateless JWT API?"

Section 5 — Spring Data + JPA (20 Qs)

Repositories, query methods, transactions, paging, audit. The ORM layer most Spring Boot apps actually run on.

Q106 · JpaRepository vs CrudRepository vs PagingAndSortingRepository

Asked at: Common · **Difficulty:** Easy · **Topic:** Spring Data hierarchy

The question: "What's the difference between JpaRepository, CrudRepository, and PagingAndSortingRepository?"

The 30-second answer: `CrudRepository` is the base — basic CRUD operations. `PagingAndSortingRepository` adds paging + sorting (sibling of CrudRepository since Spring Data 3.0). `JpaRepository` extends both with JPA-specific operations (flush, getReferenceById, saveAllAndFlush, batch operations). For JPA apps, just extend `JpaRepository` — strict superset.

Step-by-step thought process: 1. "`CrudRepository<T, ID>` : `save`, `findById`, `findAll`, `count`, `delete`, `existsById`." 2. "`PagingAndSortingRepository<T, ID>` : `findAll(Pageable)` returns `Page`, `findAll(Sort)` returns sorted `Iterable`. In Spring Data 3.0+ this is a sibling of `CrudRepository` (not a subtype); pre-3.0 it extended `CrudRepository`." 3. "`JpaRepository<T, ID>` extends both: adds `flush()`, `getReferenceById` (lazy proxy), `saveAllAndFlush`, `deleteAllInBatch`, `findAll()` returns `List` (not `Iterable`)." 4. "Spring Data 3.0 also added `ListCrudRepository` / `ListPagingAndSortingRepository` returning `List` instead of `Iterable`. `JpaRepository` is unchanged in day-to-day usage." 5. "For JPA: extend `JpaRepository`. For Mongo: `MongoRepository`. For Redis: `KeyValueRepository`. Same pattern, different backends."

Edge cases to mention: - `@Repository` annotation isn't required — Spring auto-detects repository interfaces via component scan - For read-only views: extend `Repository` (the bare interface) and add only methods you need - For DDD aggregates: avoid huge repositories with 30 query methods — split by aggregate root

What NOT to say: - "Extend `Repository` to get less" — works but unnecessary; `JpaRepository` methods don't hurt if unused - "`CrudRepository` is deprecated" — false; still actively maintained

Common follow-up: "How does Spring Data implement these methods without you writing code?"

Q107 · Query method derivation

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Query derivation

The question: "Show me how Spring Data derives queries from method names."

The 30-second answer: Methods like `findByEmail`, `findByStatusOrderByCreatedAtDesc`, `countByActiveTrue` are parsed by Spring Data's `PropertyPath` matcher. It generates JPQL automatically — no `@Query` needed. Works for simple-to-medium queries; falls back to `@Query` for complex ones.

Step-by-step thought process: 1. "`User findByEmail(String email)` → Spring parses '`findBy`' + '`Email`' → generates `SELECT u FROM User u WHERE u.email = :email`." 2. "Multi-field: `findByEmailAndStatus(String email, Status status)` → adds `AND` clause." 3. "Modifiers: `findByAgeGreaterThan`, `findByNameStartingWith`, `findByCreatedAtBetween`, `findByActiveTrue`, `findByEmailNotNull`." 4. "Sorting: `findByStatusOrderByCreatedAtDesc`." 5. "Aggregations: `countBy...`, `existsBy...`, `deleteBy...`." 6. "For complex queries (joins, subqueries): switch to `@Query` with JPQL or native SQL."

Edge cases to mention: - Long method names hurt readability — use `@Query` annotation for anything past 3-4 fields - For projections: define an interface with the fields you want, name it the method (e.g. `List<UserSummary> findByStatus(Status s)`) - `Top10ByOrderByCreatedAtDesc` returns the top 10 — useful for "latest N" patterns

What NOT to say: - "Method names are magic" — well-defined PropertyPath parsing; debuggable -
"Always use @Query for everything" — overkill for simple field lookups

Common follow-up: "What happens if you typo a field name in a method name?"

Q108 · @Query — JPQL vs native SQL

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Query annotation

The question: "When would you use native SQL via @Query vs JPQL?"

The 30-second answer: JPQL works on entities (`SELECT u FROM User u`) — DB-portable, integrates with JPA caching. Native SQL (`@Query(nativeQuery = true)`) lets you use DB-specific features (window functions, CTEs, JSON operators, vendor-specific optimizer hints). Use JPQL by default; native when you need DB power.

Step-by-step thought process: 1. "JPQL: `@Query(\"SELECT u FROM User u WHERE u.lastLogin < :cutoff\") List<User> findInactive(@Param(\"cutoff\") Instant cutoff)` . Operates on entity names, not table names. Portable across DBs." 2. "Native: `@Query(value = \"SELECT * FROM users WHERE last_login < :cutoff\", nativeQuery = true) List<User> findInactive(@Param(\"cutoff\") Instant cutoff)` . SQL syntax tied to your DB." 3. "Use native when: window functions, recursive CTEs, JSONB operators (Postgres), full-text search, vendor-specific hints." 4. "JPQL gets you: entity caching, automatic projection, DB-portable queries." 5. "For dynamic queries: use Criteria API or Spring Data JPA Specifications, not string concatenation."

Edge cases to mention: - Native queries don't auto-update with schema changes — entity-mapped JPQL does - For UPDATE/DELETE: `@Modifying` annotation + transaction context - For projections from native: use a result interface with `@SqlResultSetMapping` or DTO constructor

What NOT to say: - "Always use native for performance" — JPQL is equally performant when properly indexed - "Use string concat to build queries" — SQL injection vector + DB doesn't cache plans

Common follow-up: "How would you handle a UPDATE query with @Query?"

Q109 · Pagination with Pageable

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Paging

The question: "How does Spring Data pagination work? What does it return?"

The 30-second answer: Method takes `Pageable pageable` parameter and returns `Page<T>` . Spring Data runs two queries: one for content (LIMIT/OFFSET applied), one for total count. Page

contains content, total elements, total pages, current page, etc. Construct Pageable via

`PageRequest.of(pageNum, size, sort)`.

Step-by-step thought process: 1. "Repository: `Page<User> findByStatus(Status s, Pageable pageable)` ." 2. "Controller: `@GetMapping(\"/users\") public Page<User> list(@RequestParam Status status, Pageable pageable)` — Spring auto-binds Pageable from `?page=0&size=20&sort=name,asc` ." 3. "Spring Data runs 2 queries: `SELECT ... LIMIT 20 OFFSET 0` for content; `SELECT COUNT()` for total." 4. "Page has: `getContent` (List), `getTotalElements`, `getTotalPages`, `getNumber` (current page), `hasNext`, `hasPrevious`." 5. "For only content (skip count query): use `Slice` instead — has `hasNext` but no total. Cheaper for infinite scroll."*

Edge cases to mention: - Large OFFSET is slow on most DBs (full scan + skip). For deep paging, use cursor-based (`findByIdGreaterThan(lastId)`) - Sort by indexed column for performance - `@PageableDefault(size = 50, sort = \"createdAt\", direction = DESC)` on controller param sets default

What NOT to say: - "Page is just a list" — it has metadata; surface via API to clients - "Always page by OFFSET" — slow at scale; cursor-based for deep pages

Common follow-up: "What's the difference between Page and Slice?"

Q110 · @Transactional propagation deep-dive

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Transactions

The question: "Explain @Transactional propagation. What's the difference between REQUIRED and REQUIRES_NEW?"

The 30-second answer: **REQUIRED** (default) = join existing transaction or create new. **REQUIRES_NEW** = suspend existing, start fresh. **NESTED** = savepoint within existing. The big practical difference: **REQUIRES_NEW** commits/rolls back independently — useful for audit logs that must persist even if outer transaction rolls back.

Step-by-step thought process: 1. "REQUIRED (default): if a transaction is active, join it. Otherwise start new. The transaction encompasses all REQUIRED method calls." 2. "REQUIRES_NEW: ALWAYS start a fresh transaction. Existing transaction (if any) is SUSPENDED until this one completes. Independent commit/rollback." 3. "NESTED: creates a SAVEPOINT within the existing transaction. Can roll back to savepoint without rolling back outer. JDBC-only; not all DBs / drivers support." 4. "Use REQUIRES_NEW when you need a side effect (audit log, metric write) to persist regardless of outer rollback." 5. "Other levels: SUPPORTS (run in existing if any, no tx otherwise), MANDATORY (must be in existing tx, else error), NEVER (error if in tx), NOT_SUPPORTED (suspend tx if exists)."

Edge cases to mention: - **REQUIRES_NEW** is expensive — full new DB connection from the pool. Don't sprinkle widely. - Self-invocation doesn't trigger AOP, so a method's own @Transactional

REQUIRES_NEW call bypasses the proxy - For event-based decoupling: TransactionalEventListener with AFTER_COMMIT phase is often cleaner than REQUIRES_NEW

What NOT to say: - "REQUIRES_NEW is the same as REQUIRED" — opposite of true - "Just use REQUIRED everywhere" — fine for most, but audit/metric flows need REQUIRES_NEW

Common follow-up: *"How would you ensure an audit log persists even if the business transaction rolls back?"*

Q111 · @Transactional readOnly

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Transactions

The question: *"What does @Transactional(readOnly = true) do?"*

The 30-second answer: Hints that the method only reads. Hibernate skips dirty checking (no need to track changes). JDBC drivers may optimize. DB may route to read replicas. Doesn't actually PREVENT writes — entities marked as modified will still flush. Use for query methods; it's a perf hint, not a security guarantee.

Step-by-step thought process: 1. "`@Transactional(readOnly = true)` on a read-only service method." 2. "Hibernate optimization: skips automatic dirty checking; entities loaded in this tx aren't snapshotted for change detection." 3. "JDBC driver optimization: may set `Connection.setReadOnly(true)` — some drivers route to read replicas, skip certain validation." 4. "For Postgres: combines with `BEGIN READ ONLY` — DB itself rejects writes (different layer; this IS enforcement at DB level)." 5. "For Spring Data: derive method names + @Query methods don't auto-set readOnly. You need to declare it explicitly OR via class-level @Transactional(readOnly = true) with @Transactional on write methods overriding."

Edge cases to mention: - For high-read services: `@Transactional(readOnly = true)` at class level + override write methods with `@Transactional` (defaults readOnly to false) - Combined with read-replica routing (Spring's AbstractRoutingDataSource), readOnly drives the data source choice - For Spring 6+ and reactive: similar pattern with `@Transactional` on reactive methods

What NOT to say: - "ReadOnly is enforced; writes fail" — not at JPA level; only at DB level if driver/DB cooperates - "Don't bother with readOnly hints" — measurable perf win on Hibernate-heavy code

Common follow-up: *"How would you route readOnly transactions to a replica DB?"*

Q112 · Spring Data Projections

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** Projections

The question: *"How would you return only certain fields from a query without loading the whole entity?"*

The 30-second answer: Projection interfaces. Define an interface with getter signatures matching the fields you want; Spring Data returns proxies implementing it. Spring queries only those columns, not the whole entity. Closed projection (interface getters only) is most efficient.

Step-by-step thought process: 1. "Define projection interface:

`interface UserSummary { String getName(); String getEmail(); } .` 2. "Repository: `List<UserSummary> findByStatus(Status status)` . Spring detects projection, queries only `name` + `email` columns." 3. "Closed projection: only getter methods → most efficient, only requested columns loaded." 4. "Open projection: uses `@Value("#{target.firstName + ' ' + target.lastName}")` — SpEL expression, loads the full entity (target) and computes." 5. "Class-based projection (DTO): instead of interface, use a DTO class with a constructor matching the field list. JPQL query uses `SELECT new com.foo.UserSummary(u.name, u.email) FROM User u` ."

Edge cases to mention: - Native queries also support projections via `@SqlResultSetMapping` or by returning the projection interface - For very-large entities with only a few fields needed: projections give big perf wins - Don't reuse projection interfaces across very different queries — coupling

What NOT to say: - "Load the full entity and map to DTO in Java" — wasted DB IO + heap - "Use Map" — loses type safety

Common follow-up: "What's the difference between closed and open projections?"

Q113 · Optimistic vs pessimistic locking

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Concurrency

The question: "How would you prevent two users from concurrently updating the same Order row?"

The 30-second answer: Optimistic = use `@Version` field on the entity. On update, JPA checks version matches; if not, throws `OptimisticLockException` — caller decides retry vs error. Pessimistic = `@Lock(LockModeType.PESSIMISTIC_WRITE)` on the query → DB-level row lock for the duration of the transaction. Optimistic for low-contention, pessimistic for high.

Step-by-step thought process: 1. "Optimistic: `@Version private Long version;` on entity. JPA increments on each update; if `WHERE` clause version mismatch (someone else updated), zero rows affected → `OptimisticLockException`." 2. "Caller catches the exception, retries (with the fresh version) or surfaces error to user." 3. "Pessimistic: `@Lock(LockModeType.PESSIMISTIC_WRITE)` `@Query(\"SELECT o FROM Order o WHERE o.id = :id\") Order findForUpdate(@Param(\"id\") Long id)` . Translates to `SELECT ... FOR UPDATE` — DB locks the row until tx commits." 4. "Pick: optimistic for read-heavy with rare conflicts (most apps). Pessimistic for short-lived hot-row updates (inventory decrement, balance debit)." 5. "Pessimistic limits concurrency — only one transaction processes per row at a time. Optimistic allows parallel reads but retries on collision."

Edge cases to mention: - PESSIMISTIC_READ allows other reads but blocks other writes (less common) - For really hot rows: consider not using row locking at all — use an event log + later projection - Optimistic + a retry loop with backoff handles most conflict patterns gracefully

What NOT to say: - "Use synchronized in Java" — doesn't help across instances - "Pessimistic is safer" — slower and limits scale; only use when needed

Common follow-up: "What's the difference between PESSIMISTIC_WRITE and PESSIMISTIC_READ?"

Q114 · Spring Data Auditing

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Auditing

The question: "How would you automatically track createdAt / updatedAt / createdBy / updatedBy on every entity?"

The 30-second answer: Use Spring Data JPA auditing. Add `@EnableJpaAuditing` on a config class. Annotate entity fields with `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`. Implement `AuditorAware<String>` to provide the current user. Spring Data populates fields automatically on save.

Step-by-step thought process: 1. "`@Configuration @EnableJpaAuditing(auditorAwareRef = \"auditorProvider\") class AuditConfig`." 2. "Add fields to entity (or to a `@MappedSuperclass AbstractAuditable` base): `@CreatedDate Instant createdAt;` `@LastModifiedDate Instant updatedAt;` `@CreatedBy String createdBy;` `@LastModifiedBy String updatedBy;`." 3. "Add `@EntityListeners(AuditingEntityListener.class)` on the entity (or base class)." 4. "Provide auditor: `@Bean public AuditorAware<String> auditorProvider() { return () -> Optional.of(SecurityContextHolder.getContext().getAuthentication().getName()); }`." 5. "Now `userRepo.save(user)` auto-populates createdAt+createdBy on insert, updatedAt+updatedBy on update."

Edge cases to mention: - For system-initiated saves (cron jobs, listeners): AuditorAware returns Optional.empty(), or hardcodes "system" - For multi-tenant: include tenant ID in the audit user (createdBy = "tenant-12:user-abc") - For full audit history (not just last modified): use Envers — separate JPA module for entity revisions

What NOT to say: - "Set timestamps manually in service" — repetitive, easy to miss - "DB triggers handle this" — possible but moves logic out of code; harder to test

Common follow-up: "How would you also track WHO last updated each entity?"

Q115 · Spring Data Specifications (dynamic queries)

Asked at: Senior · **Difficulty:** Hard · **Topic:** Dynamic queries

The question: "You need to build a search endpoint with optional filters: name, status, dateRange, region. How?"

The 30-second answer: Spring Data Specifications. Each filter is a `Specification<T>` predicate; compose with `.and()`, `.or()`, `.not()`. Pass to `repository.findAll(spec, pageable)`. Cleaner than JPQL string concatenation, dynamic by design.

Step-by-step thought process: 1. "Repository extends `JpaSpecificationExecutor<User>` — adds `findAll(Specification<T>)` etc." 2. "Build specs as static methods: `public static Specification<User> hasStatus(Status s) { return (root, query, cb) -> cb.equal(root.get(\"status\"), s); }`." 3. "Compose: `Specification<User> spec = Specification.where(hasName(name)).and(hasStatus(status)).and(createdBetween(from, to));`." 4. "For optional filters: `if (name != null) spec = spec.and(hasName(name));`." 5. "Run: `Page<User> page = repo.findAll(spec, PageRequest.of(0, 20));`."

Edge cases to mention: - For complex joins, use `root.join(...)` inside the Specification lambda - Beware: Specifications use Criteria API which is verbose. For very dynamic queries (build SQL from URL params), consider QueryDSL or JOOQ - Pagination still works with Specifications — pass Pageable as usual

What NOT to say: - "Build the JPQL string from params" — SQL injection or fragile string formatting - "Use @Query with all permutations" — combinatorial explosion

Common follow-up: "When would you pick QueryDSL over Specifications?"

Q116 · @EntityGraph for fetch tuning

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Fetch strategy

The question: "You have an entity with several lazy associations. How do you prevent N+1 when fetching them?"

The 30-second answer: `@EntityGraph` on the repository method. Specifies attributes to fetch eagerly for THIS query. Hibernate generates a single JOIN query (or two with batch fetching) instead of N+1 SELECTs. Configurable per query without modifying entity defaults.

Step-by-step thought process: 1. "Problem: `findAll()` returns 100 Users; iterating `user.getOrders()` triggers a SELECT per user → 100 extra queries." 2. "Fix: `@EntityGraph(attributePaths = {\"orders\"}) List<User> findAll();` — Hibernate JOINS orders in the same query." 3. "For multi-level: `attributePaths = {\"orders.items\", \"address\"}`." 4. "Alternative: JPQL `JOIN FETCH` — `@Query(\"SELECT u FROM User u JOIN FETCH u.orders\")`. Same result, different syntax." 5. "For lazy-by-default (recommended)

entities, use *EntityGraph*/*JOIN FETCH* on the queries where you need eager loading. Don't change the entity to *EAGER* everywhere — that breaks other queries."

Edge cases to mention: - For paged queries with *JOIN FETCH* on a collection: Hibernate logs a warning + fetches all rows then paginates in memory. Use `@BatchSize` or split queries instead. -

`EntityGraph.named` references a pre-defined graph on the entity (`@NamedEntityGraph`) - For very wide entity graphs: select multiple, smaller queries can be faster than one mega *JOIN*

What NOT to say: - "Change *FetchType.LAZY* to *EAGER*" — fixes one query, ruins others - "Add `Hibernate.initialize` after the loop" — defeats N+1 mitigation, still issues per-entity queries

Common follow-up: "What's the difference between *fetch=JOIN FETCH* and `@EntityGraph`?"

Q117 · @Transactional rollback rules

Asked at: Common · **Difficulty:** Medium · **Topic:** Transactions

The question: "What exceptions cause `@Transactional` to roll back?"

The 30-second answer: By default, *RuntimeException* and *Error* cause rollback. Checked exceptions DON'T roll back by default. To roll back on a checked exception: `@Transactional(rollbackFor = MyCheckedException.class)` . To NOT roll back on a *RuntimeException*: `@Transactional(noRollbackFor = MyRtException.class)` .

Step-by-step thought process: 1. "Default behavior matches *JTA*: *RuntimeException* and *Error* → rollback. Checked exceptions → commit (counter-intuitive)." 2. "To override: `@Transactional(rollbackFor = {IOException.class, MyCheckedException.class})` — these now cause rollback." 3. " `noRollbackFor` is rarer — only use when you want to commit despite a specific runtime exception (e.g. a 'soft' failure that's logged but tolerable)." 4. "Common gotcha: catching an exception inside the method and not re-throwing → no rollback. Transaction proxy only sees what propagates out." 5. "For mandatory rollback inside the method: `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();` ."

Edge cases to mention: - For programmatic rollback signaling without throwing: `setRollbackOnly()` then return normally — outer commit will roll back - Multi-bean transactions: if any inner method calls `setRollbackOnly` , the entire outer tx rolls back at commit - For tests: `@Transactional` on test methods auto-rolls back; useful for keeping the DB clean

What NOT to say: - "All exceptions roll back" — false; checked don't by default - "Catch and ignore is fine" — silently commits incomplete work

Common follow-up: "How would you force a transaction rollback without throwing?"

Q118 · @DataJpaTest slice

Asked at: Common testing rounds · **Difficulty:** Easy-Medium · **Topic:** Testing

The question: "What does @DataJpaTest do and when would you use it?"

The 30-second answer: @DataJpaTest is a test slice that loads only JPA infrastructure — entities, repositories, transaction manager, EntityManagerFactory. Excludes services, controllers, etc. Each test wraps in a transaction that rolls back on exit (clean DB state per test). Fast.

Step-by-step thought process: 1. "Annotate test class with @DataJpaTest . Spring loads JPA + DataSource + transaction manager + repositories — nothing else." 2. "By default uses an embedded DB (H2 unless excluded). Override with Testcontainers + @AutoConfigureTestDatabase(replace = NONE) for real Postgres." 3. "Each test wraps in a transaction; @Transactional is implicit. On test exit, rollback. No DB pollution between tests." 4. "Inject TestEntityManager for arrange/assert: entityManager.persist(user); entityManager.flush(); ." 5. "Lots of times faster than @SpringBootTest — Spring loads only JPA slice."

Edge cases to mention: - For real DB testing (Postgres specifics): pair with Testcontainers - For repository methods involving native queries: H2 dialect differences may cause false pass/fail - @DataJpaTest doesn't load @Service beans — for end-to-end repo+service test, use @SpringBootTest

What NOT to say: - "Use @SpringBootTest for repos" — slow, loads unrelated config - "DataJpaTest doesn't need a DB" — needs one (embedded or real)

Common follow-up: "How would you make @DataJpaTest use Testcontainers Postgres instead of H2?"

Q119 · Hibernate's first-level cache (Session)

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Caching

The question: "What is Hibernate's first-level cache?"

The 30-second answer: The first-level cache is the JPA EntityManager's identity map within a single transaction. Same entity loaded twice in the same transaction returns the same object (==, not equals). Caches managed entities for the duration of the transaction; cleared on commit/clear.

Step-by-step thought process: 1. "Within one EntityManager: load User(id=5) → Hibernate caches. Load again in same tx → returns the same Java object reference." 2. "Identity map: ensures consistent reads within a tx. Updates to one reference are visible to other accesses of the same entity." 3. "Scope: per-transaction (or per-EntityManager). Not shared across requests." 4. "To bypass cache: entityManager.refresh(entity) — forces a reload from DB." 5. "To clear: entityManager.clear() — useful in long-running batch processing to avoid OOM."

Edge cases to mention: - For `find()` calls: first-level cache hit. For JPQL queries: ALWAYS goes to DB (queries aren't cached at L1) - For long batch jobs (loading 100K entities): flush + clear periodically to free memory - Detached entities (entity loaded then EM closed): not in cache anymore; merge to re-attach

What NOT to say: - "L1 cache spans the app" — wrong; per-EntityManager only - "L1 caches queries" — wrong; entities only

Common follow-up: "What's the second-level cache?"

Q120 · Hibernate second-level cache

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Caching

The question: "What's Hibernate's second-level cache, and when would you turn it on?"

The 30-second answer: Optional cache shared across EntityManagers (process-wide). Entities loaded once, served from cache to all sessions. Backed by a provider (Ehcache, Caffeine, Hazelcast for distributed). Turn on for slow-changing reference data (countries, currencies); avoid for frequently-modified entities.

Step-by-step thought process: 1. "L2 cache: shared cache at process level. EntityManagers across requests share cached entities." 2. "Configure:

`spring.jpa.properties.hibernate.cache.use_second_level_cache=true` + a provider (Ehcache, Caffeine via `hibernate-jcache`). " 3. "Annotate cacheable entities: `@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)` . (Don't confuse with `jakarta.persistence.Cacheable` — different annotation, also needed but separate semantics.)" 4. "Strategies: `READ_ONLY` (immutable data), `READ_WRITE` (uses soft locks), `NONSTRICT_READ_WRITE` (no locks, weak consistency), `TRANSACTIONAL` (JTA-coordinated)." 5. "Good fit: countries, currencies, plan tiers, anything updated rarely. Bad fit: orders, payments, anything updated frequently — cache invalidation overhead kills perf."

Edge cases to mention: - For distributed apps: invalidation across nodes requires a coordinated cache (Hazelcast, Infinispan) - Query cache (separate from entity cache):

`hibernate.cache.use_query_cache=true` + `setHint(QueryHints.HINT_CACHEABLE, true)` per query. Tricky to use correctly. - For most modern apps: Redis-backed caching at the service layer is more flexible than L2

What NOT to say: - "L2 cache always speeds up apps" — wrong cache strategies hurt perf - "L2 is enabled by default" — opt-in

Common follow-up: "How does L2 cache invalidate when an entity is updated?"

Q121 · N+1 query problem

Asked at: Almost universal in JPA interviews · **Difficulty:** Medium · **Topic:** Performance

The question: "What's the N+1 query problem, and how do you fix it in Hibernate?"

The 30-second answer: Loading N entities triggers N+1 queries when iterating lazy associations — one for the parent collection, N for each child fetch. Fixes: JOIN FETCH in JPQL, @EntityGraph on the repository method, @BatchSize annotation, or splitting into two explicit queries.

Step-by-step thought process: 1. "Scenario: `List<User> users = repo.findAll();` returns 100 users. Iterating `user.getOrders().size()` triggers 100 separate SELECTs." 2. "1 query (initial) + N queries (per user's orders) = N+1." 3. "Fix #1: `@Query(\"SELECT DISTINCT u FROM User u JOIN FETCH u.orders\")` — one query with JOIN." 4. "Fix #2: `@EntityGraph(attributePaths = {\"orders\"})` on the repo method — same effect." 5. "Fix #3: `@BatchSize(size = 25)` on the association — Hibernate batches lazy fetches; instead of 100 SELECTs, ~4 SELECTs with IN clauses." 6. "Fix #4: explicit two queries — fetch parents, then fetch all children for those parent IDs."

Edge cases to mention: - For paged queries with JOIN FETCH on a collection: Hibernate WARNS + fetches everything and paginates in memory. Use @BatchSize for paged + lazy collection access - For multi-level associations: `JOIN FETCH u.orders o JOIN FETCH o.items` — be careful of cartesian product blow-up - Use Spring's `JpaSystemException` or hibernate-stats to detect N+1 in dev

What NOT to say: - "Change FetchType.LAZY to EAGER" — fixes this case, ruins everything else - "It's not a problem in dev" — production data volumes turn it into a 10-second response

Common follow-up: "What's wrong with FetchType.EAGER as a default?"

Q122 · @Transactional and Spring's proxy behavior

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** AOP gotchas

The question: "You have a method calling another @Transactional method in the SAME class. Why doesn't the inner one's transaction kick in?"

The 30-second answer: Spring's @Transactional is implemented via AOP proxies. When method A calls method B in the same class, the call goes through `this` — not through the proxy. The proxy never intercepts, so B's @Transactional is ignored. Fix: inject self, extract B to another bean, or use AspectJ (compile-time weaving).

Step-by-step thought process: 1. "Spring creates a CGLIB/JDK proxy wrapping your bean. External callers go through the proxy → AOP advice runs (transaction starts, etc.)." 2. "Self-invocation: `this.methodB()` from `methodA()` → `this` is the ORIGINAL bean, not the proxy. AOP advice never fires." 3. "Solutions: (1) inject `self` field via @Autowired — `self.methodB()` goes through the proxy. (2) Extract B to a separate bean. (3) Use AspectJ weaving (compile-time) instead of proxy-

based AOP." 4. "Best practice: avoid self-invocation patterns; extract distinct concerns to separate beans. Cleaner design AND avoids the trap." 5. "Same trap applies to @Async, @Cacheable, @Retryable — any proxy-based annotation."

Edge cases to mention: - For circular bean dependencies, injecting self with @Lazy works - AspectJ load-time or compile-time weaving avoids this entirely but adds build complexity - For private @Transactional methods: also doesn't work — proxies can only intercept public methods

What NOT to say: - "Always works regardless of caller" — false; self-invocation is the classic trap - "Use ReflectionUtils to invoke through proxy" — hack; refactor instead

Common follow-up: "Why doesn't @Transactional work on private methods?"

Q123 · Spring Data audit + Envers (full history)

Asked at: Senior · **Difficulty:** Hard · **Topic:** Audit history

The question: "How would you keep a full revision history of every entity change?"

The 30-second answer: Hibernate Envers. Add `@Audited` to the entity; Envers creates parallel `_AUD` tables tracking every revision. Query history via `AuditReader`: `reader.find(User.class, userId, revision)`. Trades schema overhead and write cost for full audit trail.

Step-by-step thought process: 1. "Add dependency: `hibernate-envers`." 2. "`@Audited` on entity classes. Envers auto-creates `_AUD` tables: `USERS` → `USERS_AUD` with `REV` column, `REVTYPE` (`INSERT/UPDATE/DELETE`), original columns." 3. "Configure revision number generator: shared sequence — each transaction gets a revision. Optional `RevisionEntity` carries username, timestamp, IP, etc." 4. "Query: `AuditReader r = AuditReaderFactory.get(em); User userAtRev100 = r.find(User.class, id, 100);` — entity state at that revision." 5. "Trade-offs: every change writes to TWO tables (main + AUD). Storage grows linearly with change rate. Periodic archival or partitioning of AUD tables for hot entities."

Edge cases to mention: - For 'who-did-what': implement a custom `RevisionEntity` with `userId` field; populate via `RevisionListener` - For querying-by-date: `r.getRevisions(User.class, id)` returns list of revisions; `r.getRevisionDate(revNumber)` for timestamp - For very-high-throughput tables: Envers can be a bottleneck — consider CDC (Debezium) + Kafka pattern instead

What NOT to say: - "Just add `createdAt/updatedAt`" — only tracks last change, not full history - "Use DB triggers" — works but moves audit logic out of code

Common follow-up: "How would you record WHO made each change in Envers?"

Q124 · @ConditionalOnMissingBean for repository defaults**Asked at:** Senior · **Difficulty:** Medium · **Topic:** Autoconfig**The question:** "How would a library expose a default DataSource that users can override?"**The 30-second answer:** Library defines a `@Bean public DataSource defaultDataSource()` wrapped in `@ConditionalOnMissingBean(DataSource.class)`. If the application defines its own DataSource bean, the autoconfig backs off. Standard Spring Boot autoconfig pattern.**Step-by-step thought process:** 1. "Library's autoconfig class: `@Configuration @ConditionalOnClass(DataSource.class) class MyLibraryAutoConfig { @Bean @ConditionalOnMissingBean public DataSource libDataSource() { ... } }`." 2. "Listed in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`." 3. "At startup, Spring Boot evaluates: classpath has DataSource? Yes. User defined a DataSource? No → register lib's default. Yes → skip, user's wins." 4. "Spring Boot's own DataSourceAutoConfiguration uses this pattern — provides defaults for H2/HSQL/Derby if user hasn't configured one." 5. "For service-layer beans: same pattern. Library provides default impl, user can swap with their own."**Edge cases to mention:** - `@ConditionalOnMissingBean` evaluates at config-time; if user's bean is declared in a config that's loaded LATER, lib's default still loaded (ordering matters) - Use `@AutoConfigureBefore` / `@AutoConfigureAfter` for explicit ordering - For multiple candidates of same type: `@ConditionalOnSingleCandidate` only registers if exactly one bean exists**What NOT to say:** - "User overrides via XML" — works but unidiomatic for Spring Boot - "Throw if user provides their own" — defeats the override pattern**Common follow-up:** "How would you write a Spring Boot starter for an internal library?"**Q125 · LazyInitializationException****Asked at:** Mid-senior · **Difficulty:** Medium · **Topic:** JPA gotchas**The question:** "What's a LazyInitializationException, and how do you fix it?"**The 30-second answer:** Thrown when you access a lazy-loaded association OUTSIDE an active EntityManager / Hibernate session. Fix by ensuring the association is loaded within the transaction (JOIN FETCH, @EntityGraph) OR keeping the session open (open-session-in-view, anti-pattern). Better: fetch what you need within the service layer.**Step-by-step thought process:** 1. "Scenario: Service loads User (transaction commits, session closes). Controller serializes user → Jackson accesses lazy `user.getOrders()` → no active session → LazyInitializationException." 2. "Fix #1: in the service, eagerly fetch what's needed: `@EntityGraph(attributePaths = {"orders"}) findById(id)`. Tx returns fully-loaded entity." 3. "Fix #2: return a DTO from the service, not the entity. Map fields you need; entity association no

longer accessed downstream." 4. "Fix #3 (anti-pattern): Open Session in View — Spring keeps the session open through the controller layer. Hides the problem; causes N+1 in production silently. AVOID." 5. "Best practice: services return DTOs explicitly. Entity boundaries stay clean."

Edge cases to mention: - `spring.jpa.open-in-view=false` (recommended for new apps) — surfaces these errors in dev instead of silently working in OSIV mode - For tests: `@Transactional` keeps the test method's tx open across loads; the bug only manifests in real prod flow - `Hibernate.initialize(user.getOrders())` inside the tx forces eager load

What NOT to say: - "Mark association EAGER" — same as before; ruins other queries - "Set OSIV true to hide the error" — production performance disaster

Common follow-up: "What's open-session-in-view and why is it discouraged?"

Section 6 — Hibernate (20 Qs)

The ORM running under most JPA. Topics where it bites: entity lifecycle, lazy/eager fetch, dirty checking, N+1, caching layers, locking.

Q126 · Entity lifecycle states

Asked at: Common · **Difficulty:** Medium · **Topic:** Lifecycle

The question: "What are the JPA / Hibernate entity lifecycle states?"

The 30-second answer: Four states. **Transient** (new, no DB row, not in session). **Managed/Persistent** (in session, changes tracked, DB-synced on commit). **Detached** (was managed, but session closed; changes not tracked). **Removed** (marked for deletion; gone after commit).

Step-by-step thought process: 1. "Transient: `new User(...)` . Plain Java object, no JPA awareness. Garbage collected if not persisted." 2. "Managed/Persistent: after `em.persist(user)` or `em.find(...)` . JPA tracks changes. Setters trigger dirty checking at flush time." 3. "Detached: session closed (transaction commit, `em.close`, `em.detach`, `em.clear`). Object still in memory but disconnected from JPA. Changes NOT tracked. Re-attach with `em.merge()` if needed." 4. "Removed: `em.remove(user)` . Scheduled for DELETE at flush. Until flush, still in session." 5. "Transitions: transient → persistent (`persist`), persistent → detached (`commit/close`), detached → persistent (`merge`), persistent → removed (`remove`)."

Edge cases to mention: - `merge()` doesn't update the detached object; returns a NEW managed copy. Variable reassignment needed: `user = em.merge(user);` - Cascade types: PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL — controls if state transitions propagate to associations - For batch updates: detach periodically to avoid OOM (`em.clear()`)

What NOT to say: - "Managed = saved" — managed entities are tracked but not necessarily INSERTed yet (until flush) - "Detached means deleted" — detached just means not session-attached

Common follow-up: "What does `merge()` actually do?"

Q127 · Lazy vs eager fetching

Asked at: Common · **Difficulty:** Medium · **Topic:** Fetch strategy

The question: "What's the difference between LAZY and EAGER fetching in JPA?"

The 30-second answer: LAZY = association loaded on first access (proxy returned initially). EAGER = loaded immediately with the parent. JPA defaults: @ManyToOne/@OneToOne EAGER, @OneToMany/@ManyToMany LAZY. Override defaults; best practice is LAZY everywhere and explicit fetch (JOIN FETCH / @EntityGraph) per query.

Step-by-step thought process: 1. " `@OneToMany(fetch = FetchType.LAZY) List<Order> orders;` — orders loaded only when accessed." 2. " `@ManyToOne(fetch = FetchType.LAZY) Customer customer;` — customer loaded on first reference." 3. "Hibernate uses bytecode-instrumented proxies for lazy loading. `user.getOrders()` triggers SELECT for orders if not yet loaded." 4. "Default JPA: ManyToOne and OneToOne are EAGER (loaded with parent). OneToMany and ManyToMany are LAZY." 5. "Best practice: override defaults to LAZY everywhere. For each query, explicitly fetch what's needed with JOIN FETCH or @EntityGraph. Avoids surprise N+1 + over-fetching."

Edge cases to mention: - Lazy + outside-of-session = LazyInitializationException (Q125) - For @Embedded objects: always eager (embedded is part of the entity, not a separate row) - For collections of basic types (@ElementCollection): lazy by default but no proxy mechanism — use JOIN FETCH

What NOT to say: - "EAGER is safer" — causes massive over-fetch; harder to optimize - "LAZY is always faster" — depends on access pattern

Common follow-up: "What's a JPA proxy and how does Hibernate implement it?"

Q128 · Dirty checking — how does Hibernate know to UPDATE?

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Change detection

The question: "How does Hibernate know to issue an UPDATE for a modified entity without you calling `save()`?"

The 30-second answer: At load, Hibernate snapshots the entity's field values. At flush/commit, it compares current values to the snapshot. Differences → UPDATE issued for those columns. No `save()` call needed — dirty checking is automatic for managed entities.

Step-by-step thought process: 1. " `User user = em.find(User.class, 1);` — Hibernate loads, stores a snapshot of field values internally." 2. " `user.setEmail(\"new@x\");` — your setter mutates the entity. No DB call yet." 3. "At flush time (manual, query trigger, or transaction commit),

Hibernate iterates managed entities. For each, compares current values vs snapshot." 4. "Diff found → UPDATE generated for that entity (or just the changed columns with `@DynamicUpdate`)." 5. "To skip dirty check: use `@Transactional(readOnly = true)` — Hibernate doesn't snapshot, skips the comparison."

Edge cases to mention: - Auto-dirty-check is O(N) over managed entities — `clear()` periodically in batch - `@DynamicUpdate`: only generates UPDATE for changed columns (default updates all columns). Small win on wide tables - For deferred change-tracking: use Hibernate's bytecode enhancement to detect changes at set-time, skipping the snapshot

What NOT to say: - "Hibernate uses `==` on the entity" — wrong; field-level snapshot comparison - "save() is required" — wrong for managed entities

Common follow-up: "What's the difference between `save()` and `persist()`?"

Q129 · save vs persist vs merge

Asked at: Common · **Difficulty:** Medium · **Topic:** Entity persistence

The question: "What's the difference between `save()`, `persist()`, and `merge()`?"

The 30-second answer: `persist()` — JPA standard; makes a transient entity persistent. Inserts. No return value. Throws if already managed. `merge()` — copies a detached entity's state into a managed instance. Returns the managed copy. `save()` — Spring Data / older Hibernate alias; calls `persist` for transient, `merge` for detached.

Step-by-step thought process: 1. "`persist(entity)` : JPA standard. Adds transient entity to the session as managed. INSERT at flush. Returns void." 2. "`merge(entity)` : takes a detached entity. Creates a NEW managed copy by copying fields. Returns the managed copy — original reference is still detached. UPDATE at flush." 3. "`save(entity)` in Spring Data: smart wrapper. If transient (no ID), calls `persist`. If detached (has ID), calls `merge`. Returns the managed entity." 4.

"`saveAndFlush(entity)` in `JpaRepository`: save + immediate flush. Useful when you need the auto-generated ID right after save (e.g. for FK in another insert)." 5. "Pitfall: `user = repo.save(user);` — assign return value, because `merge` returns a different reference."

Edge cases to mention: - For batch inserts: prefer `persist` (avoids merge overhead). Or use `saveAll()` for grouped operations - For "upsert" semantics (insert or update by ID): `save()` handles both cases - For very large batches: `entityManager.persist()` + periodic flush+clear is more memory-efficient than `save()`

What NOT to say: - "save() is just persist" — no, it's smarter - "merge() updates in place" — wrong; returns a NEW managed copy

Common follow-up: "Why does `merge` return a different reference than the input?"

Q130 · @OneToMany cascade types

Asked at: Common · **Difficulty:** Medium · **Topic:** Cascading

The question: "What does CascadeType.ALL do and when should you avoid it?"

The 30-second answer: `CascadeType.ALL` = PERSIST + MERGE + REMOVE + REFRESH + DETACH all cascade to associated entities. Useful for parent-child where child can't exist without parent (Order → OrderItem). Avoid when entities can be independently managed — accidentally deleting parent would orphan-delete unrelated children.

Step-by-step thought process: 1. "Cascade controls whether operations on the parent propagate to child entities. Types: PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL." 2.

"`@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true) List<OrderItem> items;` — save Order auto-saves items. Delete Order auto-deletes items. Remove from list → orphan removed." 3. "`orphanRemoval = true` is separate from cascade — triggers DELETE when child is removed from the parent's collection." 4. "Use when: composition (child can't exist without parent). Order → OrderItem; Invoice → InvoiceLine." 5. "Avoid when: association (child has independent lifecycle). User → Posts: deleting a User shouldn't delete all their Posts; the Posts may be orphans you want to keep for audit."

Edge cases to mention: - CascadeType.REMOVE without orphanRemoval: removing from collection doesn't delete; only deleting parent does - For very large child collections: cascade DELETE issues N+1 individual DELETES. Consider native bulk DELETE WHERE parent_id = ? for performance - @ManyToMany cascade: avoid REMOVE — usually you don't want to cascade-delete from a many-to-many

What NOT to say: - "CascadeType.ALL is the default" — no default; you must specify - "Use ALL everywhere" — dangerous for non-composition relationships

Common follow-up: "What's the difference between CascadeType.REMOVE and orphanRemoval = true?"

Q131 · LOB types (BLOB/CLOB)

Asked at: Mid-level · **Difficulty:** Medium · **Topic:** LOB handling

The question: "How would you store a 50 MB file in a DB column via JPA?"

The 30-second answer: Use `@Lob` annotation. For binary: `@Lob @Column private byte[] data;`. For text: `@Lob @Column private String content;`. Maps to BLOB/CLOB DB types. For large files, prefer streaming with `Blob` interface or storing the file outside the DB (S3) and keeping just a URL.

Step-by-step thought process: 1. "`@Lob` tells JPA to use LOB types. `byte[]` → BLOB, `String` → CLOB / TEXT depending on DB." 2. "Loading: entire LOB is loaded into memory by default. For a 50

MB file: 50 MB heap per entity load. Not great." 3. "Streaming: `@Lob private Blob data;` — `Blob.getBinaryStream()` reads chunks. Avoids full memory load." 4. "For large files: don't store in DB. Upload to S3 / GCS; store the URL or key in DB. Cheaper, faster, scales better." 5. "For text content: same principle. Long text in CLOB is fine if it's content tied to the row; for documents, store externally."

Edge cases to mention: - Postgres has native BYTEA + TEXT — works fine for moderate sizes (< 10 MB) - For binary streaming: configure `hibernate.jdbc.use_streams_for_binary=true` (default true in modern Hibernate) - For object storage: pre-signed S3 URLs for direct browser upload — bypasses your server entirely

What NOT to say: - "Store 1 GB files in DB" — operational nightmare; backup time, replication lag, etc. - "DBs aren't designed for blobs" — exaggeration; small blobs are fine

Common follow-up: "Why would you store a file URL in DB instead of the file itself?"

Q132 · @Embedded and @Embeddable

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** Value objects

The question: "How would you model an Address as a value object in JPA?"

The 30-second answer: Define an `@Embeddable` class with the fields. Use `@Embedded` in the entity to include it. JPA flattens the columns into the entity's table — no separate Address table.

Step-by-step thought process: 1. "`@Embeddable class Address { String street; String city; String pincode; }`" 2. "In entity: `@Embedded private Address address;`. JPA adds street + city + pincode columns to the entity's table." 3. "For multiple addresses in one entity (e.g. billing + shipping): `@AttributeOverrides` to disambiguate column names — `@AttributeOverrides({@AttributeOverride(name=\"street\", column=@Column(name=\"billing_street\")), ...})`" 4. "For collections of embeddables: `@ElementCollection List<Address> addresses;` — Hibernate creates a separate table linked by parent ID." 5. "Use when: small value objects without independent identity. Coordinate (lat/long), Money (amount/currency), Address."

Edge cases to mention: - `@Embeddable` can have `@Embedded` inside it — nested value objects - equals/hashCode on Embeddable should be based on all fields — Hibernate uses these for ElementCollection comparison - For very-rich domain models: consider DDD value objects + JPA converters for conversion

What NOT to say: - "Make Address its own entity" — overkill for non-shared value objects - "Embeddable can't have methods" — wrong; it's a normal Java class with `@Embeddable`

Common follow-up: "How would you have both a billing and shipping Address on one entity?"

Q133 · @Inheritance strategies

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Inheritance mapping

The question: "What inheritance strategies does JPA support, and which would you pick?"

The 30-second answer: Three strategies. **SINGLE_TABLE**: one table for the whole hierarchy + discriminator column. Fast queries, lots of NULL columns. **TABLE_PER_CLASS**: one table per concrete class. No JOINS but no shared base. **JOINED**: one table per class, JOINed for queries. Most normalized. Default SINGLE_TABLE; pick TABLE_PER_CLASS for fully-independent variants, JOINED for normalized inheritance.

Step-by-step thought process: 1. " `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)` on base entity. All subclasses → one table with `@DiscriminatorColumn` distinguishing rows." 2. "Pros: no JOINS, fastest queries. Cons: many nullable columns (each subclass's unique fields are NULL for others)." 3. " `TABLE_PER_CLASS` : one table per concrete class. No JOINS but polymorphic queries do UNION ALL across tables. Slow polymorphic queries." 4. " `JOINED` : each class has its own table. Subclass tables JOIN to parent. Polymorphic queries use JOINS. Most normalized." 5. "Default: SINGLE_TABLE. Modern preference for most apps: avoid inheritance entirely in JPA — use composition + separate entities. Inheritance in ORMs gets messy fast."

Edge cases to mention: - SINGLE_TABLE + discriminator: `@DiscriminatorValue("\\"PREMIUM\\"")` per subclass - Foreign keys to subclass types: works fine if FK references the discriminator-typed row - For pure data classes: prefer composition + separate entities to inheritance

What NOT to say: - "Inheritance is the cleanest design" — for data, composition usually wins - "JOINED is always more normalized" — true but adds query cost

Common follow-up: "When would you avoid JPA inheritance entirely?"

Q134 · Hibernate SQL logging

Asked at: Common in debug rounds · **Difficulty:** Easy-Medium · **Topic:** Debugging

The question: "How do you see the SQL Hibernate is generating?"

The 30-second answer: `spring.jpa.show-sql=true` for basic output. For prod-quality logging: `spring.jpa.properties.hibernate.show_sql=false` + `logging.level.org.hibernate.SQL=DEBUG` + `logging.level.org.hibernate.orm.jdbc.bind=TRACE` (Hibernate 6+) for SQL + bind parameters. Better: use p6spy or datasource-proxy for clean logs with timings.

Step-by-step thought process: 1. "Quick toggle: `spring.jpa.show-sql=true` — prints SQL to stdout. Ugly, unstructured." 2. "For structured logs: `spring.jpa.properties.hibernate.show_sql=false` (turn off built-in) +

`logging.level.org.hibernate.SQL=DEBUG` . SQL goes through your logger (JSON, ELK-friendly)." 3. "For bind parameters: `logging.level.org.hibernate.orm.jdbc.bind=TRACE` (Hibernate 6+) or `logging.level.org.hibernate.type.descriptor.sql=TRACE` (older)." 4. "For pretty SQL: `spring.jpa.properties.hibernate.format_sql=true` — newlines, indented." 5. "For dev productivity: `p6spy` or `datasource-proxy` intercepts JDBC; logs SQL + params + execution time clean and consistent."

Edge cases to mention: - For prod: never log SQL at INFO; performance + log volume hit. Maybe at WARN for slow queries via slow-query logger - For Spring Boot 3 + native: Hibernate logging is reachable from native image; verify config translates - For tests: `@AutoConfigureTestDatabase` + verbose SQL logs are useful

What NOT to say: - "Use a SQL profiler externally" — works but slow feedback loop - "Hibernate is opaque, can't debug" — fully debuggable with right config

Common follow-up: "How would you log the SLOW queries only?"

Q135 · @NamedQuery vs @Query vs Criteria API

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Query options

The question: "When would you use @NamedQuery vs @Query vs Criteria API?"

The 30-second answer: `@NamedQuery` — JPQL defined as annotation on the entity; pre-validated at startup. `@Query` — Spring Data annotation on the repository method; flexible, common choice. Criteria API — programmatic, dynamic queries; verbose, used for dynamic filter queries. Most apps use @Query for static, Specifications for dynamic.

Step-by-step thought process: 1. " `@NamedQuery(name = \"User.findActive\", query = \"SELECT u FROM User u WHERE u.active = true\")` on entity. Reference by name. Validated at startup (catches typos early). Less common in Spring apps." 2. " `@Query(\"SELECT u FROM User u WHERE u.active = true\") List<User> findActive();` on repository method. The Spring Data standard." 3. "Criteria API: `CriteriaBuilder cb = em.getCriteriaBuilder(); CriteriaQuery<User> cq = cb.createQuery(User.class); ...` — verbose but type-safe and dynamic." 4. "For Spring apps: use @Query for static queries (90% of cases). Use Spring Data Specifications (built on Criteria) for dynamic queries (search endpoints with multiple optional filters)." 5. "For pure JPA (no Spring): @NamedQuery + Criteria API."

Edge cases to mention: - @Query validates at startup if `validation-mode=auto` — same benefit as @NamedQuery for free - For very dynamic queries: jOOQ or QueryDSL > Criteria API in ergonomics - Native queries: `@Query(nativeQuery = true)` for vendor SQL features

What NOT to say: - "Always use Criteria for safety" — too verbose for static queries - "@NamedQuery is deprecated" — still supported

Common follow-up: "Show me the equivalent query in @Query, NamedQuery, and Criteria API."

Q136 · UUID as primary key

Asked at: Senior · **Difficulty:** Medium · **Topic:** ID strategies

The question: "Pros and cons of using UUID as a primary key vs auto-increment Long?"

The 30-second answer: UUID pros: distributed-generation friendly (no DB round-trip for ID), no collision across systems, no enumeration attacks (can't guess IDs). Cons: 16 bytes vs 8 bytes per row, random UUIDs cause B-tree fragmentation (worse insert + index size). Use UUIDv7 (time-ordered) or sequential UUID variants for better B-tree behavior.

Step-by-step thought process: 1. "Long auto-increment: 8 bytes per row, sequential inserts → sequential B-tree leaves → minimal page splits, compact index." 2. "UUID v4 (random): 16 bytes, totally random → index inserts hit random leaves → page splits, fragmentation, larger index." 3. "UUID pros: generate ANYWHERE (client, service, sharded systems) — no central sequence. Multi-region apps benefit. No ID enumeration leaking (can't guess 'next' ID)." 4. "For Postgres: store as `uuid` type (16 bytes). For MySQL: `CHAR(36)` (36 bytes — wasteful) or `BINARY(16)` (efficient)." 5. "Modern compromise: UUID v7 (time-ordered) — sortable, decent B-tree behavior, distributed generation. Best of both worlds."

Edge cases to mention: - Snowflake IDs (Twitter pattern): time-ordered + machine ID + sequence. Compact (8 bytes) but require coordination - ULID: similar to UUIDv7, slightly different encoding - For high-write tables: avoid random UUIDs; pick a time-ordered variant

What NOT to say: - "UUIDs are always better" — perf trade-offs matter - "Long auto-increment is legacy" — still optimal for many cases

Common follow-up: "What's UUIDv7 and why is it better than UUIDv4 as a PK?"

Q137 · @Transactional propagation NESTED

Asked at: Senior · **Difficulty:** Hard · **Topic:** Savepoints

The question: "What's Propagation.NESTED?"

The 30-second answer: Creates a SAVEPOINT within the outer transaction. If the inner method fails, it rolls back to the savepoint (not the whole tx). Requires JDBC savepoint support (most DBs OK; some drivers limited). Less common than REQUIRES_NEW; useful for "try this, but don't break the outer" patterns.

Step-by-step thought process: 1. "REQUIRED: join existing transaction. REQUIRES_NEW: suspend existing, start new (separate connection). NESTED: SAVEPOINT in the existing tx." 2. "Inner method with

NESTED: Hibernate creates a SAVEPOINT before invoking. Inner method runs." 3. *"Inner failure: ROLLBACK TO SAVEPOINT — only inner work is undone. Outer transaction continues."* 4. *"Inner success: SAVEPOINT released. Inner work persists if outer commits."* 5. *"NOT all DB drivers support nested savepoints well — test before relying on it. For most cases, REQUIRES_NEW is cleaner."*

Edge cases to mention: - Postgres + JDBC: full savepoint support - MySQL: supports SAVEPOINT but with quirks around DDL - For deeply nested savepoints: performance degrades; keep flat

What NOT to say: - "NESTED is the same as REQUIRES_NEW" — different mechanisms (savepoint vs new tx) - "Always use NESTED for safety" — adds complexity; REQUIRES_NEW often clearer

Common follow-up: *"What's the practical difference between NESTED and REQUIRES_NEW?"*

Q138 · @Formula for derived columns

Asked at: Senior · **Difficulty:** Medium · **Topic:** Computed fields

The question: *"How would you map a calculated field (like 'fullName' or 'totalAmount') in Hibernate?"*

The 30-second answer: `@Formula("\first_name || ' ' || last_name")` — Hibernate-specific annotation. The expression is computed in SQL on every SELECT; treated as a read-only field. Useful for derived values without writing the result back to DB.

Step-by-step thought process: 1. `@Formula("\first_name || ' ' || last_name")`
`private String fullName;` — SQL expression evaluated on SELECT." 2. "Hibernate inlines this expression in the generated SQL: `SELECT first_name, last_name, (first_name || ' ' || last_name) AS fullName FROM users` ." 3. "Field is READ-ONLY — Hibernate ignores writes to it (no UPDATE SET fullName)." 4. "Use cases: computed counts (`@Formula("\(SELECT COUNT(*) FROM orders WHERE user_id = id)\") int orderCount;`), denormalized derived fields, computed flags." 5. "Alternative: DB views — define a view that includes the computation; map the entity to the view. Cleaner for complex computations."

Edge cases to mention: - @Formula is Hibernate-only (not JPA standard) — locks you into Hibernate - For performance: complex Formulas with subqueries can be expensive at scale; consider denormalization - Don't filter or sort on @Formula — index isn't applied to inline expressions

What NOT to say: - "Compute in Java" — wastes a DB round trip - "Use DB triggers to maintain the column" — works but adds operational complexity

Common follow-up: *"When would you use a DB view over a @Formula?"*

Q139 · Hibernate batch insert / update

Asked at: Mid-senior · **Difficulty:** Medium-Hard · **Topic:** Performance

The question: "How would you insert 10,000 entities efficiently with Hibernate?"

The 30-second answer: Set `spring.jpa.properties.hibernate.jdbc.batch_size=50` (or similar). Hibernate groups inserts into JDBC batches. Loop: persist 50, flush, clear. Avoid auto-increment IDs (breaks batching for INSERT); use sequence allocation or UUIDs.

Step-by-step thought process: 1. "Set `hibernate.jdbc.batch_size=50` — Hibernate batches that many SQL operations into one JDBC batch." 2. "Insert loop: `for (i=0; i<10000; i++) { em.persist(entity); if (i % 50 == 0) { em.flush(); em.clear(); } }`. Flush sends batch; clear releases entity references (memory)." 3. " `hibernate.order_inserts=true` and `hibernate.order_updates=true` — groups same-type operations for efficient batching." 4. "Auto-increment IDs break batching (Hibernate needs to call DB for each ID separately). Use sequence with allocation (`@SequenceGenerator(allocationSize = 50)`) or UUIDs to allow batching." 5. "For bulk operations: native SQL `INSERT INTO ... SELECT` or `COPY` (Postgres) — far faster than ORM for pure bulk loads."

Edge cases to mention: - For very large bulks (millions): bypass JPA. Use JdbcTemplate batchUpdate or native copy commands - Disable second-level cache and queries cache during batch to avoid eviction overhead - For unidirectional @OneToMany: batching may not work as expected; prefer @ManyToOne owning side

What NOT to say: - "Just save() in a loop" — fires 10,000 separate INSERTs - "Hibernate is slow for bulk" — it's slow without batching config; properly configured it's fine for tens of thousands

Common follow-up: "Why does auto-increment ID break batching?"

Q140 · @Where and @SQLDelete (soft delete)

Asked at: Senior · **Difficulty:** Medium · **Topic:** Soft deletes

The question: "How would you implement soft delete in Hibernate so queries automatically exclude deleted rows?"

The 30-second answer: `@SQLDelete(sql = "UPDATE users SET deleted = true WHERE id = ?")` overrides DELETE to UPDATE. `@SQLRestriction("deleted = false")` (Hibernate 6.3+; legacy `@Where` is deprecated) filters all queries to exclude soft-deleted rows. Implicit and global at the entity level.

Step-by-step thought process: 1. "Add `boolean deleted` column. `@SQLDelete(sql = "UPDATE users SET deleted = true WHERE id = ?")` — Hibernate uses this instead of DELETE." 2. " `@SQLRestriction("deleted = false")` on the entity class — appended to every SELECT for this entity. (Pre-Hibernate 6.3 this was `@Where(clause = ...)` — now deprecated.)" 3. " `repo.delete(user)` now soft-deletes. `repo.findAll()` skips soft-deleted rows." 4. "Trade-off: queries CAN'T see soft-deleted rows easily — to audit deleted: define a separate repo method using

@Query without the restriction. Or use native query." 5. "For audit + restore: keep soft-deleted rows in same table. For privacy compliance: actual DELETE after grace period."

Edge cases to mention: - @SQLRestriction affects EVERYTHING — even relationships. Soft-deleted child still loaded via parent traversal? Depends on cascade - Modern alternative: Hibernate Envers + actual DELETE — full history without soft-delete tax - For Spring Boot 3 + Hibernate 6.3+: prefer @SQLRestriction over the legacy @Where ; same semantics, supported API

What NOT to say: - "Soft delete is always best" — adds query complexity; consider use case - "Just add a WHERE clause to every repo method" — error-prone; @Where centralizes

Common follow-up: "How would you handle 'show deleted users' for an admin endpoint?"

Q141 · NaturalId

Asked at: Senior · **Difficulty:** Medium · **Topic:** Hibernate-specific

The question: "What's a NaturalId in Hibernate?"

The 30-second answer: A business-meaningful unique identifier (e.g. email for User, ISBN for Book) separate from the synthetic primary key. @NaturalId marks the field; Hibernate provides fast Session.byNaturalId(User.class).using("email", "x@y").load() with L2 cache support.

Step-by-step thought process: 1. "PK is usually synthetic (Long ID). NaturalId is the business-meaningful unique field (email, SSN, ISBN)." 2. " @NaturalId @Column(unique = true) String email; on entity." 3. "Lookup: User user = session.byNaturalId(User.class).using("email", email).load(); . Hibernate-specific API; uses dedicated L2 cache region." 4. "Mutability: by default @NaturalId is immutable. Override with @NaturalId(mutable = true) if it can change." 5. "For Spring Data: derive method names work (findByEmail) but lose the L2 cache benefit. For hot lookup paths, use NaturalId loader directly."

Edge cases to mention: - Composite NaturalId: multiple fields annotated together - Mutable NaturalId triggers cache invalidation on change - For very read-heavy lookups by natural ID: significant win over standard findByEmail

What NOT to say: - "Use email as PK directly" — works but ties PK to changeable business value - "NaturalId is the same as @Column(unique=true)" — adds cache + Hibernate API support

Common follow-up: "When would you choose NaturalId over a standard repository method?"

Q142 · Hibernate hbm vs annotations

Asked at: Senior · **Difficulty:** Easy-Medium · **Topic:** Mapping styles

The question: "Hibernate XML hbm.xml vs JPA annotations — which would you use in 2026?"

The 30-second answer: Annotations. XML hbm files are pre-JPA Hibernate-specific. Modern apps use JPA annotations (@Entity, @Column, etc.) — same metadata, IDE support, refactor-safe, type-safe. XML is legacy.

Step-by-step thought process: 1. "Pre-Hibernate-3 (early 2000s): mapping defined in XML hbm files. Separate from Java code." 2. "Hibernate 3+ adopted JPA annotations: @Entity, @Table, @Column, @OneToMany. Same metadata, on the Java class." 3. "Annotations: refactor-safe (rename column via IDE), single-file (metadata + code together), type-checked at compile time." 4. "XML hbm: only useful for legacy apps maintaining backward compat. For new code: no reason to choose XML." 5. "Mix possible (annotations + XML override) — useful when annotating someone else's class isn't possible."

Edge cases to mention: - For metadata-driven dynamic apps: programmatic configuration via `MetadataSources` and `MetadataBuilder` API - ORM.xml (JPA's XML format) is an option for overriding annotations without touching the source - For modern Spring Boot: pure annotation; never write hbm/orm.xml unless legacy

What NOT to say: - "Use XML for cleaner separation" — annotations are cleaner in practice - "XML hbm is deprecated" — supported but not preferred

Common follow-up: "Why are annotations preferred over XML in modern frameworks?"

Q143 · Hibernate's flush modes

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Flushing

The question: "What are Hibernate's flush modes?"

The 30-second answer: AUTO (default — flush before queries that might be affected, and at commit), COMMIT (flush only at commit), ALWAYS (flush before every query), MANUAL (only when you explicitly call `em.flush()`). AUTO is usually right; MANUAL useful for batch processing where you control flush points.

Step-by-step thought process: 1. "FlushMode controls WHEN Hibernate sends pending changes to the DB." 2. "AUTO (default): flush before queries that might see uncommitted changes (Hibernate makes a best guess). Always at commit." 3. "COMMIT: only flush at commit. Queries see stale data until commit." 4. "ALWAYS: flush before every query (paranoid mode; slow)." 5. "MANUAL: never auto-flush; you call `em.flush()` explicitly. Useful for batch jobs where you want control." 6. "Set via `em.setFlushMode(FlushModeType.COMMIT)` or `@Transactional` and configure session."

Edge cases to mention: - For long-running transactions with many reads: COMMIT can speed things up (fewer flushes) but risks read-your-own-writes inconsistency - For batch jobs: MANUAL + explicit flush+clear gives memory + perf control - For Spring Data: defaults to AUTO; rarely need to change

What NOT to say: - "Always use ALWAYS for safety" — kills performance - "FlushMode doesn't matter" — affects query consistency

Common follow-up: "What's the difference between AUTO and ALWAYS?"

Q144 · @Convert for type conversion

Asked at: Mid-level · **Difficulty:** Easy-Medium · **Topic:** Type conversion

The question: "How would you store a Java enum or LocalDate in a custom DB format?"

The 30-second answer: `@Convert(converter = MyConverter.class)` on the field, with a `MyConverter implements AttributeConverter<Java, DB>` class. Java type ↔ DB type conversion is handled at flush/load. Useful for enums as strings, Money as cents, custom date formats.

Step-by-step thought process: 1. "Define converter: `@Converter class MoneyConverter implements AttributeConverter<Money, Long> { public Long convertToDatabaseColumn(Money m) { return m.cents(); } public Money convertToEntityAttribute(Long cents) { return Money.fromCents(cents); } }`" 2. "Apply to field: `@Convert(converter = MoneyConverter.class) Money price;`" 3. "Or auto-apply: `@Converter(autoApply = true)` — used for ALL fields of that type." 4. "For enums: by default JPA stores as ORDINAL (integer position) — fragile. Use `@Enumerated(EnumType.STRING)` to store as name, more robust." 5. "For LocalDate/Instant: JPA 2.2+ supports natively. Older versions need @Convert."

Edge cases to mention: - @Convert doesn't apply to IDs or @Version fields (JPA restriction) - For JSON columns (Postgres jsonb): write converter Object<->String - Auto-apply can surprise — be explicit for clarity

What NOT to say: - "Store JSON as String, parse in service" — works but loses typing - "Use @Lob for any custom format" — wrong tool

Common follow-up: "How would you store an enum as a string vs integer?"

Q145 · Hibernate stats and slow query detection

Asked at: Senior · **Difficulty:** Medium · **Topic:** Production debugging

The question: "How would you find slow Hibernate queries in production?"

The 30-second answer: Enable Hibernate statistics:

`spring.jpa.properties.hibernate.generate_statistics=true` + `hibernate.session.events.log.LOG_QUERIES_SLOWER_THAN_MS=100` (or the Hibernate 6+ alias `hibernate.log_slow_query=100`). Logs queries slower than threshold. Combine with Micrometer for dashboards. For deep analysis: use JFR (Java Flight Recorder) with the JPA event provider, or APM tools like Datadog / NewRelic.

Step-by-step thought process: 1. "`hibernate.generate_statistics=true`" — Hibernate maintains query counts, cache hit rates, fetch counts per query. Expose via Spring's `org.springframework.boot.actuate.metrics.hibernate.HibernateMetrics` (auto-configured). 2. "`hibernate.session.events.log.LOG_QUERIES_SLOWER_THAN_MS=100`" — logs queries taking > 100ms. Includes SQL + params + duration." 3. "Surface to Micrometer / Prometheus: `hibernate_query_cache_hits`, `hibernate_query_cache_misses`, `hibernate_transactions_total`, etc." 4. "For deep dive: JFR with the `hibernate-jfr` module — captures queries + cache events + transactions at near-zero overhead." 5. "APM (Datadog APM, NewRelic): auto-instruments JPA — query plans, slow query reports, anomaly detection."

Edge cases to mention: - Statistics generation has small overhead; usually fine for prod but profile if needed - For `pg_stat_statements` (Postgres-side): correlates DB-observed slow queries with Hibernate's view - For dev: use Hibernate's built-in query plan cache stats (`hibernate.query.plan_cache_max_size`) to spot plan-cache thrashing on dynamic queries

What NOT to say: - "Just add timing to every method" — gets you per-method timing, not per-query - "Hibernate doesn't expose internals" — `generate_statistics` + Micrometer gives you everything

Common follow-up: "How would you set up alerts on slow Hibernate queries in production?"

Section 7 — REST Design (15 Qs)

The principles that separate REST APIs you regret in 6 months from ones that age well. Versioning, pagination, idempotency, error responses, OpenAPI.

Q146 · RESTful principles in practice

Asked at: Architect · **Difficulty:** Medium · **Topic:** REST fundamentals

The question: "What makes an API actually RESTful?"

The 30-second answer: Resources identified by URLs (`/orders/123`), standard HTTP verbs (GET/POST/PUT/DELETE/PATCH), stateless requests (no server-session dependency), uniform interface (same patterns across resources), JSON (or other) representation. HATEOAS is part of the original definition but rarely used in practice. "REST" in industry usually means "level 2 of Richardson's maturity model."

Step-by-step thought process: 1. "Resources, not actions: `/orders/123` not `/getOrder?id=123` ." 2. "Verbs map to HTTP: GET = read, POST = create, PUT = replace, PATCH = partial update, DELETE = remove. HEAD and OPTIONS for metadata." 3. "Stateless: every request includes all info needed. No server session storing per-request state." 4. "Uniform interface: pagination via query params consistently (`?page=&size=`), same error format everywhere, same auth pattern (Authorization header)." 5. "JSON is standard for body. URLs are case-sensitive lowercase, kebab-case for multi-word resources (`/order-items`)."

Edge cases to mention: - "REST" in practice $\neq$ strict Roy Fielding REST. Industry "REST" usually means HTTP+JSON+verbs without HATEOAS - For collections + items: `/orders` for the collection, `/orders/{id}` for an item - For nested resources: `/orders/{orderId}/items/{itemId}` — useful when items have no global identity

What NOT to say: - "Use POST for everything" — defeats the verb contract - "REST requires HATEOAS" — purists yes; industry no

Common follow-up: "What's the Richardson maturity model?"

Q147 · API versioning strategies

Asked at: Senior · **Difficulty:** Medium · **Topic:** API evolution

The question: "How would you version a REST API?"

The 30-second answer: Three common strategies. **URL versioning** (`/api/v1/users`) — explicit, easy to route, most popular. **Header versioning** (`Accept: application/vnd.app.v2+json`) — cleaner URLs but harder to debug. **Query param versioning** (`?version=2`) — flexible but cache-unfriendly. URL versioning wins in practice for clarity.

Step-by-step thought process: 1. "URL versioning: `/api/v1/orders` and `/api/v2/orders` as separate endpoints. Explicit in every request. Easy to route (Spring's `@RequestMapping("/api/v1")` at controller class level). Most common." 2. "Header versioning: `Accept: application/vnd.myapp.v2+json` or custom `X-API-Version: 2` header. Cleaner URLs. Harder to test (every curl needs the right header)." 3. "Query param: `?version=2` . Cache-unfriendly (params pollute the cache key). Rarely the right call." 4. "For Spring: separate `@RestController` per version + base path. Or content-negotiation-based via `produces = \"application/vnd.app.v2+json\"` ." 5. "Backward compat: support old versions for a deprecation window (6-12 months). Communicate deprecation via headers (`Deprecation: true; sunset=2027-01-01`)."

Edge cases to mention: - Avoid versioning if you can — additive changes (new optional fields) don't need versions - Major version bump = breaking changes; minor changes shouldn't require it - For GraphQL: schema evolution handles versioning differently (additive only)

What NOT to say: - "Just create a v2 alongside v1 forever" — maintenance debt - "Versions don't matter; clients should adapt" — breaks every integration

Common follow-up: "How would you sunset an old API version without breaking clients?"

Q148 · Pagination patterns

Asked at: Common · **Difficulty:** Medium · **Topic:** Pagination

The question: "OFFSET vs cursor pagination — when each?"

The 30-second answer: OFFSET (`?page=N&size=M`) — easy to implement, supports random page jumps. Slow at high offsets (DB scans + skips). Cursor (`?after=lastId`) — fast at any depth, but no random access (only next/prev). Use OFFSET for shallow paging (< 1000 pages). Cursor for deep paging or infinite scroll.

Step-by-step thought process: 1. "OFFSET: `SELECT * FROM orders ORDER BY id LIMIT 20 OFFSET 10000` . DB scans 10020 rows, returns last 20. At deep offsets (millions of rows), painfully slow." 2. "Cursor: `SELECT * FROM orders WHERE id > :lastId ORDER BY id LIMIT 20` . DB uses index seek; $O(\log N)$ regardless of depth. But you can't jump to page 50 directly." 3. "OFFSET pros: simple, supports page-N URLs, total count available. Cons: slow at high offsets, inconsistent under concurrent inserts (might skip / duplicate rows)." 4. "Cursor pros: fast at any depth, consistent (stable cursor doesn't shift with inserts). Cons: no random access, more complex client implementation." 5. "Pick: OFFSET for admin UIs with shallow pages. Cursor for mobile feeds, infinite scroll, large datasets."

Edge cases to mention: - For cursor: pick a unique, monotonic column (auto-increment ID, created_at + ID tiebreaker) - For mixed: support both — OFFSET for shallow + cursor (`after_id=X`) for deep - Total count expensive at scale — for cursor APIs, often skip total count entirely

What NOT to say: - "OFFSET is always fine" — kills you at scale - "Cursor is always better" — wrong UX for many use cases

Common follow-up: "How would you implement cursor pagination in Spring Data?"

Q149 · Idempotency for POST

Asked at: Senior backend · **Difficulty:** Medium-Hard · **Topic:** Idempotency

The question: "A client retries a POST due to network failure. How do you ensure they don't get charged twice?"

The 30-second answer: Idempotency keys. Client sends `Idempotency-Key: <unique-uuid>` header on each request. Server stores the result of the first request keyed by this; subsequent requests with the same key return the cached result without re-executing. Standard pattern in payment APIs (Stripe, Razorpay).

Step-by-step thought process: 1. "GET, PUT, DELETE are naturally idempotent (HTTP spec). POST is not — multiple POSTs create multiple resources." 2. "Idempotency-Key header: client generates UUID per logical operation. Sends with every retry of the same operation." 3. "Server: on first request, persist the result keyed by (Idempotency-Key, optionally + account ID). On subsequent requests with same key, return the cached result." 4. "Storage: Redis (TTL-based, fast) or DB table. Key should expire after some window (24 hours, typically — enough for retries, not forever)." 5. "For Razorpay-style payments: idempotency-key on every order/payment creation. Even with multiple retries, only one order is created."

Edge cases to mention: - For in-flight requests with same idempotency key: lock or return \"already processing\" — design choice - Idempotency key should be required for non-idempotent POST endpoints in production payment systems - For UI: generate the key client-side at the START of the action, not per-retry (otherwise retries get different keys)

What NOT to say: - "Just check by amount + user + timestamp" — fragile heuristic - "POST is never idempotent" — only without an idempotency key

Common follow-up: "How would you implement Idempotency-Key in Spring?"

Q150 · HTTP status codes (the real ones to know)

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** HTTP

The question: "What HTTP status code would you return for [various scenarios]?"

The 30-second answer: 200 OK (general success), 201 Created (resource created, include Location header), 204 No Content (success, no body to return — e.g. DELETE), 400 Bad Request (client sent garbage), 401 Unauthorized (not authenticated), 403 Forbidden (authenticated but lacks permission), 404 Not Found, 409 Conflict (state conflict, e.g. version mismatch), 422 Unprocessable Entity (validation failure), 429 Too Many Requests (rate limit), 500 Internal Server Error (server bug).

Step-by-step thought process: 1. "2xx success: 200 OK general; 201 Created (new resource); 202 Accepted (queued); 204 No Content." 2. "3xx redirects: 301 permanent; 302 temporary; 304 not modified (cache validators)." 3. "4xx client errors: 400 (general malformed); 401 (no auth); 403 (auth but forbidden); 404 (resource not found); 409 (state conflict); 422 (validation failure); 429 (rate limit)." 4. "5xx server errors: 500 (general bug); 502 (gateway timeout — proxy can't reach upstream); 503 (unavailable — overload or maintenance); 504 (upstream timed out)." 5. "401 vs 403: 401 = 'I don't know who you are'; 403 = 'I know you but you can't do this'."

Edge cases to mention: - 422 vs 400: 400 for malformed (broken JSON); 422 for valid JSON with semantic errors - 204 returns no body — clients that expect JSON should handle this - 429 should include **Retry-After** header

What NOT to say: - "Always return 200 with an error JSON" — defeats HTTP semantics, breaks middlewares - "Use 500 for any error" — opaque; clients can't react appropriately

Common follow-up: "What's the difference between 401 and 403?"

Q151 · RFC 7807 ProblemDetail error format

Asked at: Senior · **Difficulty:** Easy-Medium · **Topic:** Error response

The question: "What's the standard error response format for REST APIs?"

The 30-second answer: RFC 9457 ProblemDetail (obsoletes RFC 7807, same fields + content type). JSON object with `type` (URI identifying the error type), `title` (short summary), `status` (HTTP code), `detail` (human-readable explanation), `instance` (URI of the specific occurrence). Spring 6 has built-in ProblemDetail class. Standardizes error shape across services.

Step-by-step thought process: 1. "RFC 9457 (obsoletes RFC 7807; same fields + same content type): standard problem-details for HTTP APIs. Content-Type: `application/problem+json`. Older Spring docs reference 7807; the upgraded RFC 9457 didn't change the format." 2. "Required fields: `type` (URI like `https://api.example.com/errors/insufficient-balance`), `title` (`"Insufficient balance"`), `status` (422)." 3. "Optional: `detail` (`"Account 123 has 50 balance, requested 100"`), `instance` (`"/accounts/123/transactions/456"`)." 4. "Custom fields: add app-specific keys (e.g. `errorCode: \"ERR_INSUFFICIENT_BALANCE\"`, `accountId: 123`)." 5. "Spring 6: `ProblemDetail.forStatusAndDetail(HttpStatus.UNPROCESSABLE_ENTITY, \"Insufficient balance\")` — built-in support."

Edge cases to mention: - For backward compat with non-7807 clients: include both standard `error` field + ProblemDetail - For validation errors: extend ProblemDetail with a `fields` array of per-field errors - For internationalization: title/detail localized based on Accept-Language

What NOT to say: - "Just return {error: '...'}" — non-standard; client needs custom parsing - "Use HTML error pages" — REST API should return JSON

Common follow-up: "How would you extend ProblemDetail for field-level validation errors?"

Q152 · REST vs gRPC

Asked at: Architect rounds · **Difficulty:** Medium-Hard · **Topic:** Protocol choice

The question: "When would you pick gRPC over REST?"

The 30-second answer: gRPC for internal service-to-service (binary protocol, smaller payloads, generated clients in any language, streaming). REST for public APIs (debuggable, browser-friendly, every dev knows it). Polyglot mobile + web frontends benefit from gRPC's typed clients; consumer APIs benefit from REST's universal tooling.

Step-by-step thought process: 1. "gRPC: Protocol Buffers (binary), HTTP/2 (multiplexed, streaming), generated typed clients in 10+ languages." 2. "REST: JSON (text), HTTP/1.1 (typically), no client generation, debug with curl." 3. "For internal microservices: gRPC wins. Smaller payloads (5-10x), faster (no JSON parse), strongly-typed (no schema drift), bidirectional streaming." 4. "For public APIs: REST

wins. Browser-friendly, every dev knows it, every tool supports it, JSON is debuggable." 5. "For mobile (Android/iOS): gRPC's typed clients are nice; bandwidth savings matter." 6. "For web frontend: gRPC-Web is a thing but heavier than REST. Often: gRPC internally, REST/GraphQL for browser."

Edge cases to mention: - Streaming: gRPC has 4 modes (unary, server-stream, client-stream, bidi). REST has SSE for server-stream only - For polyglot teams: gRPC's `.proto` schema is a single source of truth - For evolution: gRPC has clear backward-compat rules; REST relies on docs

What NOT to say: - "gRPC is always faster" — for low-volume APIs, REST is fine - "REST is dying" — overstated; still the lingua franca

Common follow-up: "What's gRPC-Web and when would you use it?"

Q153 · API Gateway responsibilities

Asked at: Architect · **Difficulty:** Medium · **Topic:** Architecture

The question: "What does an API Gateway do that your microservices shouldn't?"

The 30-second answer: Auth (JWT validation, OAuth2 introspection), rate limiting, request/response transformation, routing (path-based, header-based), CORS, request logging, retry/circuit-breaker, response caching, request validation (against OpenAPI). Centralized concerns that every service would otherwise duplicate.

Step-by-step thought process: 1. "API Gateway sits at the edge — single entry point for client requests, routes to backend services." 2. "Cross-cutting concerns: auth (decode JWT, populate context), rate limit per user/key, log requests, add tracing headers." 3. "Without a gateway: every service duplicates auth, rate limit, logging — fragile, inconsistent." 4. "Common options: Spring Cloud Gateway (Spring-native), Kong, Istio (service mesh, gateway as a side feature), AWS API Gateway, Cloudflare." 5. "For Spring teams: Spring Cloud Gateway is reactive (Project Reactor), filter-chain extensible, integrates with Spring Cloud (config, discovery)."

Edge cases to mention: - Don't put business logic in the gateway — keep it transport-layer - Service mesh (Istio, Linkerd) handles some gateway responsibilities at the network layer - For high throughput: gateway shouldn't introduce significant latency; benchmark before adopting

What NOT to say: - "Each service handles its own auth" — inconsistent + duplicated - "Just use nginx" — works for routing but no app-aware features

Common follow-up: "How would you implement per-user rate limiting in Spring Cloud Gateway?"

Q154 · OpenAPI / Swagger best practices

Asked at: Common · **Difficulty:** Easy-Medium · **Topic:** API docs

The question: "How would you document a Spring REST API for external consumers?"

The 30-second answer: OpenAPI 3.0 via springdoc (`springdoc-openapi-starter-webmvc-ui`). Auto-generates spec from annotations. Add `@Operation` , `@Schema` , `@Parameter` , `@ApiResponse` for richer docs. Publish spec at `/v3/api-docs` ; Swagger UI at `/swagger-ui.html` . Generate client SDKs from spec.

Step-by-step thought process: 1. "Add `springdoc-openapi-starter-webmvc-ui` (Spring Boot 3). Auto-generates OpenAPI spec from controllers." 2. "Annotate for richer docs: `@Operation(summary = \"Create order\")` , `@Schema(description = \"User's order ID\", example = \"ORD-123\")` , `@Parameter(description = \"Pagination cursor\")` ." 3. "`@ApiResponse(responseCode = \"422\", content = @Content(schema = @Schema(implementation = ProblemDetail.class)))` documents error responses." 4. "Publish: `/v3/api-docs` (JSON spec), `/swagger-ui.html` (interactive UI). Pin spec version for stable URLs." 5. "Generate client SDKs: `openapi-generator-cli` reads the spec, produces typed clients in any language. Public APIs: generate Python/JS SDKs to share with integrators."

Edge cases to mention: - For prod: secure Swagger UI (basic auth or disable entirely), expose only the JSON spec - For multi-version APIs: configure `GroupedOpenApi` beans per version - For private APIs: don't publish the spec externally; use it internally for SDK generation only

What NOT to say: - "Write OpenAPI yaml by hand" — drifts from code; auto-generation is the standard - "Springfox" — deprecated for Spring Boot 3; use springdoc

Common follow-up: "How would you generate a Python client SDK from your Spring API?"

Q155 · CORS preflight + Cache-Control

Asked at: Mid-senior · **Difficulty:** Medium · **Topic:** Web specifics

The question: "How would you reduce CORS preflight overhead for a high-traffic API?"

The 30-second answer: Set `Access-Control-Max-Age` to a long duration (e.g. 86400 seconds — 1 day). Browser caches the preflight result for that long; subsequent requests skip preflight. Configure in Spring: `corsConfig.setMaxAge(86400L)` or per-controller via `@CrossOrigin(maxAge = 86400)` .

Step-by-step thought process: 1. "For each non-simple cross-origin request, browser sends an OPTIONS preflight. Adds latency (extra round trip)." 2. "`Access-Control-Max-Age` header tells the browser to cache the preflight response for N seconds. During that window, no preflight needed." 3. "Spring: in CORS config, `.maxAge(86400L)` (1 day). Or per-controller: `@CrossOrigin(origins = \"...\", maxAge = 86400)` ." 4. "Browser limits: Chrome caps at ~2 hours, Firefox at ~24 hours. Setting 86400 is fine — browser caps it anyway." 5. "For very high traffic: significantly reduces request count (every non-simple cross-origin request × the preflight chunk)."

Edge cases to mention: - Different browsers cap differently — set high, accept the cap - If CORS policy changes, cached preflight may be stale until expiry; deploy + wait - Simple requests (GET, no custom headers) don't need preflight regardless

What NOT to say: - "Disable CORS for performance" — security regression - "Preflight is unavoidable" — preflight caching avoids most of them

Common follow-up: "What makes a CORS request 'simple' vs 'preflighted'?"

Q156 · ETags and conditional requests

Asked at: Senior · **Difficulty:** Medium · **Topic:** HTTP caching

The question: "How would you implement client-side caching for a slow-to-compute GET endpoint?"

The 30-second answer: ETag header. Server includes `ETag: <hash>` in responses. Client sends back `If-None-Match: <hash>` on subsequent requests; if unchanged, server returns 304 Not Modified with empty body — saves bandwidth + compute. Spring's `ShallowEtagHeaderFilter` automates basic cases.

Step-by-step thought process: 1. "ETag = unique identifier for the response's content. Could be a content hash (`MD5(body)`), version number, lastModified timestamp." 2. "Response: `Cache-Control: max-age=0` + `ETag: 'abc123'`". Client caches the response but must revalidate." 3. "On next request: `If-None-Match: 'abc123'` header from client." 4. "Server: compute current ETag. If matches client's, return 304 Not Modified (no body). If different, return 200 with new body + new ETag." 5. "Spring: `ShallowEtagHeaderFilter` computes MD5 of response automatically. For expensive responses, define ETag from the resource's version field (database row's @Version)."

Edge cases to mention: - Last-Modified is similar (timestamp-based; sends `If-Modified-Since`); ETag is more precise - Weak ETags (`W/'abc123'`) for semantically-equal-but-byte-different responses - For high-throughput, computing MD5 on every response is wasted CPU — use the resource's @Version

What NOT to say: - "Use Cache-Control max-age only" — doesn't help once cache expires - "ETag doesn't work with dynamic content" — wrong; computed from current state

Common follow-up: "How would you generate an ETag from a JPA entity?"

Q157 · API rate limiting strategies

Asked at: Senior backend · **Difficulty:** Medium-Hard · **Topic:** Rate limiting

The question: "How would you implement rate limiting for an API?"

The 30-second answer: Three common algorithms. **Token bucket:** refills at a rate; each request consumes a token. **Fixed window:** count requests per window (per-minute, per-hour). **Sliding window:** smooths the fixed-window edge cases. For distributed apps: store counters in Redis with TTL. Libraries: bucket4j (Spring), Redis Cell, Cloudflare/API Gateway built-ins.

Step-by-step thought process: 1. "Token bucket: bucket fills at rate R per second, max capacity C . Each request needs a token. If empty, reject (429). Allows bursts up to C ." 2. "Fixed window: count requests in each N -second window. Reset at window boundary. Simple but allows $2\times$ rate at the boundary." 3. "Sliding window: count requests in the rolling N -second window. Smoother than fixed, more expensive (track per-request timestamps)." 4. "Distributed: each app instance can't keep local count alone. Use Redis: `INCR + EXPIRE` for fixed window, atomic Lua script for token bucket." 5. "Per dimension: per IP, per user, per API key, per endpoint, or combinations. Different limits for different tiers (free vs paid)."

Edge cases to mention: - Return `Retry-After` header in 429 — tells client when to retry - For SaaS: per-account rate limits stored in DB, enforced via Redis counters - For abuse prevention: add per-IP limit on top of per-account - For Spring: `bucket4j-spring-boot-starter` handles common cases

What NOT to say: - "Use in-memory rate limit" — broken across instances - "Just rely on the load balancer" — most LBs don't do per-user limiting

Common follow-up: "How would you implement per-user rate limiting in Spring with Redis?"

Q158 · Webhook design (server-to-server callbacks)

Asked at: Senior · **Difficulty:** Medium-Hard · **Topic:** Webhooks

The question: "You're designing a payment system that calls back to merchants when a payment status changes. What does good webhook design look like?"

The 30-second answer: Signed payloads (HMAC), retries with exponential backoff, deduplication via event ID, replayable from a dashboard, separate webhook log per merchant, version the payload format. Acknowledge with 2xx; retry on non-2xx. Treat as best-effort — merchant downtime shouldn't crash your delivery system.

Step-by-step thought process: 1. "Sign payloads: `X-Signature: HMAC-SHA256(body, secret)`. Merchant verifies signature → confirms it's really you. Prevents spoofing." 2. "Retry on failure: exponential backoff (5s, 1m, 10m, 1h, 6h, 24h). Cap retries (5-10 attempts). Give up after some window." 3. "Event ID: include a unique ID in the payload. Merchant uses to dedupe — they might receive the same event twice (retry race)." 4. "Async delivery: don't block the original request on webhook delivery. Queue events; background worker delivers + retries." 5. "Dashboard: merchants can see webhook history, replay failed deliveries, rotate secret. Critical for debugging." 6. "Payload versioning: include version field; bump major version for breaking changes; support older versions for a deprecation window."

Edge cases to mention: - 5-10 second timeout on outbound webhook calls — don't wait long for slow merchants - For Razorpay-style: webhook signatures are mandatory; replay attacks blocked via timestamp + signature - For at-least-once delivery: merchants must build idempotent webhook handlers

What NOT to say: - "Send and forget" — payment events need delivery guarantees - "Retry forever" — eventually give up; alert merchant

Common follow-up: "How would a merchant verify your webhook signature?"

Q159 · HTTP/1.1 vs HTTP/2 vs HTTP/3

Asked at: Senior · **Difficulty:** Medium · **Topic:** HTTP versions

The question: "Should you care about HTTP/2 for your Spring app?"

The 30-second answer: HTTP/2 = multiplexing (multiple requests per connection), header compression, server push. HTTP/3 = HTTP/2 over QUIC (UDP-based, no head-of-line blocking, faster connection setup). For browser-facing APIs behind a CDN: CDN handles it; doesn't matter for Spring. For service-to-service: gRPC uses HTTP/2; gRPC speed comes from that.

Step-by-step thought process: 1. "HTTP/1.1: one request per connection. Browsers open 6 connections per host to parallelize." 2. "HTTP/2: multiplexing — many requests over one connection. Header compression. Useful for sites with many small assets (CSS, JS, images)." 3. "HTTP/3: HTTP over QUIC (UDP). Faster connection setup (0-RTT), no head-of-line blocking. Browsers shipping it since 2020." 4. "For Spring backend: doesn't really matter — your CDN / LB terminates the protocol and re-issues HTTP/1.1 internally." 5. "Where it matters: gRPC (uses HTTP/2 multiplexing as core feature), browser → CDN → origin path (CDN terminates)."

Edge cases to mention: - Tomcat supports HTTP/2 since 9.0; configure with `server.http2.enabled=true` - For mTLS-heavy environments: HTTP/2 reuses connections, fewer TLS handshakes — measurable improvement - HTTP/3 needs UDP open — some firewalls block UDP, falls back to HTTP/2

What NOT to say: - "HTTP/2 is irrelevant for APIs" — gRPC alone makes it relevant - "Switch to HTTP/3 immediately" — for backend services, ROI is tiny

Common follow-up: "Why does HTTP/2 matter for gRPC?"

Q160 · API lifecycle and deprecation

Asked at: Architect · **Difficulty:** Medium · **Topic:** API governance

The question: "How would you deprecate an old API version cleanly?"

The 30-second answer: Announce deprecation publicly (changelog, email to integrators). Add `Deprecation: true` + `Sunset: <date>` HTTP headers on every response from the old version. Communicate timeline (3-12 months typically). Track usage to identify holdouts. Reach out personally to top consumers. Remove only after sunset date.

Step-by-step thought process: 1. "Public announcement: changelog post, email to all API key holders, dashboard banner." 2. "Response headers: `Deprecation: @1735689600` (Unix timestamp) + `Sunset: Sun, 01 Jan 2027 00:00:00 GMT`. Standard headers (IETF RFC 8594, RFC 7234)." 3. "Sunset date: typically 6-12 months out. Long enough for clients to migrate; short enough that you can actually retire." 4. "Track usage: log requests to the old version with API key. Identify top consumers, reach out individually." 5. "After sunset: remove the endpoints. Return 410 Gone with a link to migration docs."

Edge cases to mention: - For high-value integrators: dedicated migration support, more lead time - For free-tier consumers: less leeway; deprecate aggressively if needed - For Razorpay-style payments: regulatory considerations may extend sunset windows

What NOT to say: - "Just delete the old version" — breaks every integration - "Keep all versions forever" — engineering debt becomes unmanageable

Common follow-up: "What's the Sunset HTTP header and how would you use it in Spring?"
